ما هو Event

الخطوة الأولى: لماذا ظهرت فكرة الـ Event أصلًا؟

تخيل أنك كتبت هذا البرنامج:

```
------------------------
|                      |
|     [ Login ]        |
|                      |
------------------------
```

وفيه زر واحد فقط.

وشغلت البرنامج.

السؤال:

هل البرنامج هيضغط على الزر بنفسه؟

الإجابة:

❌ لا.

السؤال الثاني:

هل البرنامج يعرف أنك تريد الضغط على الزر؟

الإجابة:

❌ لا.

إذن البرنامج الآن ماذا يفعل؟

الإجابة:

هو فقط...

```
ينتظر...
```

الخطوة الثانية: من الذي يحرك البرنامج؟

في برامج الـ Console كنت تكتب:

```C++
cout << "Enter Number:";
cin >> x;
```

البرنامج هو الذي يقول للمستخدم:

> اكتب رقمًا.

ثم ينتظر.

لكن في Windows Forms الوضع مختلف.

البرنامج لا يقول:

> اضغط الزر الآن.

بل المستخدم هو الذي يقرر.

قد يضغط الآن.

وقد يضغط بعد دقيقة.

وقد لا يضغط أبدًا.

إذن من الذي يقود البرنامج؟

الإجابة:



وليس البرنامج.

وهذه أول فكرة مهمة جدًا.

الخطوة الثالثة: ماذا يمكن أن يفعل المستخدم؟

دعنا نعد الأشياء.

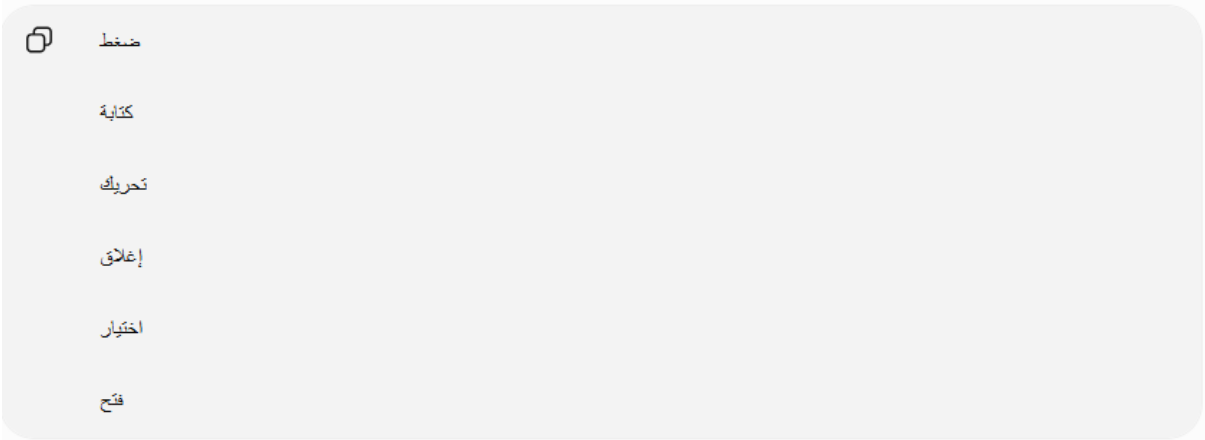
يمكن أن:

- يضغط Button.
- يكتب داخل TextBox.
- يحرك الماوس.
- يغلق النافذة.
- يغير حجم النافذة.
- يختار عنصرًا من ComboBox.
- يضغط مفتاحًا من الكيبورد.

كل هذه أشياء يمكن أن تحدث.

الخطوة الرابعة: ما القاسم المشترك؟

انظر إلى هذه القائمة.



ما المشترك بينهم؟

كلهم...

حدثوا أثناء تشغيل البرنامج.

وهنا جاءت Microsoft وقالت:

> أي شيء يحدث أثناء تشغيل البرنامج...
>
> سنسميه **Event**.

إذن

```
Event

=

حدث يقع أثناء تشغيل البرنامج.
```

وليس أكثر من ذلك.

الخطوة الخامسة: هل الـ Event هو الضغط؟

هنا يقع معظم المبتدئين في الخطأ.

الضغط ليس اسمه Event.

بل الضغط **هو نوع من أنواع الـ Events**.

مثل:

```
Click Event
```

وكذلك

```
TextChanged Event
```

وكذلك

```
Load Event
```

وكذلك

```
MouseMove Event
```

إذن الـ Click ليس الـ Event الوحيد.

هو مجرد مثال.

الخطوة السادسة: لماذا يحتاج البرنامج إلى معرفة أن حدثًا وقع؟

تخيل أن المستخدم ضغط على الزر.

إذا لم يعرف البرنامج ذلك...

فهل سيفعل شيئًا؟

الإجابة:

❌ لا.

إذن يجب أن توجد طريقة يقول بها Windows للبرنامج:

> انتبه...
>
> المستخدم ضغط على الزر.

هذا الإشعار اسمه:

```
Click Event
```

الخطوة السابعة: تخيل Windows كأنه موظف استقبال

تخيل أن عندك شركة.

أنت المدير.

لكن لا تراقب الباب.

يوجد موظف استقبال.

كلما جاء شخص...

الموظف يقول لك:

> فيه عميل وصل.

هو لم يحل المشكلة.

هو فقط أخبرك أن شيئًا حدث.

هذا بالضبط ما يفعله Windows.

هو يراقب كل شيء.

ثم يقول لبرنامجك:

> المستخدم ضغط.

أو

> المستخدم كتب.

أو

> المستخدم أغلق النافذة.

هذه كلها إشعارات.

وهذه الإشعارات اسمها:

```
Events
```

الخطوة الثامنة: هل الـ Event ينفذ الكود؟

الإجابة:

❌ لا.

وهذه نقطة مهمة جدًا.

الـ Event لا يحتوي على الكود.

هو فقط يقول:

> حدث شيء.

الخطوة التاسعة: من الذي ينفذ الكود إذن؟

أنت.

أنت تكتب:

```C#
private void button1_Click(...)
{
    MessageBox.Show("Hello");
}
```

هذا الكود يسمى:

```
Event Handler
```

وترجمته:

> **معالج الحدث**

أي:

> الكود الذي سيعمل عندما يحدث Event معين.

الخطوة العاشرة: لماذا اسمه Handler؟

كلمة

```
Handle
```

تعني:

> يتعامل مع.

إذن

```
Event Handler
```

تعني:

> الكود الذي يتعامل مع الحدث.

الخطوة الحادية عشرة: ماذا يحدث لحظة الضغط؟

دعنا نبطئ الزمن.

أنت ضغطت الزر.

↓

Windows لاحظ الضغط.

↓

Windows أنشأ

```
Click Event
```

↓

Windows سأل:

> هل لهذا الزر كود مرتبط بحدث Click؟

↓

إذا وجد...

↓

نفذ الكود.

يعني التسلسل الحقيقي هو:

```
المستخدم ضغط
        |
        ▼
Windows اكتشف الضغط
        |
        ▼
أنشأ Click Event
        |
        ▼
بحث عن Event Handler
        |
        ▼
وجد الكود
        |
        ▼
نفذ الكود
```

الخطوة الثانية عشرة: هل كل Control له Events؟

الإجابة:

نعم.

لكن...

ليس كلهم لهم **نفس** الـ Events.

لماذا؟

لأن وظيفة كل Control مختلفة.

مثلاً

Button

وظيفته:

```
الضغط عليه.
```

إذن من الطبيعي أن يكون أهم Event عنده:

```
Click
```

TextBox

وظيفته:

```
كتابة النص.
```

إذن من الطبيعي أن يكون عنده:

```
TextChanged
```

Form

وظيفتها:

```
الفتح والإغلاق.
```

إذن من الطبيعي أن يكون عندها:

```
Load

Closing
```

الخطوة الثالثة عشرة: هل توجد Events مشتركة؟

نعم.

مثلاً:

```
Click
```

يمكن أن يوجد في:

- Button
- PictureBox
- Label

لأن الثلاثة يمكن الضغط عليهم.

لكن

```
TextChanged
```

لن تجده في Button.

لأن Button لا يكتب المستخدم بداخله.

الفكرة النهائية التي أريد أن ترسخ في ذهنك

تخيل البرنامج كأنه إنسان.

وأنت تتحدث معه.

كلما حدث شيء...

Windows يهمس في أذنه.

ويقول:

```
ضغط المستخدم.

كتب المستخدم.

حرك الماوس.

فتح الفورم.

أغلق الفورم.
```

البرنامج لا يعرف هذه الأشياء بنفسه.

بل Windows يخبره بها.

وهذه الرسائل اسمها:

```
Events
```

ثم البرنامج ينظر إليك ويقول:

> "كنتَ قد أخبرتني سابقًا أنه إذا حدث Click، أنفذ هذا الكود."

فيذهب وينفذه.

الخريطة الذهنية التي أحب أن تحفظها

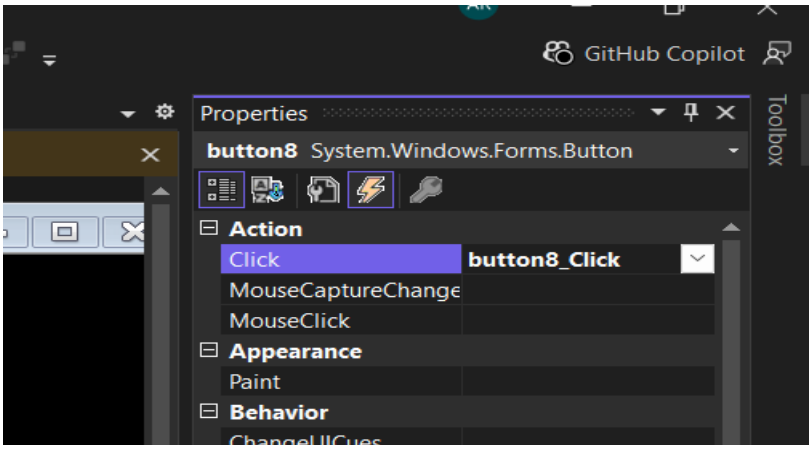


الخلاصة الذهبية

احفظ الجملة دي لأنها هتريحك جدًا:

> **الـ Event ليس هو الكود، وليس هو الضغط نفسه، وإنما هو إشعار (Notification) يرسله Windows إلى البرنامج ليخبره أن شيئًا معينًا قد حدث أثناء التشغيل. أما الـ Event Handler فهو الكود الذي يكتبه المبرمج ليتعامل مع هذا الإشعار.**

اضافات



11) ليه اسمها كده؟

لأن Visual Studio بيولد اسم افتراضي بالشكل ده:

```
اسم الكونترول + اسم الحدث
```

يعني

```
button8
```

•

```
MouseLeave
```

=

```
button8_MouseLeave
```

ولو كان Click

```
button8_Click
```

ولو كان MouseMove

```
button8_MouseMove
```

ولو كان Paint

```
button8_Paint
```

وهكذا.

الملخص الصغير جدًا:

- **Event (الحدث)** = حاجة حصلت أثناء تشغيل البرنامج.
  - مثال: Click - MouseLeave - MouseEnter - KeyDown.
- **Event Handler (دالة معالجة الحدث)** = الدالة اللي بتتنفذ لما الحدث يحصل.

مثال:

```C#
private void button8_MouseLeave(object sender, EventArgs e)
{
    MessageBox.Show("Mouse Left");
}
```

- **MouseLeave** ← ده اسم الحدث.
- **button8_MouseLeave(...)** ← دي دالة معالجة الحدث (Event Handler).

التسلسل كامل:

```
أنت حركت الماوس خارج الزر
        ↓
حصل Event اسمه MouseLeave
        ↓
بلغ البرنامج Windows
        ↓
البرنامج لقى إن الحدث ده مربوط بالدالة:
button8_MouseLeave(...)
        ↓
استدعى الدالة ونفذ الكود اللي جواها
```

أهم نقطة تحفظها:

❌ اسم الدالة ≠ الحدث.

✅ الحدث هو:

```
Click
MouseLeave
MouseEnter
KeyDown
```

✅ أما الدالة:

```
button1_Click
button8_MouseLeave
textBox1_TextChanged
```

دي مجرد **Event Handler**، واسمها اختاره Visual Studio تلقائيًا عشان يبقى واضح هي بتعالج حدث إيه. وكان ممكن تسميها أي اسم تاني، بشرط تفضل مربوطة بنفس الحدث.

يعني الحدث هو **السبب**، والدالة هي **رد الفعل** على الحدث.

pizza

🎯 1. فلسفة التمييز بين `Update` و `Calculate`

الأستاذ مش بيختار الأسماء عشوائياً، هو بيمشي على قاعدة هندسية شهيرة اسمها (Separation of Concerns) يعني فصل المهام، والأسماء هنا بتعبر عن وظيفتين مختلفتين تماماً:

- أي دالة تبدأ بـ `Update...` : وظيفتها الأساسية هي **"التعامل مع الشاشة والمستخدم" (UI Interaction)**. يعني بتاخد قيمة معينة وتروح تطبعها جوه `Label` أو تغير حالة أداة على الشاشة عشان الزبون يشوفها بعينه.
- أي دالة تبدأ بـ `Calculate...` : وظيفتها **"رياضية بحتة" (Business Logic)**. الدالة دي "عمياء" ميعرفش شكل الشاشة إيه، هي كل شغلتها تاخد الأرقام، تضرب وتجمع، وترجع ناتج رقمي (`return float`) للجهة اللي طلبتها بدون ما تدخل في تفاصيل الشاشة.

🔄 2. تتبع التسلسل العبقري اللي أنت شرحته

التسلسل اللي في دماغك هو بالظبط اللي بيحصل في الذاكرة:

1. **الزبون يضغط على الحجم الوسط (`rbMedium`):**
   البرنامج يستدعي دالة `UpdateSize()` . من اسمها: "يا شاشة حدّثي نص الحجم للزبون واكتبه له `Medium` ".
2. **جوه `UpdateSize()` ، أول حاجة بتتعمل هي استدعاء `UpdateTotalPrice()` :**
   البرنامج بيقول: "بما إن الحجم اتغير، يلا يا شاشة روحي حدّثي الـ `Label` بتاع السعر الإجمالي للزبون".
3. **جوه `UpdateTotalPrice()` ، بيتم استدعاء الدالة الحسابية `CalculateTotalPrice()` :**
   دالة التحديث بتقول: "أنا عشان أطبع السعر على الشاشة، لازم الأول أعرف السعر كام بالظبط، يا دالة **الحساب** اجمعي لي الحسبة الحالية وهاتي لي الرقم".
4. **دالة `CalculateTotalPrice()` بتشغل الثلاثة الفارس بتوعها:**
   تروح تستدعي الـ 3 دالات الحسابية الفرعية:
   - `GetSelectedSizePrice()` ← تحسب سعر الحجم الحالي وتجيبه.
   - `CalculateToppingsPrice()` ← تصفر العداد وتحسب سعر الإضافات الحالية وتجيبه.
   - `GetSelectedCrutPrice()` ← تحسب سعر العجينة وتجيبه.

وبعد ما تجمعهم، الرقم يرجع لدالة الـ `Update` اللي بتاخده وتطبعه فوراً في الـ `Label` قدام الزبون مع علامة الـ `$` .

💡 التعقيب النهائي: ليه الأسلوب ده عبقري؟

تخيل لو الأستاذ مكنش عمل التسلسل ده، وكان كتب كود الحساب وكود الطباعة كله سايح في مكان واحد جوه ضغطة الزرار! كان الكود هيبقى ضخم، ومكرر 20 مرة مع كل أداة، ولو حبيت تعدل شكل العملة من `$` لـ `LE` مثلاً، كنت هتلف على الكود كله تعدله.

لكن بالطريقة دي، لو حبيت تعدل طريقة الحساب بتروح لدوال الـ `Calculate` ، ولو حبيت تعدل شكل العرض على الشاشة بتروح لدوال الـ `Update` .

1. دوال الجروب بوكس (تحديث جزئي ومحدد بالحدث):

الدوال دي مثل `UpdateSize()` أو `UpdateToppings()` هي دوال **"مشروطة بالضغط" (Event-Driven)**. يعني لا تعمل إلا إذا قام الزبون بضغط زرار معين أو اختيار محدد في منطقتها فقط.

- **طريقة عملها:** لما الزبون يضغط على حجم معين، الدالة دي بتشتغل عشان تغير النص الخاص بالحجم على الشاشة، وتستدعي معها دالة السعر الشاملة لتحديث الحسبة بناءً على هذا التغيير البسيط.

2. دالة `UpdateOrderSummary` (تحديث كلي شامل عند الحاجة):

الدالة دي هي **"المظلة الكبيرة"**، ولا تنتظر ضغطة زرار محدد من العميل؛ بل يتم استدعاؤها من المبرمج نفسه (جوه الكود) عندما يريد تحديث **كل شيء دفعة واحدة بالكامل** بناءً على جميع المدخلات الحالية في البرنامج.

- **طريقة عملها:** تذهب وتستدعي الـ 5 دوال الفرعية وراء بعضها، فتقوم بقراءة حالة الحجم، وحالة العجينة، وحالة الإضافات، وحالة مكان الأكل، وتحديث السعر الإجمالي، كل هذا في نفس اللحظة.

📌 الخلاصة اللي تؤكد صحة كلامك:

- **الدوال الفرعية:** تحديث "موضعي" ناتج عن حركة أو ضغطة العميل الحالية على عنصر محدد.
- **الدالة الكبيرة (`UpdateOrderSummary`):** تحديث "إجمالي شامل" نستخدمه لتأسيس الشاشة في البداية (`Load`)، أو إذا أردنا إعادة قراءة وتحديث الفاتورة بأكملها في خطوة واحدة دون انتظار ضغط العميل على كل عنصر على حدة.

بعض الامثله المستخدمه لهذه الداله

يا عيني عليك يا علي! سؤالك ده هو أول خطوة في تصميم الأنظمة الوظيفية الكبيرة. طالما بقيت فاهم إن الدالة دي بتعمل **"تحديث كلي شامل بناءً على الوضع الحالي للبرنامج"**، فإحنا بنحتاجها في كذا مكان استراتيجي في البرامج الحقيقية.

تعال أقول لك على أشهر الأماكن اللي بنستخدم فيها الدالة دي في الأنظمة الكبيرة:

1. بعد قفل شاشة التعديل (Edit Mode)

تخيل لو الزبون عمل طلب وخلص، وبعدين داس على زرار "تعديل الطلب" ففتحت له شاشة فرعية تانية اختار منها حاجات جديدة (مثلاً غير الحجم من وسط لكبير وضاف مشروم) وداس "حفظ والتزام".
أول ما الشاشة الفرعية تقفل ونرجع للشاشة الرئيسية، بننادي فوراً دالة `()UpdateOrderSummary` عشان تقرأ التعديلات الجديدة وتحدث الفاتورة الشاشة كلها مرة واحدة قدام الزبون بالبيانات الجديدة.

2. عند تطبيق كود خصم أو كوبون (Promo Code)

لو البرنامج بتاعك فيه خانة يكتب فيها العميل كود خصم (مثلاً: `PIZZA20` عشان يديله خصم 20%). أول ما العميل يكتب الكود ويدوس "تطبيق"، إحنا هنا محتاجين نعيد حساب السعر الإجمالي، وممكن كمان نغير نصوص في الملخص عشان تظهر نسبة الخصم.
فبننادي الدالة الكبيرة عشان تسمع في الشاشة كلها وتحدث الحسبة بالأسعار الجديدة بعد الخصم.

3. عند تحميل طلب قديم من قاعدة البيانات (Load Saved Order)

تخيل لو زبون مسجل في البرنامج، ودخل على "طلباتي السابقة" واختار طلب كان طالبه الأسبوع اللي فات وداس "إعادة الطلب".
البرنامج هيروح يجيب بيانات الحجم والعجينة والإضافات من قاعدة البيانات ويحطها في المتغيرات، وبعدين ننادي دالة `()UpdateOrderSummary` عشان ترص البيانات دي وتحدث الشاشة الفاتورة كلها مرة واحدة بالطلب القديم ده.

4. في برامج الكاشير: عند تغيير "نوع العميل" (Customer Type)

في بعض الأنظمة، لو اللي بيشتري "طالب جامعة" بياخد خصم تلقائي، ولو "عميل عادي" بياخد السعر الطبيعي. أول ما الكاشير يغير نوع العميل من القائمة، بننادي الدالة الكبيرة عشان تعيد قراءة كل الاختيارات اللي على الشاشة وتحسب السعر الإجمالي بناءً على تسعيرة الفئة الجديدة وتحدث الشاشة كلها في ثانية.

💡 القاعدة الذهبية اللي تمشي بيها:

أي وقت في البرنامج يحصل فيه **"تغيير جماعي"** في البيانات (مش مجرد ضغطة زرار مفرد من العميل)، أو البيانات تجيلك جاهزة من برة (قاعدة بيانات، شاشة تانية، كود خصم)، على طول بتجري تنادي الدالة الكبيرة `()UpdateOrderSummary` عشان تضمن إن الشاشة كلها اتحدثت ومفيش بيان قديم معلق.

More about TextBox

الفيجوال استوديو في شاشة الخصائص (Properties Window) مش بيكتب لك كلمة `Inherit` كخيار صريح ومستقل برة القائمة؛ هو **بيوريك النتيجة الفعلية الموروثة** عشان يسهل عليك!

إزاي الكلام ده؟

لما بتنزل التكست بوكس لأول مرة، الفيجوال استوديو بيروح يبص على الفورم، يلاقيها `No` . فبيكتب لك جوة مربع الخصائص بتاع التكست بوكس كلمة `No` (بس من غير ما يحط جنبها علامة تعديل، يعني ده السلوك الموروث).

ولما أنت بتغير الفورم لـ `Yes` ، التكست بوكس بيورث القيمة دي فوراً، فالفيجوال استوديو بروح يغير الكتابة اللي قدامك في شاشة الخصائص ويخليها `Yes` عشان يقول لك: "يا علي، التكست بوكس ده شغال حالياً يمين لشمال".

🔍 الدليل القاطع (تحدي عملي ليك جوة الـ Visual Studio):

عشان تتأكد إنهم لسه على وضع الوراثة ومحدش غيرهم مخصوص، افتح القائمة المنسدلة (Drop-down) بتاعة الخاصية `RightToLeft` لأي تكست بوكس فيهم، هتلاقي الخيارات المتاحة جواها كدة بالظبط:

1. `Yes`
2. `No`
3. `Inherit` (هتلاقيها موجودة تحتهم في القائمة!)

طالما أنت ملقطتش بإيدك كلمة `Yes` أو `No` من القائمة دي وضغطت عليها، فالتكست بوكس بيفضل على وضع الوراثة التلقائي، والقيمة اللي بتشوفها مكتوبة برة (`Yes` أو `No`) بتتغير لوحدها حسب شقلبة الفورم الكبيرة.

ليه مش بتظهر كلمه انهرت فى اول مره

أنا فاهم وجهة نظرك جداً يا علي، ومن حقك تفكر كدة! منطقياً كلامك صح 100%: طالما الأداة بتورث، فالمفروض يكتب لي اسم الحالة وهي `Inherit` عشان أكون عارف.

لكن مهندسي شركة ميكروسوفت لما جفوا يصمموا الفيجوال استوديو، فكروا بطريقة تانية تخص **"تجربة المستخدم" (User Experience)** وسهولة التصميم. تعال أقول لك هما ليه فضّلوا يكتبوا القيمة (`No` أو `Yes`) بدل ما يكتبوا كلمة `Inherit` :

1. عشان ما تضطرش تفتح قائمة الخصائص لكل أداة

تخيل لو أنت بتصمم شاشة معقدة فيها 50 أداة (أزرار، تكست بوكس، ليبل، قوائم). لو الفيجوال استوديو كتب قدامهم كلهم كلمة `Inherit` ، وجيت أنت تبص بصه سريعة على جدول الخصائص عشان تتأكد إن الشاشة مظبوطة عربي ولا إنجليزي:

- **لو مكتوب `Inherit` :** عينك هتشوف كلمة `Inherit` متكررة 50 مرة! هتعرف منين هما حالياً قالبين عربي ولا إنجليزي؟ هتضطر تروح تبص على الفورم الكبيرة عشان تفتكر، أو تفتح البرنامج وتجربه عشان تشوف النتيجة.
- **الوضع الحالي (الذكي):** لما بيبصوا على الجدول ويلاقوا مكتوب قدام الـ 50 أداة كلمة `No` بالخط العادي، بيفهموا في ثانية واحدة وبمجرد النظر إن الشاشة كلها حالياً شغالة من الشمال لليمين.

2. تسهيل القراءة السريعة (What You See Is What You Get)

ميكروسوفت بتمشي بمبدأ: **"وريّ المبرمج الأداة بتعمل إيه حالياً علطول".**
بما إن التكست بوكس حالياً بيكتب من الشمال لليمين (بسبب الوراثة)، فالأسهل لعينك وعقلك وأنت بتصمم إنك تقرأ كلمة `No` (يعني مش مقلوب)، بدل ما تقرأ كلمة `Inherit` وتقعد تترجم في مخك: "وراثة.. يعني واخد من الفورم.. والفورم بـ No.. يبقى التكست بوكس بـ No". هما اختصروا عليك اللفة الذهنية دي كلها!

📌 الخلاصة اللي تريح بالك:

الفيجوال استوديو اعتبر إن كلمة `Inherit` دي **"طريقة العمل في الخلفية"**، لكن اللي يهمك أنت كمصمم هو **"النتيجة الحالية على الشاشة".**

وعشان يجمع بين الميزتين:

1. بيكتب لك النتيجة الحالية (`No` أو `Yes`).
2. وفي نفس الوقت بيسيب الخط **خفيف ومش عريض** عشان يديهك داخلياً إن القيمة دي مش ثابتة وممكن تتغير لو عبرت الفورم الكبيرة.

هو ده اللي بيحصل بالظبط في كواليس الفيجوال استوديو: **داخلياً هو شغال وراثة ومربوط بالفورم، بس برة بيوريك "النتيجة النهائية" اللي الأداة شغالة بيها حالياً عشان عينك ترتاح وماتقعدش تخمن.**

تخيل الفيجوال استوديو بيكلمك ويقول لك:

"يا علي، أنا عارف إن التكست بوكس ده عامل وراثة ومتبع الفورم، وبما إن الفورم دلوقتي `Yes` ، فالتكست بوكس شغال `Yes` .. فبدل ما أكتب لك كلمة وراثة وأحيرك، أنا هكتب لك النتيجة علطول اللي هي `Yes` ".

وطالما الكلمة مكتوبة بخط عادي (مش بخط عريض غامق `Bold`)، ده معناه إن القيمة دي موروثة تلقائياً وأنت مغيرتهاش بإيدك. أول ما بتغير ها أنت بإيدك وتثبتها على `Yes` أو `No` ، الخط بيبقى عريض، والوراثة بتتكسر فوراً!

مش فاهم ليه سموها Word Wrap بالذات؟ وإيه علاقة كلمة "التفاف أو تغليف" بنزول النص لسطر جديد؟ عاوز مثال من واقع الحياة يوضح سبب التسمية دي".

أصل كلمة **Wrap** تاريخياً جاي من فكرة **"الشيء الكروي أو الدائري الذي يدور حول نفسه ليعود من حيث بدأ".**

إليك 3 أسباب واضحة جداً تربط بين كلمة "التفاف/تغليف" وبين اللي بيحصل في الشاشة:

1. مفهوم "الدوران والالتفاف" (The Loop/U-Turn)

تخيل لو أنت سايق عربية في شارع سد، ووصلت لآخر الشارع، بتعمل إيه؟ بتلف وترجع في الاتجاه المعاكس. العملية دي بالإنجليزية اسمها **Wrap around**.

في الكمبيوتر، المؤشر (الماوس) بيمشي في السطر من الشمال لليمين. أول ما يوصل للحافة (الحيطة)، الكود بيجبره يعمل "دوران" أو **Wrap** (يلف ويرجع تاني يبدأ من أقصى الشمال في السطر الجديد). فكلمة **Wrap** هنا بتوصف حركة الدوران والرجوع لنقطة البداية.

2. التسمية مأخوذة من "الآلة الكاتبة القديمة" (Typewriter)

البرمجة ورثت مصطلحات كتير من الآلة الكاتبة القديمة. في الآلة الكاتبة، الورقة كانت بتكون ملفوفة حول **"أسطوانة دائرية" (Roller)**.

لما الكاتب كان بيوصل لآخر السطر، الأسطوانة كانت بتلف (تتغلف/تدور) عشان تطلع الورقة لفوق وبتحرك المؤشر لأول السطر الجديد. الحركة الدائرية للأسطوانة دي اسمها اللغوي **Wrapping**. لما نقلوا المفهوم للكمبيوتر، سموا نزول السطر التلقائي **Word Wrap** تيمناً بحركة التفاف الأسطوانة الدائرية دي.

3. فكرة "الاحتواء والتغليف" (Containment)

في اللغة، لما بنقول بنغلف (Wrap) حاجة، معناه إننا بنحجمها وبنمنعها تخرج برة الغلاف.

- النص لو سيبته من غير الخاصية دي، هيكون "سايب وممتد" ويهرب برة حدود المربع.
- تشغيل الخاصية بيعمل **"تغليف" (Wrapping)** للنص، كأنك حطيت الكلام جوة كرتونة أو علبة مقفولة، فمجبور ينطوي وينزل تحت بعضه عشان يفضل "جوة الغلاف" (الذي هو حدود التكست بوكس) وما يخرجش برة.

الخلاصة:

السبب في التسمية هو **"وصف حركة النص المستديرة"**؛ النص بيمشي في خط مستقيم، وأول ما يخلص السطر **بيلتف (Wrap)** حول الحافة وينزل يبدأ سطر جديد، كأنه شريط بيلف حول الأسطوانة.

أقرب وأجمل مثال من واقع الحياة يوضح لك سبب التسمية (الالتفاف والتغليف) بالملي هو **"بكرة المناديل" (أو بكرة السلوفان/الورق).**

تخيل لو عندك شريط طويل جداً من الورق مكتوب عليه كلام:

1. لو الخاصية مقفولة (`WordWrap = False`):

ده معناه إن الشريط مفرود بالكامل على الأرض بطوله الحقيقي وممتد من أوضتك لحد الصالة برة. الشريط هنا **مش ملفوف**، واخد مساحته الحقيقية كلها برة وممتد في خط مستقيم طويل لا ينتهي.

2. لو الخاصية شغالة (`WordWrap = True`):

أنت قررت تلم الشريط الطويل ده وتعمل له **"تغليف والتفاف" (Wrapping)** حول بكرة كرتون صغيرة.

- **إيه اللي حصل للشريط؟** الشريط الطويل جداً انطوى فوق بعضه ودار ولف حولين نفسه، فبقى كله محكوم ومحفوظ جوة حجم البكرة الصغيرة اللي في إيدك ومخرجش برة حدودها!

نرجع بقى للشاشة والتكست بوكس:

الكلام اللي العميل بيكتبه هو **"الشريط الورقي الطويل"**، والتكست بوكس هو **"البكرة الصغيرة"**.

- لما بتفعل الـ `WordWrap` ، أنت بتقول للكمبيوتر: **"لِف لي النص ده واطويه فوق بعضه (سطر تحت سطر) زي بكرة المناديل بالظبط، عشان يفضل كله ملفوف ومحوي جوة حدود المربع الصغير وما يهربش برة الشاشة".**

علشان كده سموها **Wrap** (لف وتغليف)، لأن النص بيلف وينطوي فوق بعضه عشان يتحجم جوة مساحة صغيرة بدل ما يفضل مفرود برة!

"هل لازم أفعّل خاصية الـ Multiline عشان خاصية الـ WordWrap تشتغل؟ وليه؟"

نعم، إجباري ولابد وحتماً إن خاصية الـ `Multiline` تكون شغالة (`True`) عشان خاصية الـ `WordWrap` تبدأ تشتغل وتظهر وظيفتها.

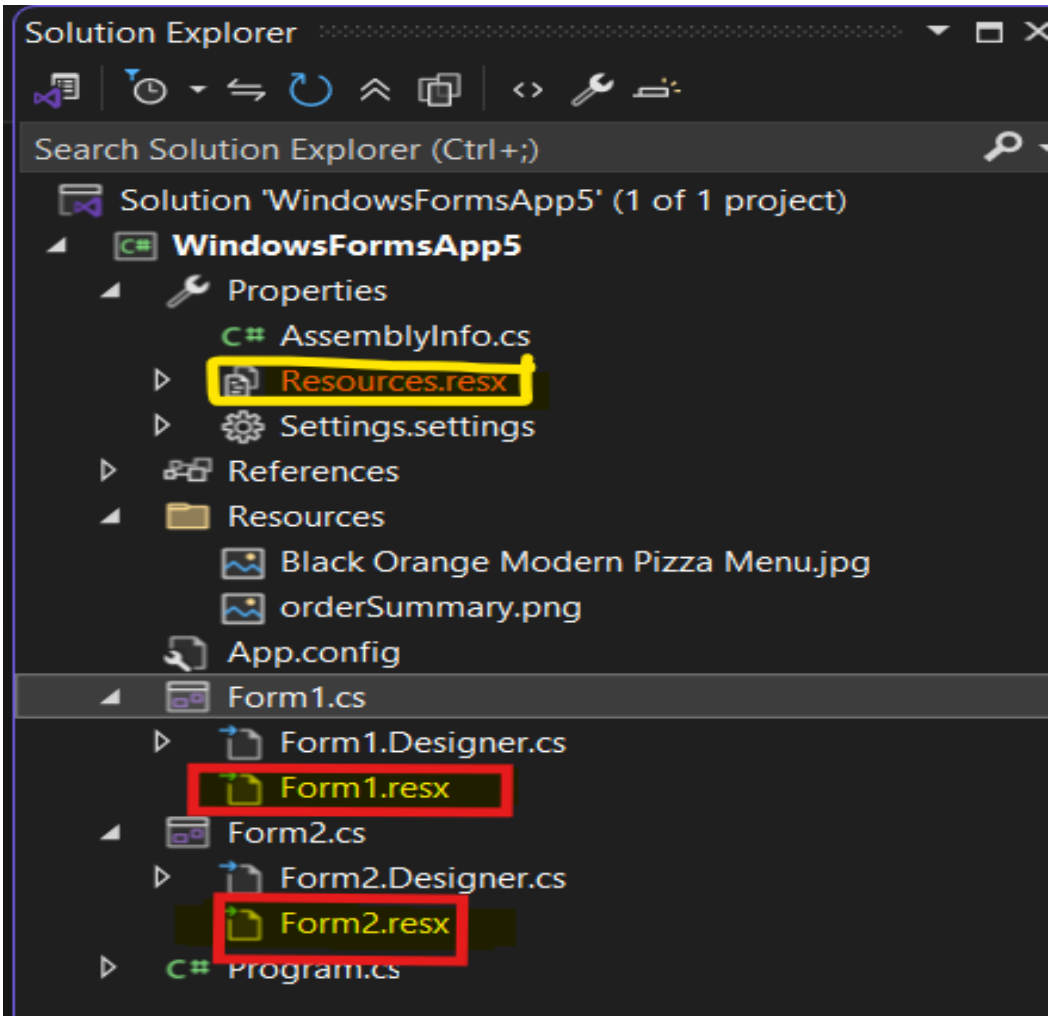
ليه إجباري ومنطقي جداً؟

تعال نفكر فيها بعقلية المبرمج:
خاصية الـ `WordWrap` وظيفتها الأساسية إيه؟ إنها لما تلاقي الكلام كتير، بتاخده وتنزله في **"سطر جديد تحت السطر الحالي".**

طب لو الـ `Multiline` مقفولة (`False`)؟ ده معناه إن التكست بوكس أصلاً **ممنوع ومستحيل يكون فيه أكتر من سطر واحد بس!** فبالتالي، الـ `WordWrap` حتى لو أنت مشغلها، هتيجي تنزل الكلام لسطر تحت مش هتلاقي سطر تحت تنزل فيه! فتصبح الخاصية مالهاش أي قيمة ولا تأثير.

علشان كده، الـ `WordWrap` بتوقف عاجزة ومبتشتغلش إلا لو أنت فتحت لها الباب الأول وخليت الـ `Multiline = True` .

PictureBox


Solution Explorer
Search Solution Explorer (Ctrl+;)
Solution 'WindowsFormsApp5' (1 of 1 project)
WindowsFormsApp5
Properties
AssemblyInfo.cs
Resources.resx
Settings.settings
References
Resources
Black Orange Modern Pizza Menu.jpg
orderSummary.png
App.config
Form1.cs
Form1.Designer.cs
Form1.resx
Form2.cs
Form2.Designer.cs
Form2.resx
Program.cs

أولاً: ترجمة الاختيارين

1) Local Resource

نفكها كلمة كلمة:

```
Local    = محلي
Resource = مورد
```

الترجمة:

> **مورد محلي**

لماذا سُميت Local؟

لأن كلمة **Local** معناها:

> "خاص بهذا المكان فقط."

والمقصود بالمكان هنا هو:

- Form1
- أو Form2
- أو Control معين

يعني المورد **لن يشارك مع باقي المشروع**.

---

2) Project Resource File

نفكها كلمة كلمة:

```
Project = المشروع

Resource = مورد

File = ملف
```

الترجمة:

> **ملف موارد المشروع**

لماذا سُمي Project؟

لأن المورد سيكون متاحًا **لكل المشروع** وليس لفورم واحدة.

ولماذا كلمة File؟

لأن Visual Studio سيضع المورد داخل ملف موجود فعلًا اسمه

```
Resources.resx
```

لذلك قالت

> Project Resource **File**

أي

> **استخدم ملف موارد المشروع.**

> **ملاحظة مهمة:**
> كلمة **File** هنا ليست هي الفرق الحقيقي بين الاختيارين، وإنما تشير إلى أن المورد سيُحفظ في ملف الموارد العام للمشروع (`Resources.resx`). أما المورد المحلي أيضًا يُحفظ في ملف (`Form1.resx` أو `Form2.resx`)، لذلك لا تجعل كلمة **File** تشغلك؛ الفرق الحقيقي هو **محلي أم خاص بالمشروع كله**.

ثانياً: أين يتم تخزين الصور؟

أنت أرسلت الصورة دي، وهي أفضل شرح.

عندك داخل المشروع:

```
WindowsFormsApp5
│
├── Resources.resx      ← (1)
│
├── Form1.cs
│     └── Form1.resx    ← (2)
│
└── Form2.cs
      └── Form2.resx    ← (3)
```

لاحظ أن عندك **3 ملفات resx**.

الملف الأول

```
Resources.resx
```

هذا هو

> **Project Resource File**

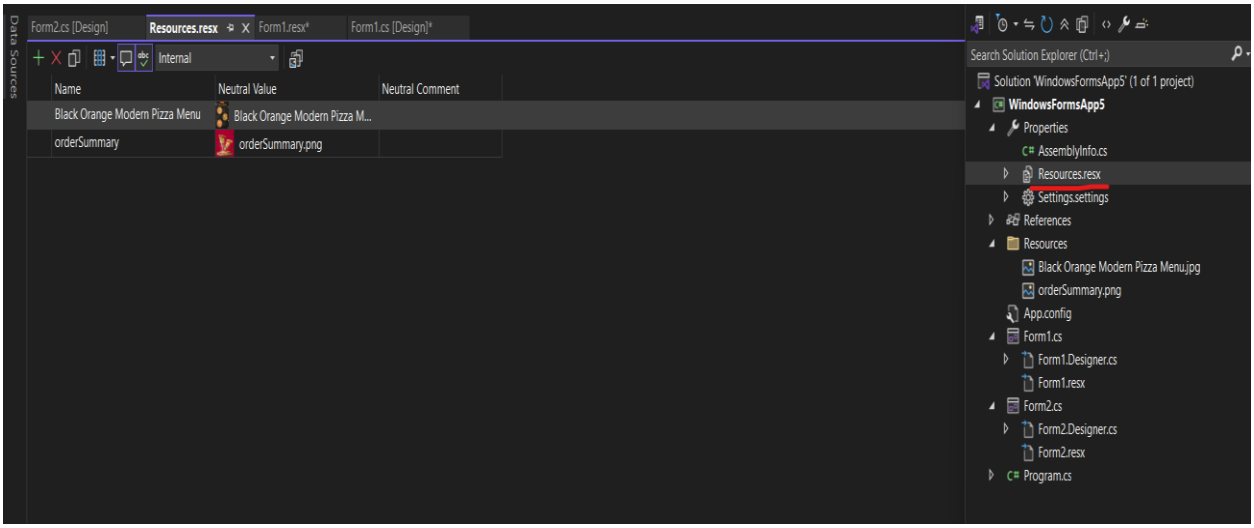
يعني مخزن المشروع كله.

أي صورة تضعها هنا

يستطيع استخدامها

- Form1
- Form2
- Form3
- Button
- PictureBox

أي جزء داخل المشروع.



الملف الثاني

```
Form1.resx
```

هذا خاص بـ

```
Form1
```

فقط.

أي صورة توضع هنا

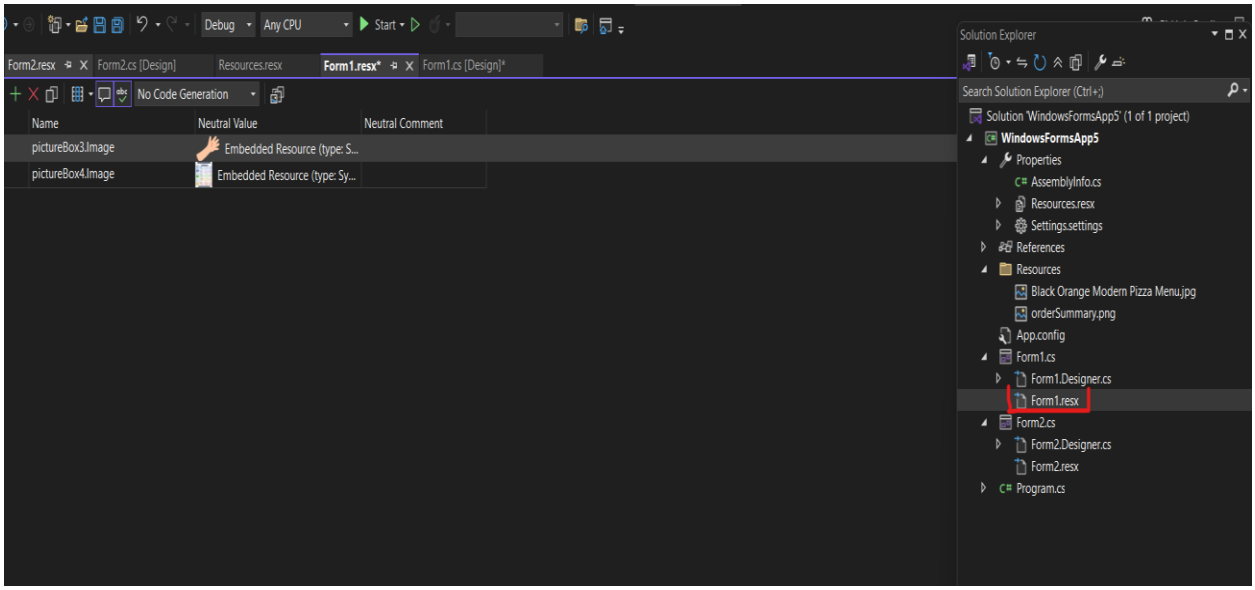
يستطيع استخدامها

```
Form1 ✓
```

لكن

```
Form2 ✗
Form3 ✗
```

لن تراها.



لن تراها.

ملاحظة مهمة جدًا:

عندما نقول إن `Form1.resx` خاص بـ **Form1**، فنحن لا نقصد نافذة الـ Form فقط، بل نقصد **الفورم نفسها وكل الـ Controls الموجودة بداخلها**، مثل:

- **"Button"**
- **"PictureBox"**
- **"Label"**
- **"TextBox"**
- **"CheckBox"**
- **"وغيرها..."**

فلو أضفت صورة إلى `PictureBox` داخل `Form1` واخترت **Local Resource**، فستُحفظ داخل:

```
Form1.resx
```

وكذلك لو أضفت صورة خلفية (**BackgroundImage**) أو أي مورد للفورم نفسها (وليس لأحد الـ Controls)، واخترت **Local Resource**، فسيُحفظ أيضًا داخل:

```
Form1.resx
```

إذن `Form1.resx` **هو ملف الموارد الخاص بالفورم نفسها وكل ما تحتويه من Controls**، وليس خاصًا بالـ Controls فقط.

الملف الثالث

```
Form2.resx
```

نفس الفكرة.

خاص بـ

```
Form2
```

فقط.

الفرق العملي

لو اخترت

```
Local Resource
```

Visual Studio تعمل هكذا:

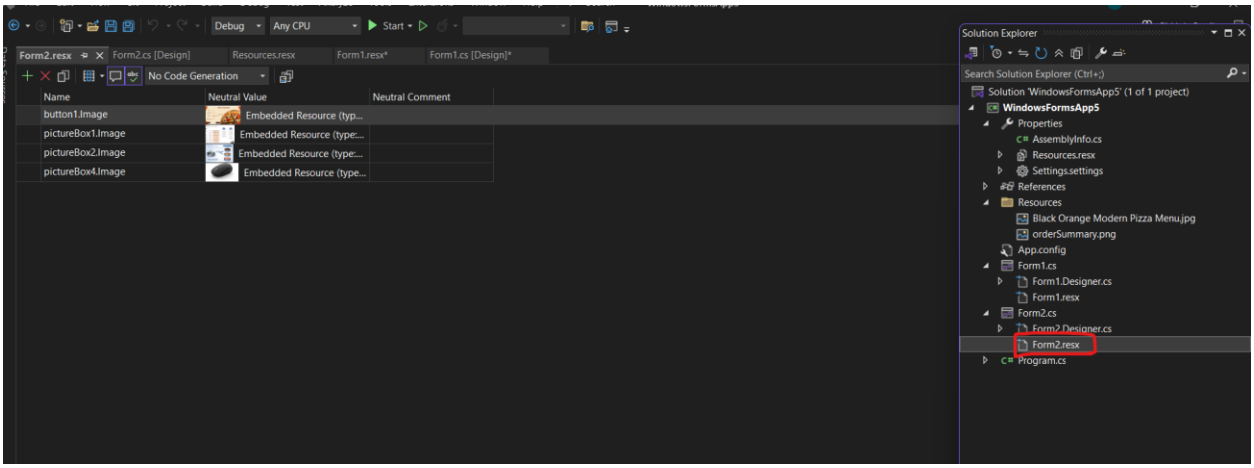
```
PictureBox داخل Form1
        |
        ▼
Form1.resx
```

ولو الصورة داخل

```
Form2
```

ستعمل:

```
PictureBox داخل Form2
        |
        ▼
Form2.resx
```



أما لو اخترت

```
Project Resource File
```

سواء كنت في

```
Form1
```

أو

```
Form2
```

أو

```
Form100
```

فإن الصورة دائمًا تذهب إلى

```
Resources.resx
```

مثال من مشروعك

تخيل عندك شعار شركتك.

```
Logo.png
```

وسيظهر في

```
Form1
Form2
Form3
```

أفضل مكان له هو

```
Resources.resx
```

لأن الجميع سيستخدمه.

أما لو عندك صورة

```
Background_Login.png
```

ستستخدم فقط في

```
LoginForm
```

فيمكن وضعها في

```
LoginForm.resx
```

لماذا يوجد النوعان؟

لأن Microsoft أرادت أن تعطيك حرية الاختيار.

إذا كان المورد

```
خاص بفورم واحدة
```

استخدم

```
Local Resource
```

أما إذا كان

```
مشتركًا بين المشروع كله
```

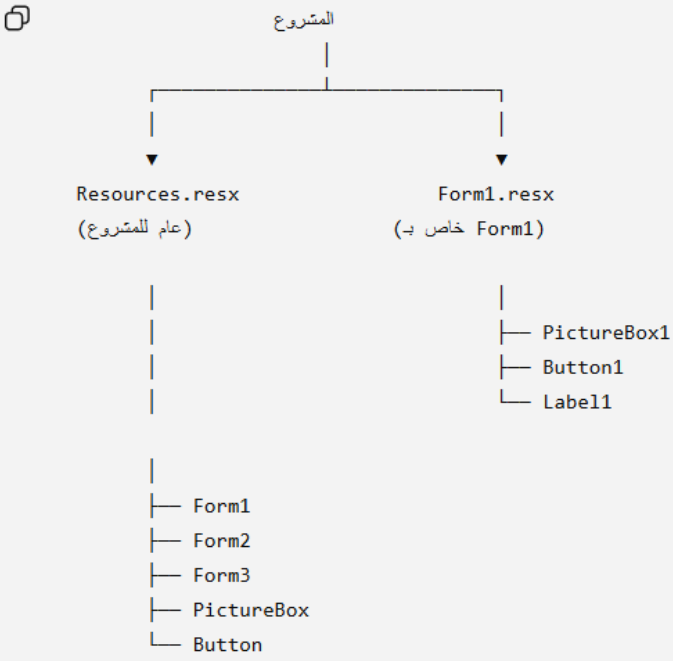
استخدم

```
Project Resource File
```

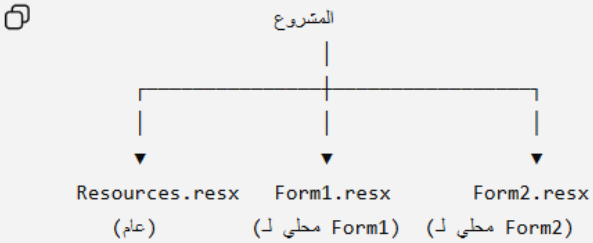
جدول المقارنة

Local Resource	Project Resource File
الترجمة: مورد محلي	**الترجمة**: ملف موارد المشروع
يُحفظ داخل `Form1.resx` أو `Form2.resx`	يُحفظ داخل `Resources.resx`
خاص بالفورم الحالية	خاص بالمشروع كله
لا يستطيع باقي الـ Forms استخدامه مباشرة	جميع الـ Forms تستطيع استخدامه
مناسب للصور الخاصة بفورم واحدة	مناسب للصور المشتركة مثل Logo أو Icons
يتكرر لكل Form ملف `.resx` خاص بها	يوجد ملف واحد فقط للمشروع كله

خريطة ذهنية



ولو أضفت Form2:



الخلاصة التي أحب أن تحفظها

```
Local Resource

يعني:

هذا المورد يخص هذه الـ Form فقط.

ويُحفظ داخل:

Form1.resx
أو
Form2.resx
```

```
Project Resource File

يعني:

هذا المورد يخص المشروع كله.

ويُحفظ داخل:

Resources.resx
```

💡 نصيحة للمشاريع الحقيقية

في المشاريع الكبيرة، يستخدم المبرمجون غالبًا **Project Resource File** للصور والأيقونات والشعارات المشتركة، لأن أي تغيير للصورة يتم من مكان واحد (`Resources.resx`) ويظهر في كل أجزاء البرنامج. أما **Local Resource** فيكون مناسبًا عندما تكون الصورة أو المورد خاصًا بفورم معينة ولا يحتاجه أي جزء آخر من المشروع.

لدينا ثلاث طرق للتعامل مع الصور في Windows Forms:

1. "Resources.ImageName"
2. "Local Resource"
3. "Image.FromFile()"

اشرح كل طريقة، وأين تُخزن الصور، وما الفرق بينها، ومتى نستخدم كل واحدة؟

الطريقة الأولى

```#C
pictureBox1.Image = Resources.Strong;
```

✅ هذه مرتبطة بـ **Project Resource File**.

لماذا؟

لأن:

```#C
Resources.Strong
```

تعنى:

> هات الصورة المسماة **Strong** من ملف

```
Resources.resx
```

أى من **ملف موارد المشروع**.

يعنى التسلسل يكون:

```
Resources.resx
      |
      ▼
Strong.png
      |
      ▼
Resources.Strong
      |
      ▼
PictureBox.Image
```

إذن هذه هى الطريقة الخاصة بـ

> ✔ **Project Resource File**

الطريقة الثانية

```C#
pictureBox1.Image = Image.FromFile(@"C:\Question.png");
```

❌ **هذه ليست Local Resource.**

وده أهم تصحيح.

هذه الطريقة ليس لها علاقة لا بـ

```
Form1.resx
```

ولا بـ

```
Resources.resx
```

هى طريقة ثالثة تمامًا.

ما الذى يحدث هنا؟

البرنامج يذهب مباشرة إلى الهارد ديسك.

```
C:
 |
 └── Question.png
```

ويقرأ الصورة.

التسلسل

```
Hard Disk

C:\Question.png

      |

      ▼

Image.FromFile()

      |

      ▼

PictureBox.Image
```

لاحظ

لا يوجد

```
Resources.resx
```

ولا

```
Form1.resx
```

إذن عندك ثلاث طرق مختلفة

الأولى

```C#
Resources.Strong
```

⬅️ Project Resource

الثانية

```
Local Resource
```

تُحفظ فى

```
Form1.resx
```

وغالبًا لا تكتبها بنفسك، بل Visual Studio تولد الكود تلقائيًا داخل ملفات الـ Designer والـ resx.

الثالثة

```C#
Image.FromFile(...)
```

⬅️ تحميل الصورة من ملف موجود على الهارد أثناء تشغيل البرنامج.

الفرق بين الطريقة الأولى والثالثة

Resources.Strong	()Image.FromFile
الصورة موجودة داخل المشروع	الصورة موجودة خارج المشروع
تُحفظ داخل `Resources.resx`	تُقرأ من الهارد ديسك
لا تحتاج وجود الملف بعد نشر البرنامج (لأنها مدمجة مع البرنامج)	يجب أن يكون الملف موجودًا في نفس المسار وإلا سيحدث Exception
أسرع قليلًا لأن المورد مدمج مع التطبيق	أبطأ قليلًا لأنه يقرأ من القرص
مناسبة للشعارات والأيقونات والصور الثابتة	مناسبة إذا كانت الصور يضيفها المستخدم أو تتغير باستمرار

سؤال قد يخطر ببالك

أين تظهر Local Resource في الكود؟

الإجابة: غالبًا لن تراها بالشكل الذي رأيته مع `Resources.Strong`.

ستجد Visual Studio تكتب في ملفات الـ `Designer` شيئًا مثل:

```
#C
this.pictureBox1.Image = ((System.Drawing.Image)(resources.GetObject("pictureBox1.Image")));
```

وهنا `resources` يشير إلى `Form1.resx` (المورد المحلي للفورم)، وليس إلى `Resources.resx` الخاص بالمشروع.

فهذه هي الطريقة التي يستخدم بها **Local Resource** Visual Studio خلف الكواليس.

إذا كانت الصور في الأصل موجودة على الهارد ديسك، فما الفرق بين Project Resource وLocal Resource وImage.FromFile()؟

والإجابة هي:

> **نعم... ولكن يوجد فرق مهم جدًا في "وقت" و"طريقة" الوصول للصورة.**

وده هو السر كله.

الطريقة الأولى والثانية

(Project Resource + Local Resource)

الصورة **في البداية** كانت موجودة على الهارد ديسك.

مثلاً

```
D:\Images\Logo.png
```

أنت اخترتها من Visual Studio.

بعدها Visual Studio عملت نسخة منها داخل المشروع.

مثلاً

```
Resources.resx
```

أو

```
Form1.resx
```

ثم أثناء عملية **Build**، يتم **دمج (Embed)** هذه الموارد داخل ملف البرنامج (`exe.`).

بعد كده...

حتى لو حذفت الصورة الأصلية من الهارد

```
D:\Images\Logo.png
```

البرنامج سيظل يعرضها.

لماذا؟

لأن الصورة أصبحت **داخل البرنامج نفسه**.

الطريقة الثالثة

```C#
Image.FromFile(...)
```

هنا البرنامج لا يحتفظ بنسخة من الصورة.

كل مرة يعمل فيها

```C#
Image.FromFile(...)
```

يذهب إلى

```
C:\Question.png
```

ويقرأها وقت التشغيل.

لو الملف غير موجود

↓

سيحدث Exception.

تخيلها كده

Project Resource

أنت عندك كتاب.

صورت صفحة.

ثم وضعتها داخل شنطتك.

بعد ذلك رميت الكتاب.

هل ما زالت الصورة معك؟

✔ نعم.

()Image.FromFile

أنت لم تصور الصفحة.

كل مرة تريد قراءتها...

تذهب إلى المكتبة.

لو المكتبة أغلقت

❌ لن تستطيع قراءتها.

إذن من أين تأتي الصورة؟

Project Resource



```
الهارد
   |
   ▼
Resources.resx
   |
   ▼
EXE
   |
   ▼
البرنامج يقرأها من داخله
```

Local Resource



```
الهارد
   |
   ▼
Form1.resx
   |
   ▼
EXE
   |
   ▼
البرنامج يقرأها من داخله
```

Image.FromFile



```
الهارد
   |
   ▼
البرنامج
   |
   ▼
كل مرة يذهب للهارد مباشرة
```

الخلاصة التى أحب أن تحفظها

الثلاث طرق تبدأ من الهارد ديسك، لكن الفرق هو:

- **Project Resource** و **Local Resource**: يأخذان **نسخة** من الصورة أثناء تصميم البرنامج، ثم تُدمج داخل ملف الـ **EXE**. بعد ذلك لا يحتاج البرنامج إلى الصورة الأصلية على الهارد.
- **Image.FromFile()**: لا يأخذ نسخة، بل يقرأ الصورة من **مكانها على الهارد في كل مرة** أثناء تشغيل البرنامج، لذلك إذا حُذفت أو تغير مسارها سيفشل البرنامج في تحميلها.

إذن الفرق الحقيقي ليس "هل الصورة جاءت من الهارد؟"، وإنما:

> **هل البرنامج يحتفظ بنسخة داخله؟ أم يعتمد على وجود الملف على الهارد وقت التشغيل؟**

ماذا تعنى علامه @ عند وضعها فى المسار جلب الصوره؟

علامة:

```C#
@
```

تسمى:

> Verbatim String Literal

وترجمتها:

> سلسلة نصية حرفية.

الخطوة الأولى: الكومبايلر بيقرأ الـ String إزاي؟

لما تكتب:

```C#
string Name = "Ali";
```

الكومبايلر يقرأ ما بين علامتي الاقتباس على أنه **نص (String)**.

الخطوة الثانية: هل كل حرف داخل الـ String عادي؟

الإجابة: **لا**.

فيه بعض الحروف لها **معنى خاص**.

أشهرها:

```C#
\n
```

معناها:

> انزل سطر جديد.

مثال:

```C#
Console.WriteLine("Ali\nAhmed");
```

النتيجة:

```
Ali
Ahmed
```

لاحظ أن:

```
\n
```

لم تُطبع كما هي، بل نفذت أمر **النزول لسطر جديد**.

الخطوة الثالثة: لماذا حدث ذلك؟

لأن الكومبايلر رأى:

```
\
```

فقال:

> "انتظر... الـ Backslash معناها أن الحرف الذي بعدها له معنى خاص."

فقرأ:

```
\n
```

على أنها أمر.

الخطوة الرابعة

يوجد أوامر كثيرة تبدأ بـ

```
\
```

مثل:

```
\n
```

سطر جديد.

```
\t
```

.Tab

```
\\
```

.Backslash

الخطوة الخامسة

انظر إلى المسار:

```
C:\Users\OMRAN\Desktop
```

كم Backslash فيه؟

```
C: \ Users \ OMRAN \ Desktop
```

كل واحدة منها تبدأ بـ

```
\
```

الكومبايلر يبدأ يقرأ:

```
\U
```

ويقول:

> هل هذه Escape Sequence؟

ثم يكمل القراءة.

الخطوة السادسة

كيف أخبر الكومبايلر أن هذه ليست أوامر؟

يوجد حلان.

الحل الأول

أكتب كل Backslash مرتين.

```C#
"C:\\Users\\OMRAN\\Desktop"
```

لماذا؟

لأن:

```
\\
```

تعني:

> اطبع Backslash واحدة.

الحل الثاني

أضع قبل الـ String

```C#
@
```

فتصبح:

```C#
@"C:\Users\OMRAN\Desktop"
```

وهنا تقول للكومبايلر:

> **اقرأ هذا النص حرفيًا كما هو، ولا تعتبر الـ `\` بداية لأوامر خاصة.**

مقارنة

بدون @

```C#
"C:\\Users\\OMRAN\\Desktop\\Photo.jpg"
```

الكومبايلر يحتاج إلى:

```
\\
```

لكي يعرف أنك تريد Backslash حقيقية.

مع @

```C#
@"C:\Users\OMRAN\Desktop\Photo.jpg"
```

الكومبايلر يقرأ النص كما هو، لذلك لا تحتاج إلى مضاعفة الـ Backslash.

الخلاصة التي أحب أن تحفظها

```
#C ‹/›
@
```

تعني:

> اقرأ النص حرفيًا (Verbatim)، ولا تعامل `\` على أنها بداية Escape Sequence.

Tic-Tac-Toe Game

ملخص تصميم الدكتور فى مشروع XO

فكرة التصميم

الدكتور استخدم أسلوب **تنفيذ مهمة (Command)** وليس أسلوب **إرجاع قيمة (Query)**.

يعنى الدالة لا ترجع فقط نتيجة، وإنما تنفذ عدة أوامر أثناء عملها.

مثال:

```
#C ‹/›
CheckWinner();
```

هذه الدالة لا تكتفى بفحص الفائز، بل قد تقوم أيضًا بـ:

- استدعاء `CheckValues()`.
- تحديد الفائز.
- تغيير لون الأزرار.
- إنهاء اللعبة (`EndGame()`).
- إظهار رسالة انتهاء اللعبة.

لماذا هذا ليس أفضل تصميم؟

لأن الدالة لها أكثر من مسئولية، وهذا يخالف مبدأ:

Single Responsibility Principle (SRP)

كما أن اسمها (`CheckWinner`) يوحى بأنها ستقوم بالفحص فقط، بينما هى فى الحقيقة تقوم بمعالجة كاملة لحالة اللعبة.

ما العيب فى هذا الأسلوب؟

بعد استدعاء:

```C#
CheckWinner();
```

قد تكون اللعبة انتهت بالفعل وتم تغيير حالة البرنامج بالكامل، لذلك يجب على الدالة التى استدعتها أن تراعى ذلك عند رجوع التنفيذ إليها.

أى أن الدالة لها **Side Effects** (تأثيرات جانبية)، وليس مجرد إرجاع قيمة.

ما التصميم الأفضل؟

الأفضل غالبًا أن تكون الدالة:

```C#
CheckWinner()
```

ترجع فقط حالة اللعبة أو الفائز، مثل:

```C#
enWinner winner = CheckWinner();
```

ثم تكون الدالة المستدعية هى التى تقرر:

```C#
if (winner != enWinner.GameInProgress)
{
    EndGame();
}
```

وبذلك يتم فصل المسئوليات ويصبح الكود أوضح وأسهل فى الصيانة.

لماذا استخدم الدكتور هذا التصميم؟

الدكتور ذكر بنفسه أنه **ليس أفضل تصميم**.

والسبب أنه أراد تبسيط المشروع للطلاب، وتقليل عدد الدوال والأفكار فى مشروع صغير مثل XO، حتى يكون التركيز على تعلم أساسيات Windows Forms وليس على تصميم البرامج الاحترافى.

الخلاصة

- ✅ التصميم يعمل بشكل صحيح.
- ✅ مناسب للتعليم والمشاريع الصغيرة.
- ❌ ليس أفضل تصميم من ناحية هندسة البرمجيات.
- ✅ فى المشاريع الكبيرة يفضل فصل الفحص (Query) عن تنفيذ الأوامر (Command) والالتزام بمبدأ **Single Responsibility**.

ملاحظة شخصية (من استنتاجى أثناء المذاكرة):

بعد تحليل الكود، فهمت أن تصميم الدكتور يعتمد على **تنفيذ المهمة بالكامل** داخل `CheckWinner` ، وليس مجرد فحص الفائز. لذلك عند استدعائها يجب الانتباه إلى أنها قد تكون غيرت حالة البرنامج (GameOver, Winner, UI...) قبل أن يعود التنفيذ إلى الدالة المستدعية. لهذا السبب أرى أن التصميم الذى يعتمد على إرجاع قيمة فقط يكون أوضح وأسهل فى تتبع مسار التنفيذ، خاصة فى المشاريع الكبيرة.

PictureBox Exercise Solution

ما هو Sender

ما هو `sender` ؟

كل Event Handler في WinForms ستجد له غالبًا معاملين:

```C#
private void rbBoy_CheckedChanged(object sender, EventArgs e)
```

اليوم سنركز على:

```C#
sender
```

ما معنى `sender` ؟

`sender` في اللغة الإنجليزية معناها:

> **المرسل (The Sender).**

ولذلك سُمِّي بهذا الاسم لأنه يمثل **الكنترول الذي أرسل أو تسبب في حدوث الـ Event**.

يعني هو الكنترول الذي تسبب في تشغيل هذه الدالة.

ما معنى `sender` ؟

`sender` في اللغة الإنجليزية معناها:

> **المرسل (The Sender).**

ولذلك سُمِّي بهذا الاسم لأنه يمثل **الكنترول الذي أرسل أو تسبب في حدوث الـ Event.**

يعني هو الكنترول الذي تسبب في تشغيل هذه الدالة.

مثال من الواقع

تخيل أن هناك شخصين:

- 👨‍💼 باشمهندس علي.
- 👨‍💼 باشمهندس سيد.

باشمهندس سيد اتصل بمهندس علي.

فمهندس علي رد على المكالمة.

لكن قبل أن يتكلم، أحب يعرف:

> **مين اللي رن عليّ؟**

ففتح بيانات المكالمة، فوجد أن:

> **المرسل (Sender) هو باشمهندس سيد.**

إذن عرف من الذي بدأ الاتصال.

نفس الفكرة تحدث في البرنامج.

أثناء تشغيل البرنامج، حدث شيء معين، مثل أنك اخترت:

```
rbBoy
```

فحدث الـ Event:

```
CheckedChanged
```

وبسبب حدوث هذا الـ Event تم استدعاء الـ Event Handler:

```C#
rbBoy_CheckedChanged(...)
```

وداخل هذه الدالة، إذا أردنا أن نعرف:

> **مين الكنترول اللي سبب استدعاء الدالة؟**

سننظر إلى:

```C#
sender
```

فنجد أن:

```
sender = rbBoy
```

أي أن `rbBoy` هو **المرسل (Sender)** الذي تسبب في استدعاء هذه الدالة.

مثال

عندك:

```
rbBoy
rbGirl
rbBook
rbPen
```

ولكل واحد Event اسمه:

```
CheckedChanged
```

لو اخترت:

```
rbBoy
```

سيتم استدعاء:

```
#C
rbBoy_CheckedChanged(...)
```

وفي هذه اللحظة يكون:

```
sender = rbBoy
```

لأن `rbBoy` هو الذي تسبب في حدوث الـ Event.

ولو اخترت:

```
rbGirl
```

سيتم استدعاء:

```
#C
rbGirl_CheckedChanged(...)
```

وفي هذه المرة:

```
sender = rbGirl
```

ولو اخترت:

```
rbBook
```

سيكون:

```
sender = rbBook
```

وهكذا مع أي RadioButton.

لماذا استخدم `sender`؟

كان من الممكن أن يكتب:

```C#
lblTitle.Text = rbBoy.Tag.ToString();
```

وهذا صحيح.

لكن الأستاذ كتب:

```C#
lblTitle.Text = ((RadioButton)sender).Tag.ToString();
```

لأنه يريد أن يعلمك استخدام `sender` .

لماذا عمل Casting؟

لأنه يعرف أن الكنترول الذي تسبب في حدوث الـ Event هو:

```
RadioButton
```

لذلك قام بتحويل `sender` إلى:

```C#
(RadioButton)sender
```

وبعد ذلك أصبح يستطيع الوصول إلى:

```C#
.Tag
```

ما الفائدة من ذلك؟

اليوم لا يوجد فرق كبير.

لكن في المستقبل قد يجعل أكثر من Control ينادي نفس الـ Event Handler.

بدل أن يكتب:

```
rbBoy_CheckedChanged

rbGirl_CheckedChanged

rbBook_CheckedChanged

rbPen_CheckedChanged
```

يمكن أن يجعلهم جميعًا ينادوا دالة واحدة.

وعندها يستطيع معرفة من الذي تسبب في استدعاء هذه الدالة عن طريق:

```C#
sender
```

وبذلك يميز بين الكنترولات المختلفة.

الخلاصة

- `sender` هو **الكنترول الذي تسبب في حدوث الـ Event**.
- عند حدوث أي **Event** (مثل `Click` أو `CheckedChanged`)، يحمل `sender` **الكنترول الذي حدث عليه هذا الـ Event**.
- إذا كنت تعرف نوع هذا الكنترول، تستطيع عمل **Casting** إليه.
- الفائدة الحقيقية لـ `sender` تظهر عندما تستخدم **Event Handler واحدة لأكثر من Control**، فتستطيع معرفة أي **Control** هو الذي تسبب في استدعاء هذه الـ **Event Handler**.

> **احفظها بهذه الجملة:**
>
> **Event يحدث على Control → فيتم استدعاء Event Handler → و `sender` يشير إلى الـ Control الذي حدث عليه الـ Event.**

الخلاصه التامه للفرق

فهم الـ Events و `sender` و `EventArgs (e)` من الصفر

المشهد الأول: البرنامج مغلق

تخيل أن عندك برنامج مشهور مثل **Microsoft Word**.

قبل أن تضغط عليه، الوضع يكون كالتالى:

```
Word.exe

البرنامج مغلق ❌
```

لا يوجد:

- أزرار تعمل.
- TextBox.
- TreeView.
- أحداث (Events).

كل شىء ساكن.

المشهد الثانى: تشغيل البرنامج

الآن المستخدم ضغط مرتين على أيقونة Word.

```
👤
Double Click

↓

Word.exe
```

يقوم Windows بتشغيل البرنامج.

ثم يبدأ الـ CLR فى إنشاء كل الكائنات الموجودة داخل البرنامج.

فيقوم بإنشاء:

```
Form

├── Save Button
├── Open Button
├── Print Button
├── TextBox
├── CheckBox
├── TreeView
└── ...
```

الآن البرنامج أصبح يعمل.

المشهد الثالث: ماذا يفعل البرنامج الآن؟

ولا أى شىء.

يقف وينتظر.

```
برنامج Word

Waiting...
```

لماذا؟

لأن WinForms نظامه اسمه

> **Event Driven Programming**

أى:

> البرنامج لا يفعل شيئًا حتى يحدث Event.

المشهد الرابع: بداية الأحداث

الآن المستخدم بدأ يتعامل مع البرنامج.

مثلاً:

- ضغط على زر.
- كتب داخل TextBox.
- اختار Node من TreeView.
- عمل Check على CheckBox.
- حرك الماوس.

كل واحدة من هذه تسمى

```
Event
```

لأنها حدث حصل أثناء تشغيل البرنامج.

ولهذا سموها **Events**.

المشهد الخامس: المستخدم ضغط على زر Save

لنفترض أن المستخدم ضغط على زر Save.

```
👤

Click

↓

Save Button
```

فى هذه اللحظة حدث Event.

اسم هذا الحدث

```
Click
```

المشهد السادس: من الذى يتم استدعاؤه؟

بمجرد حدوث الحدث...

يقوم الـ Framework باستدعاء الدالة المرتبطة بهذا الحدث.

وهى

```C#
buttonSave_Click(...)
```

هذه الدالة تسمى

```
Event Handler
```

لأنها المسئولة عن التعامل مع الحدث.

أهم نقطة

لاحظ جيدًا.

الزر نفسه لا يريد أن يعرف أى شىء.

ولا الـ TreeView.

ولا الـ CheckBox.

الذى يريد أن يعرف هو فقط

> **الـ Event Handler**

هو الذى يقول:

> "أنا تم استدعائى... لكن أريد أن أعرف ماذا حدث."

وهنا يبدأ دور

```
sender
```

و

```
e
```

أول سؤال يسأله الـ Event Handler

يقول:

> من الذى استدعانى؟

بمعنى:

> من الكنترول الذى أطلق هذا الحدث؟

فيأتى

```C#
sender
```

ويجيب:

```
أنا...

Save Button
```

إذن

```
sender

↓

Save Button
```

لو كان الحدث من TreeView

سيكون

```
sender
↓
TreeView
```

ولو كان من TextBox

```
sender
↓
TextBox
```

إذن

> sender = الكنترول الذى أطلق الحدث.

ثانى سؤال يسأله الـ Event Handler

بعد أن عرف من الذى استدعاه...

يقول:

> جميل...

لكن ماذا حدث بالضبط؟

هنا يأتى دور

```
e
```

ما هو e ؟

كثير من الناس يعتقد أن

```
e
```

هو الكنترول.

وهذا خطأ.

وكثير يعتقد أنه يحمل معلومات عن الكنترول.

وهذا أيضًا خطأ.

الحقيقة هى:

> **e يحمل المعلومات الخاصة بالحدث الذى وقع فقط.**

لاحظ...

لم أقل:

> معلومات الكنترول.

بل قلت:

> معلومات الحدث.

مثال 1

حدث

```
Click
```

الـ Event Handler يسأل:

> هل يوجد معلومات مهمة فى هذا الحدث؟

الإجابة:

لا.

مجرد ضغطة.

لذلك

```C#
EventArgs e
```

لا يحمل تقريبًا أى بيانات إضافية.

مثال 2

حدث

```
MouseClick
```

الـ Event Handler يسأل:

> ماذا أحتاج أن أعرف؟

ربما يحتاج أن يعرف:

- هل الضغط يمين؟
- أم شمال؟
- أين حدث الضغط؟

إذن

```C#
MouseEventArgs e
```

يحمل

```
Button

X

Y

Clicks
```

لاحظ

هذه ليست معلومات عن الزر.

بل معلومات عن

> الحدث نفسه.

مثال 3

حدث

```
KeyDown
```

الـ Event Handler يقول:

> أريد أن أعرف...

أى زر ضغطه المستخدم؟

فيأتى

```
KeyEventArgs
```

ويحمل

```
KeyCode

Shift

Control

Alt
```

مثال 4

حدث

```
AfterCheck
```

على TreeView.

الـ Event Handler لا يهمه حجم الـ TreeView.

ولا لونه.

ولا مكانه.

كل ما يهمه:

> المستخدم عمل Check على أى Node؟

إذن

```
TreeViewEventArgs
```

يحمل

```
Node
```

ومنها تستطيع أن تعرف

```
#C ‹/›
e.Node.Checked
```

لاحظ هذه الجملة جيدًا

الـ Event Handler لا يهتم بالكنترول كله.

هو يهتم فقط بالإجابة عن سؤالين.

```
1- من الذى أطلق الحدث؟

↓

sender
```

```
2- ما المعلومات الخاصة بهذا الحدث؟

↓

e
```

لا أكثر.

لماذا يتغير نوع e ؟

لأن كل حدث يحتاج بيانات مختلفة.

حدث Click لا يحتاج بيانات.

أما MouseClick فيحتاج:

- مكان الماوس.
- الزر المضغوط.

أما KeyDown فيحتاج:

- المفتاح المضغوط.

أما TreeView فيحتاج:

- الـ Node التى حدث عليها الحدث.

ولهذا تجد:

```C#
EventArgs
```

أو

```C#
MouseEventArgs
```

أو

```C#
KeyEventArgs
```

أو

```C#
TreeViewEventArgs
```

كل واحد يحمل بيانات تناسب الحدث الذى وقع.

تخيل أن الـ **Event Handler** موظف داخل البرنامج.

هو قاعد طول الوقت لا يعمل شيئًا.

```
Event Handler

😴 Waiting...
```

طالما المستخدم لم يفعل شيئًا...

هو لا يعمل.

البرنامج بدأ يشتغل.

```
البرنامج يعمل...

        |
        ▼

المستخدم فعل شيئًا
```

مثلاً:

- ضغط Button.
- كتب فى TextBox.
- عمل Check.
- اختار Node.

فى هذه اللحظة فقط...

حدث Event.

```
        |
        ▼
حدث Event
```

بمجرد حدوث الحدث...

الـ Framework ينادى على الموظف.

```
        |
        ▼
تم استدعاء الـ Event Handler
```

الـ Event Handler يقول:

> "أنا تم استدعائى...
>
> إذًا حدث شىء أثناء تشغيل البرنامج."

لكن...

هو لا يعرف ماذا حدث.

ولا من الذى تسبب فيه.

لذلك أول سؤال يسأله هو:

```
مين اللى استدعانى؟
```

```
sender
```

أى:

> الكنترول الذى أطلق هذا الحدث.

بعد أن عرف مصدر الحدث...

يسأل السؤال الثانى.

```
طيب...

إيه اللى حصل فى الحدث ده؟
```

وليس:

> إيه معلومات الكنترول؟

لا.

هو لا يهتم بالكنترول نفسه.

هو يهتم فقط بما حدث أثناء هذا الحدث.

فتأتى الإجابة داخل:

```
e
```

والتى تحمل البيانات الخاصة بالحدث.

مثل:

- مكان الماوس.
- الزر المضغوط.
- الـ Node المختارة.
- المفتاح المضغوط.

على حسب نوع الحدث.

أهم نقطة

الـ Event Handler لا يهتم بالكنترول نفسه.

ولا يعرف عنه شيئًا.

كل ما يهمه هو:

> **أنا تم استدعائى بسبب حدث حصل أثناء تشغيل البرنامج.**

إذن أريد فقط أن أعرف شيئين:

```
1- من الذى أطلق هذا الحدث؟

↓

sender
```

```
2- ما البيانات الخاصة بهذا الحدث؟

↓

e
```

ولا يهتم بأى شىء آخر.

الملخص

ما هو Event؟

هو أى حدث يحدث أثناء تشغيل البرنامج نتيجة لتفاعل المستخدم مع أحد الكنترولات.

ما هو Event Handler؟

هو الدالة التى يتم استدعاؤها عندما يحدث حدث أثناء تشغيل البرنامج.

ووظيفته الأساسية هى التعامل مع هذا الحدث.

عندما يتم استدعاء الـ Event Handler

فهو لا يهمه البرنامج كله.

ولا يهمه الكنترول كله.

كل ما يهمه هو الحدث الذى استدعاه.

ولذلك يسأل سؤالين فقط:

السؤال الأول

> من الذى أطلق هذا الحدث؟

الإجابة:

```
sender
```

السؤال الثانى

> ما المعلومات الخاصة بهذا الحدث؟

الإجابة:

```
e
```

ثم يبدأ فى تنفيذ الكود الذى كتبه المبرمج.

القاعدة الذهبية

البرنامج يعمل

|
▼

المستخدم يفعل شيئًا

|
▼

Event حدث

|
▼

يتم استدعاء الـ Event Handler

|
▼

الـ Event Handler يقول:

"أنا لا يهمنى البرنامج كله...

ولا الكنترول كله...

كل ما يهمنى هو الحدث الذى استدعانى."

|
▼

فيبدأ بسؤالين فقط:

1- من الذى أطلق هذا الحدث؟
↓
sender

2- ما المعلومات الخاصة بهذا الحدث؟
↓
e

ثم ينفذ الكود.

القاعدة الذهبية التى أريدك أن تتذكرها دائمًا

البرنامج يعمل ← المستخدم يفعل شيئًا ← يحدث Event ← يتم استدعاء الـ Event Handler ← يسأل سؤالين فقط:

من الذى أطلقنى؟ ← `sender`

وماذا حدث بالضبط؟ ← `e`

MaskedTextBox

ما هو `MaskedTextBox` ؟

هو أحد الكنترولات الموجودة في WinForms.

ويستخدم عندما تريد من المستخدم أن يكتب البيانات **بشكل أو بصيغة محددة**.

أولًا: ترجمة الاسم

اسم الكنترول هو:

```
MaskedTextBox
```

لو قسمناه:

```
Masked + Text + Box
```

1. Masked

الترجمة الحرفية هي:

> **مُقَنَّع** أو **مغطى بقناع**.

وهذه الترجمة صحيحة لغويًا.

لكن في البرمجة لا نقف عند الترجمة الحرفية، بل نفهم **لماذا سُمِّي بهذا الاسم**.

مثال يوضح الفكرة

تخيل أنك ذهبت إلى حفلة تنكرية.

كل شخص يرتدي **قناعًا (Mask)**.

القناع يجعل الشخص يظهر بشكل معين، ولا يسمح لك برؤية شكله الحقيقي بالكامل.

بنفس الفكرة...

الـ **Mask** في البرمجة يضع **قناعًا على طريقة إدخال البيانات**.

فبدل أن يكون المستخدم حرًا يكتب أي شيء:

```
__________
```

يصبح مربع النص وكأنه يرتدي قناعًا مثل:

```
__/__/____
```

أو

```
(___) ___-____
```

فهذا "القناع" يجبر المستخدم أن يكتب بهذا الشكل فقط.

لذلك نقول إن الـ **Mask** يفرض **نمطًا (Pattern)** أو **قالبًا** للإدخال.

إذن:

- **الترجمة الحرفية:** مُقنَّع.
- **المعنى البرمجي:** عليه قناع (Mask) يفرض شكلًا معينًا للإدخال.

2. Text

تعني:

> نص.

---

3. Box

تعني:

> صندوق.

إذن معنى الاسم كله

```
Masked Text Box
```

أي:

> **مربع نص عليه Mask (قناع) يجبر المستخدم على الكتابة بصيغة محددة.**

لماذا سُمِّي بهذا الاسم؟

لأن هذا الكنترول في الأصل هو:

```
TextBox
```

ولكن أضيف إليه:

```
Mask
```

ليتحكم في طريقة كتابة البيانات.

ما وظيفة الـ Mask؟

وظيفته أن يمنع المستخدم من إدخال بيانات بأي شكل.

بل يجبره على الكتابة بالشكل الذي حدده المبرمج.

مثال (التاريخ)

بدل أن يكتب المستخدم:

```
15122006
```

أنت تريده أن يكتب:

```
15/12/2006
```

فتضع له Mask.

فيظهر له:

```
__/__/____
```

ولا يستطيع إلا إدخال الأرقام في أماكنها.

مثال (رقم الهاتف)

بدل أن يكتب:

```
01012345678
```

يمكن أن يظهر له:

```
010-____-____
```

ويكمل الأرقام فقط.

مثال (الرقم القومي)

يمكن أن تجعل الكنترول يقبل:

```
30208151234567
```

ولا يسمح بإدخال الحروف.

الفرق بين `TextBox` و `MaskedTextBox`

أولًا: TextBox

هو مربع نص عادي.

يسمح للمستخدم أن يكتب أي شيء تقريبًا.

مثل:

```
Ahmed
12345
Hello
@@@@
```

المستخدم حر.

ثانيًا: MaskedTextBox

هو أيضًا مربع نص.

لكن عليه **Mask**.

لذلك لا يسمح للمستخدم أن يكتب إلا بالشكل الذي حددته أنت.

متى أستخدم TextBox؟

عندما لا يهم شكل البيانات.

مثل:

- الاسم.
- العنوان.
- الملاحظات.
- الرسائل.

متى أستخدم MaskedTextBox؟

عندما يكون للبيانات شكل ثابت.

مثل:

- التاريخ.
- الوقت.
- رقم الهاتف.
- الرقم القومي.
- الرمز البريدي.

جدول ملخص

الكنترول	الوظيفة
TextBox	يسمح للمستخدم بكتابة أي نص تقريبًا.
MaskedTextBox	يجبر المستخدم على إدخال البيانات بصيغة أو نمط (Mask) يحدده المبرمج.

الخلاصة

- **Masked** ترجمتها الحرفية **مُقنَّع**.
- المقصود برمجيًا أن مربع النص عليه **Mask (قناع)** يفرض طريقة معينة للإدخال.
- **Text** = نص.
- **Box** = صندوق.
- **MaskedTextBox** = مربع نص عليه قناع يحدد للمستخدم كيف يكتب البيانات.
- نستخدمه عندما تكون البيانات لها **صيغة ثابتة** مثل التاريخ أو رقم الهاتف، بينما نستخدم **TextBox** عندما نريد من المستخدم أن يكتب بحرية.

هناك خيار آخر اسمه Custom

بعد أن تعرفنا على القوالب الجاهزة الموجودة داخل **Mask Description**، يوجد خيار آخر مهم جدًا اسمه:

```
Custom
```

وسنشرحه بالتسلسل.

أولًا، اعرف أن:

> **Custom نفسه يعتبر واحدًا من الـ Templates (القوالب).**

يعنى أنت **لا تختار قالبًا جاهزًا ثم تحوله إلى Custom**.

بل عند فتح نافذة **Input Mask** يكون الترتيب كالتالى:

1. تفتح نافذة **Input Mask**.
2. تنظر إلى قائمة **Mask Description**.
3. إما أن تختار قالبًا جاهزًا مثل:
   - Phone Number
   - Short Date
   - Time
   - ZIP Code
4. أو تختار مباشرة:
   - **Custom**

إذا اخترت **Custom**، ستكتب أنت الـ **Mask** بنفسك، ولن تعتمد على أى قالب جاهز.

ما معنى Custom؟

Custom ترجمتها:

> **مخصص أو من تصميمك أنت.**

يعنى بدل ما تختار Template جاهزة من Microsoft، أنت ستكتب الـ Mask بنفسك.

مثال

نفترض أن شركتك لديها رقم موظف بالشكل التالي:

```
EMP-1234
```

يعنى لازم المستخدم يكتب:

```
EMP-0001
EMP-2548
EMP-9999
```

لن تجد هذا ضمن القوالب الجاهزة.

إذن تختار:

```
Mask Description:
Custom
```

ثم تكتب الـ Mask بنفسك.

مثال آخر

لنفترض أنك تريد أن يكون الإدخال هكذا:

```
AA-999
```

مثل:

```
AB-123
XY-456
```

فهنا أيضًا تستخدم:

```
Custom
```

وتكتب الـ Mask التى تريدها.

لماذا لم تضع Microsoft كل القوالب؟

لأن كل برنامج له احتياجات مختلفة.

مثلاً:

- شركة تريد كود موظف.
- مستشفى تريد رقم ملف مريض.
- جامعة تريد رقم طالب.
- مصنع يريد رقم منتج.

كل هذه أشكال مختلفة، ولا يمكن لـ Microsoft أن توفرها كلها.

لذلك أعطتك خيار:

> **Custom** = "اصنع القالب الذى يناسب برنامجك."

مثال فى Visual Studio

لو اخترت:

```
Custom
```

ستستطيع كتابة Mask مثل:

```
000-000
```

فتصبح الإدخالات مثل:

```
123-456
987-654
```

أو:

```
LL-0000
```

فتكون:

```
AB-1234
CD-9876
```

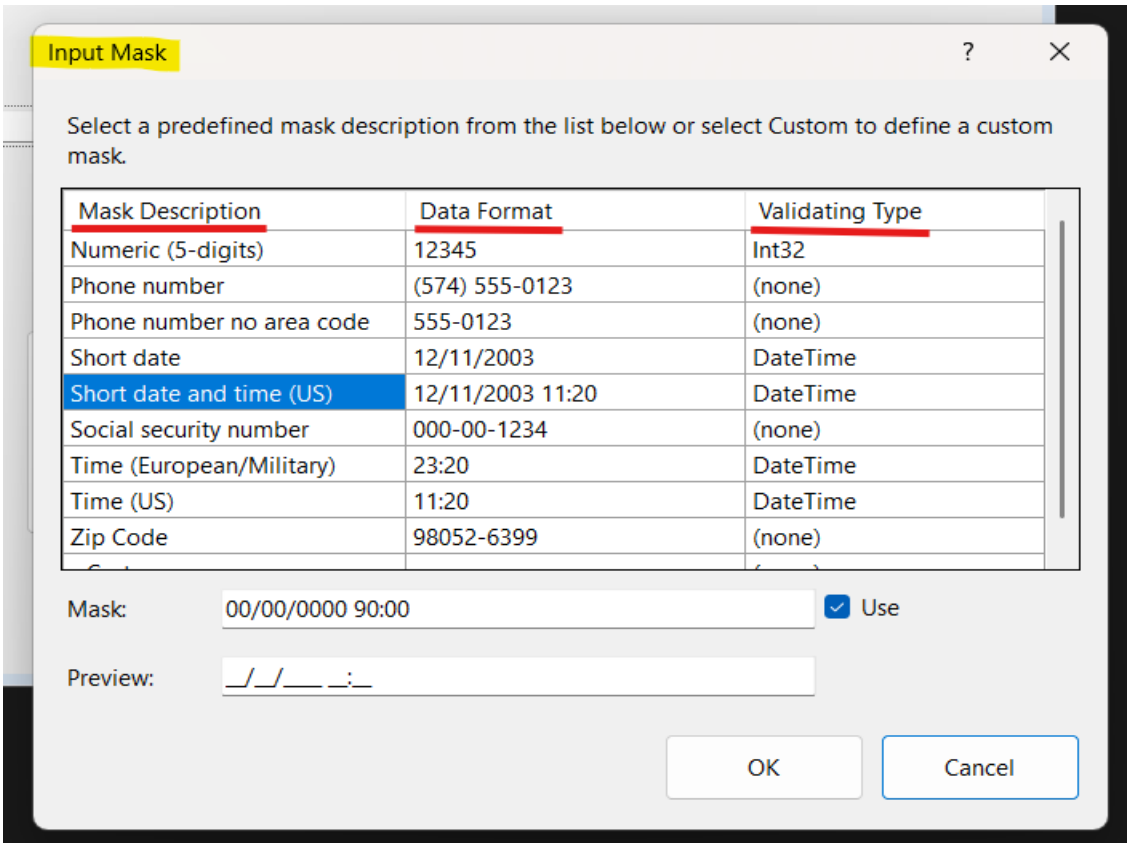
الخلاصة

القوالب الجاهزة مناسبة للأشياء الشائعة مثل:

- Date
- Time
- Phone Number
- ZIP Code

أما إذا أردت **شكلًا خاصًا ببرنامجك**، فتختار:

> **Custom** → ثم تكتب الـ Mask بنفسك.



نافذة Input Mask

هذه النافذة تحتوى على مجموعة من **الـ Templates الجاهزة** التى وفرتها Microsoft.

كل **صف** يمثل Template كاملة.

يعنى كل صف يحتوى على:

- اسم الـ Template.
- شكل البيانات.
- نوع التحقق.

أنت لا تختار كل عمود وحده، بل تختار **الصف بالكامل**.

العمود الأول

Mask Description

الترجمة

```
Mask Description
```

تعنى:

> **وصف الـ Mask** أو **اسم القالب الجاهز**.

وظيفته

هذا العمود يخبرك:

> **ما وظيفة هذا الـ Mask؟**

فهو مجرد اسم للقالب.

مثلاً:

```
Phone Number
```

معناه:

> هذا قالب مخصص لرقم الهاتف.

أو

```
Short Date
```

يعنى:

> هذا قالب مخصص للتاريخ القصير.

أو

```
Time (US)
```

يعنى:

> هذا قالب مخصص للوقت بالنظام الأمريكى.

إذن هذا العمود لا يحتوى على طريقة الإدخال.

هو فقط:

> اسم الـ Template التى ستختارها.

العمود الثانى

Data Format

الترجمة

```
Data Format
```

تعنى:

> **صيغة البيانات** أو **شكل البيانات**.

وظيفته

بعد أن اخترت Template معينة...

هذا العمود يخبرك:

> **كيف سيكتب المستخدم البيانات؟**

مثال

اخترت:

```
Short Date
```

ستجد:

```
12/11/2003
```

هذا ليس تاريخًا معينًا.

بل مجرد مثال يوضح شكل الإدخال.

أى أن المستخدم سيكتب:

```
يوم / شهر / سنة
```

مثال آخر

لو اخترت:

```
Phone Number
```

ستجد:

```
(574) 555-0123
```

يعنى:

يعنى:

> المستخدم سيكتب رقم الهاتف بهذا الشكل.

مثال ثالث

لو اخترت:

```
Time (US)
```

سيظهر:

```
11:20
```

وهذا يوضح شكل كتابة الوقت.

من الذى يجبر المستخدم على الالتزام بهذا الشكل؟

الذى يفرض هذا الشكل هو الـ **Mask**.

فهى لا تكتفى بعرض الشكل فقط، بل تمنع أى إدخال لا يطابقه.

مثلاً إذا كانت الـ Mask تسمح بالأرقام فقط، فلن تستطيع كتابة أى حرف.

ولو كانت الـ Mask الخاصة بالتاريخ تتطلب وجود الشرطة المائلة `/` ، فستفرض عليك الكتابة بهذا الشكل.

إذن:

> **الـ Data Format يوضح شكل الإدخال، أما الـ Mask فهى التى تفرض هذا الشكل وتمنع أى إدخال مخالف له.**

إذن وظيفة هذا العمود

أن يريك:

> شكل البيانات الذى سيلتزم به المستخدم أثناء الكتابة.

العمود الثالث

Validation Type

الترجمة

```
Validation Type
```

تعنى:

> **نوع التحقق.**

وظيفته

بعد أن ينتهى المستخدم من الكتابة...

يسأل البرنامج:

> **هل أريد أن أتحقق من نوع هذه البيانات؟**

مثال

اخترت:

```
Short Date
```

ستجد:

```
DateTime
```

لأن البيانات المفروض تكون:

> تاريخًا صحيحًا.

فإذا كتب المستخدم:

```
31/02/2026
```

فالـ Mask ستسمح بها لأن الشكل صحيح.

لكن:

```
DateTime
```

سيكتشف أن هذا التاريخ غير صحيح.

مثال آخر

اخترت:

```
Numeric (5-digits)
```

ستجد:

```
Int32
```

لأن البرنامج يريد التأكد أن القيمة تمثل:

عددًا صحيحًا من النوع `Int32` .

فقد تكون القيمة كلها أرقام، لكن إذا كانت أكبر من المدى الذى يسمح به `Int32` فسيفشل التحقق.

مثال ثالث

اخترت:

```
Phone Number
```

ستجد:

```
None
```

لماذا؟

لأن رقم الهاتف ليس تاريخًا.

وليس عددًا ستجرى عليه عمليات حسابية.

هو مجرد بيانات مكتوبة بشكل معين.

إذن لا يوجد نوع معين يريد البرنامج التحقق منه.

هل None تعنى أن المستخدم يستطيع كتابة أى شيء؟

لا.

وهذه نقطة مهمة جدًا.

فالـ **Mask** ما زالت تمنع أى إدخال مخالف.

مثلاً إذا كانت الـ Mask تسمح بالأرقام فقط، فلن تستطيع كتابة حرف.

إذن:

- الذى يمنع الحروف هو **الـ Mask**.
- أما **None** فتعنى فقط:

> "لا أريد التحقق من نوع البيانات بعد الانتهاء من الإدخال."

العلاقة بين الأعمدة الثلاثة

لنفرض أنك اخترت هذا الصف:

Mask Description	Data Format	Validation Type
Short Date	12/11/2003	DateTime

معناه:

- اسم هذا القالب هو **Short Date**.
- المستخدم سيكتب البيانات بهذا الشكل **12/11/2003**.
- أثناء الكتابة ستفرض الـ **Mask** هذا الشكل وتمنع أى إدخال مخالف.
- وبعد الانتهاء، سيتحقق البرنامج أنها تمثل **تاريخًا صحيحًا**.

ولو اخترت:

Mask Description	Data Format	Validation Type
Phone Number	555-0123 (574)	None

فالمعنى:

- هذا قالب لرقم الهاتف.
- المستخدم سيكتب بهذا الشكل.
- أثناء الكتابة ستفرض الـ **Mask** هذا الشكل.
- وبعد الانتهاء، لن يحاول البرنامج تحويل القيمة إلى تاريخ أو عدد، بل سيكتفى بأن الإدخال مطابق للـ Mask.

مثال من الحياة

تخيل أنك ستملأ استمارة حكومية.

أولًا

الموظف يسألك:

> ماذا ستكتب؟

فتقول:

> تاريخ الميلاد.

وهذا يمثل:

```
Mask Description
```

ثانيًا

الموظف يعطيك النموذج ويقول:

> اكتب بهذه الطريقة:

```
__/__/____
```

وهذا يمثل:

```
Data Format
```

كما أن النموذج نفسه لن يسمح لك بالكتابة فى غير الأماكن المخصصة، وهذا يشبه دور الـ **Mask** فى فرض شكل الإدخال.

ثالثًا

بعد أن تنتهى...

الموظف ينظر إلى الورقة ويقول:

> هل هذا تاريخ صحيح بالفعل؟

وهذا يمثل:

```
Validation Type
```

جدول الملخص

العمود	الترجمة	وظيفته
Mask Description	وصف أو اسم الـ Mask	يحدد اسم القالب الجاهز الذى ستستخدمه.
Data Format	شكل أو صيغة البيانات	يوضح للمستخدم الشكل الذى يجب أن يكتب به البيانات، والـ **Mask** هى التى تفرض هذا الشكل وتمنع أى إدخال مخالف.
Validation Type	نوع التحقق	بعد انتهاء الإدخال، يحدد هل سيتحقق البرنامج من نوع البيانات (`DateTime`، `Int32`) أم لن يتحقق (`None`).

أسهل طريقة لحفظ الفرق

تخيلها كأنها ثلاث أسئلة متتالية:

1. **Mask Description → "أنا أملأ بيانات إيه؟"**
   - تاريخ؟
   - رقم هاتف؟
   - وقت؟
2. **Data Format → "هكتبها إزاى؟"**
   - `12/11/2003`
   - `555-0123 (574)`

   والـ **Mask** هى التى ستجبرنى على الالتزام بهذا الشكل.
3. **Validation Type → "بعد ما أكتبها، هل البرنامج هيتأكد إنها فعلًا من النوع ده؟"**
   - `DateTime`
   - `Int32`
   - أو `None` (لا يوجد تحقق إضافى من نوع البيانات).

الفرق بين `None` و `Int32` و `DateTime`

بعد أن ينتهى المستخدم من إدخال البيانات، يأتى دور **Validation Type**.

وهنا يسأل البرنامج نفسه:

> **هل أكتفى بأن شكل الإدخال صحيح؟ أم أتحقق أيضًا أن نوع البيانات صحيح؟**

وهنا يظهر الفرق بين `None` و `Int32` و `DateTime` .

أولًا: `None`

تعنى:

> **لا يوجد تحقق إضافى من نوع البيانات بعد انتهاء الإدخال.**

فالبرنامج سيكتفى بأن المستخدم التزم بشكل الإدخال الذى فرضته الـ **Mask**.

مثال

لنفترض أن عندك Mask لرقم هاتف:

```
(000) 000-0000
```

المستخدم كتب:

```
(123) 456-7890
```

الإدخال مطابق للـ Mask.

إذن البرنامج سيقبله وينتهى الأمر.

لن يسأل نفسه:

- هل هذا رقم هاتف حقيقى؟
- هل هذا عدد سأستخدمه فى العمليات الحسابية؟

لأن:

```
Validation Type = None
```

يعنى:

> **لا أريد أى تحقق إضافى بعد انتهاء الإدخال.**

ثانيًا: `Int32`

تعنى:

> بعد انتهاء الإدخال، سيحاول البرنامج التأكد أن القيمة يمكن تحويلها إلى عدد صحيح من النوع `Int32` .

قد تسأل:

> **طالما الـ Mask أصلاً تسمح بالأرقام فقط، فلماذا أتحقق مرة أخرى؟**

الإجابة:

لأن الـ Mask تتحقق من **شكل الإدخال فقط**.

أما `Int32` فيتحقق من **نوع البيانات** وهل يمكن تخزينها داخل متغير من النوع `int` .

مثال عملى

لنفرض أن الـ Mask تسمح بكتابة حتى 20 رقمًا.

كتب المستخدم:

```
99999999999999999999
```

الـ Mask ستوافق.

لماذا؟

لأنها قالت فقط:

> "المطلوب أرقام."

وهذا الشرط تحقق بالفعل.

لكن عندما يأتى دور:

```
Validation Type = Int32
```

سيحاول البرنامج تنفيذ شىء يشبه:

```
#C ‹›
int number = Convert.ToInt32(maskedTextBox1.Text);
```

وهنا سيحدث خطأ.

لماذا؟

لأن هذا الرقم أكبر من أكبر قيمة يستطيع النوع `Int32` تخزينها.

إذن:

- الـ Mask قالت: ✔ الشكل صحيح.
- أما `Int32` فقال: ❌ لا أستطيع تحويل هذه القيمة إلى `int` .

وهذا هو سبب وجود `Int32` .

فهو لا يتحقق أن المدخل أرقام فقط، بل يتحقق أيضًا أن هذه الأرقام **صالحة كعدد صحيح من النوع** `Int32` .

ثالثًا: `DateTime`

تعنى:

> بعد انتهاء الإدخال، سيحاول البرنامج التأكد أن القيمة تمثل تاريخًا صحيحًا.

وهنا أيضًا يوجد فرق بين دور الـ Mask ودور `DateTime` .

مثال

لنفترض أن الـ Mask هى:

```
00/00/0000
```

كتب المستخدم:

```
31/02/2026
```

الـ Mask ستقبلها.

لماذا؟

لأنها لا تعرف شيئًا عن الشهور وعدد الأيام.

كل ما تعرفه هو:

- رقمان.
- ثم `/` .
- ثم رقمان.
- ثم `/` .
- ثم أربعة أرقام.

وهذا ما كتبه المستخدم.

لكن عندما يأتى دور:

```
Validation Type = DateTime
```

سيحاول البرنامج تنفيذ شىء يشبه:

```C#
DateTime date = Convert.ToDateTime(maskedTextBox1.Text);
```

وسيكتشف أن:

```
31/02/2026
```

ليس تاريخًا صحيحًا.

لأن شهر فبراير لا يحتوى على 31 يومًا.

إذن:

- الـ Mask قالت: ✔ الشكل صحيح.
- أما `DateTime` فقال: ❌ هذا ليس تاريخًا صحيحًا.

الفكرة التى يجب أن تحفظها

الـ **Mask** تهتم بـ:

> **كيف يكتب المستخدم البيانات؟**

أما **Validation Type** فيهتم بـ:

> **هل البيانات التى كتبها المستخدم تمثل فعلًا النوع المطلوب؟**

جدول صغير للمقارنة

Validation Type	ماذا يتحقق؟	مثال
None	لا يوجد تحقق إضافى، يكتفى بأن الإدخال مطابق للـ Mask.	رقم هاتف.
Int32	يتحقق أن القيمة يمكن تحويلها إلى عدد صحيح من النوع `Int32`.	يرفض رقمًا ضخمًا لا يمكن تخزينه فى `int`.
DateTime	يتحقق أن القيمة تمثل تاريخًا صحيحًا.	يرفض `31/02/2026` رغم أن شكلها صحيح.

أسهل طريقة لحفظها

- **Mask** → *"هل كتبت بالشكل المطلوب؟"* ✅
- **None** → *"تمام، لا أريد أى فحص إضافى."*
- **Int32** → *"هل هذا يصلح أن يكون `int`؟"*
- **DateTime** → *"هل هذا تاريخ حقيقى؟"*

مهم

> البيانات التى أكتبها داخل الـ MaskedTextBox تكون فى الأصل String لأن خاصية `Text` ترجع String دائمًا.
>
> إذا اخترت `ValidationType = Int32` أو `DateTime` ، فإن WinForms يحاول بعد انتهاء الإدخال تحويل هذا الـ String إلى النوع الذى اخترته ليتأكد أنه صالح.
>
> أما إذا كان `ValidationType = None` ، فلا يحاول تحويله إلى أى نوع، ويظل مجرد String كما هو.

س: اشرح الـ Custom Mask فى MaskedTextBox شرحًا كاملًا، مع شرح جميع رموز الـMask ، وترجمة كل رمز، ووظيفته، وأمثلة عليه، وجدول ملخص فى النهاية.

Custom Mask

عندما تختار **Custom**، فأنت لا تستخدم قالبًا جاهزًا من Microsoft.

بل أنت الذي تكتب الـ **Mask** بنفسك.

مثال:

```C#
MaskedTextBox1.Mask = "00/00/0000";
```

هنا أنت تقول للـ `MaskedTextBox` :

> "اسمح للمستخدم أن يكتب البيانات بهذا الشكل فقط."

وسيظهر للمستخدم:

```
__/__/____
```

ولا يستطيع الكتابة إلا بما يتوافق مع هذا الشكل.

كيف يقرأ البرنامج الـ Mask؟

البرنامج يقرأها **حرفًا حرفًا**.

مثلًا:

```
00/00/0000
```

يفسرها هكذا:

الرمز	المعنى
0	يجب إدخال رقم
0	يجب إدخال رقم
/	ضع علامة / ثابتة
0	يجب إدخال رقم
0	يجب إدخال رقم
/	ضع علامة / ثابتة
0	يجب إدخال رقم
0	يجب إدخال رقم
0	يجب إدخال رقم
0	يجب إدخال رقم

لذلك المستخدم يرى:

```
__/__/____
```

رموز الـ Mask

0

```
0 - Digit, required.
Value between 0 and 9.
```

الترجمة

رقم **إجباري**.

المعنى

لا بد أن يكتب المستخدم رقمًا من:

```
0 → 9
```

ولا يستطيع ترك المكان فارغًا.

مثال

```
Mask

0000
```

يسمح بـ:

```
1234
9876
0001
```

ولا يسمح بـ:

```
12
```

لأن الرقمين الآخرين مطلوبان.

9

```
9 – Digit or space, optional.
```

الترجمة

رقم أو فراغ **اختياري**.

المعنى

يمكن للمستخدم أن يكتب رقمًا.

أو يترك المكان فارغًا.

إذا ترك الخانة فارغة فلن يحدث خطأ، لأن هذه الخانة اختيارية.

مثال

```
9999
```

يقبل:

```
12
```

ويقبل أيضًا:

```
1234
```

لأن كل خانة اختيارية.

```
# - Digit or space, optional.
If this position is blank in the mask, it will be rendered as a space in the Text property.
```

الترجمة

رقم أو فراغ **اختياري**.

يشبه جدًا:

```
9
```

لكن هناك فرق صغير داخليًا.

إذا تُركت هذه الخانة فارغة، فستظهر كمسافة (**Space**) داخل خاصية:

```
#C ‹›
Text
```

أما الرمز `9` فلا يذكر هذا السلوك.

وهذا الفرق غالبًا لن تحتاجه في البداية.

L

```
L - Letter, required.
Restricts input to the ASCII letters a-z and A-Z.
```

الترجمة

حرف **إجباري**.

المعنى

لا يقبل إلا الحروف الإنجليزية:

```
A-Z
a-z
```

ولا يقبل أرقامًا.

والمقصود بـ **ASCII letters** هو الحروف الإنجليزية فقط، لذلك لن يقبل الحروف العربية أو أى حروف من لغات أخرى.

مثال

```
LLL
```

يقبل:

```
ABC
```

ولا يقبل:

```
A12
```

ولا يقبل:

```
أحمد
```

لأنها ليست حروفًا إنجليزية.

?

```
? - Letter, optional.
Restricts input to the ASCII letters a-z and A-Z.
```

الترجمة

حرف **اختياري**.

يمكن كتابة حرف.

أو ترك المكان فارغًا.

وإذا كتب المستخدم حرفًا، فلا بد أن يكون حرفًا إنجليزيًا فقط:

```
A-Z
a-z
```

ولا يقبل الحروف العربية أو الأرقام.

&

```
& - Character, required.
```

الترجمة

أي محرف **إجباري**.

المعنى

يعني يقبل:

- حرف.
- رقم.
- رمز.

لكن لا يمكن تركه فارغًا.

مثال

لو كانت الـ Mask:

```
&&&
```

فيقبل:

```
A1@
```

أو

```
5$B
```

لأن أى محرف مسموح به.

C

```
C - Character, optional.
Any non-control character.
```

الترجمة

أي محرف **اختياري**.

مثل:

```
&
```

لكن يمكن تركه فارغًا.

والمقصود بـ:

```
Any non-control character
```

هو أى محرف يمكن للمستخدم كتابته على لوحة المفاتيح، مثل:

- الحروف.
- الأرقام.
- الرموز.

أما محارف التحكم (**Control Characters**) مثل:

- Enter
- Tab
- Backspace

فهذه ليست بيانات يكتبها المستخدم داخل الـ Mask، لذلك لا تُعتبر إدخالًا صالحًا.

A

```
A - Alphanumeric, required.
```

الترجمة

حرف أو رقم **إجباري**.

كلمة **Alphanumeric** تعنى:

> Alphabet + Numeric

أى:

> حرف أو رقم.

مثال

```
AAA
```

يقبل:

```
A12
```

أو

```
AB3
```

لكن لا يقبل:

```
@#$
```

لأن الرموز ليست حروفًا أو أرقامًا.

a

```
a - Alphanumeric, optional.
```

نفس:

```
A
```

لكن اختياري.

يمكن كتابة حرف أو رقم.

ويمكن أيضًا ترك الخانة فارغة.

.

```
. - Decimal placeholder.
```

الترجمة

مكان مخصص للفاصل العشري.

المعنى

هذا الرمز يمثل مكان العلامة العشرية.

وغالبًا تكون:

```
.
```

فى بعض الدول.

أو:

```
,
```

فى دول أخرى حسب إعدادات النظام (**Regional Settings**).

لذلك يسمى:

```
Placeholder
```

أى:

> **مكان مخصص** يضع فيه البرنامج الفاصل العشرى المناسب.

مثال

قد يظهر للمستخدم:

```
123.45
```

أو حسب إعدادات النظام:

```
123,45
```

,

```
, - Thousands placeholder.
```

الترجمة

مكان مخصص لفاصل الآلاف.

المعنى

يمثل الفاصل الذى يفصل بين الآلاف.

مثل:

```
1,000
```

أو:

```
1,000,000
```

وكذلك يعتمد شكل الفاصل على إعدادات النظام.

لذلك هو أيضًا يسمى:

```
Placeholder
```

أى مكان مخصص يضع فيه البرنامج فاصل الآلاف المناسب.

:

```
: - Time separator.
```

الترجمة

فاصل الوقت.

المعنى

يمثل العلامة التى تفصل بين:

الساعات والدقائق.

مثل:

```
12:30
```

وفى بعض إعدادات النظام قد يختلف شكل الفاصل، لذلك يستخدم البرنامج الفاصل المناسب تلقائيًا.

/

```
/ - Date separator.
```

الترجمة

فاصل التاريخ.

المعنى

يمثل العلامة التى تفصل بين:

- اليوم.
- الشهر.
- السنة.

مثل:

```
25/12/2026
```

وفى بعض إعدادات النظام قد يستخدم البرنامج علامة مختلفة تلقائيًا.

$

```
$ - Currency symbol.
```

الترجمة

رمز العملة.

المعنى

يمثل رمز العملة.

مثل:

```
$150
```

وفى بعض الحالات قد يعرض رمز العملة المناسب حسب إعدادات النظام.

>

```
< - Shift down.
Converts all characters that follow to lowercase.
```

الترجمة

تحويل الأحرف التالية إلى **حروف صغيرة**.

المعنى

بعد كتابة الرمز:

```
<
```

فإن جميع الأحرف التى يكتبها المستخدم بعده تتحول تلقائيًا إلى:

```
lowercase
```

حتى لو كتبها بحروف كبيرة.

مثال

```
<LLLL
```

إذا كتب المستخدم:

```
ABCD
```

ستصبح:

```
abcd
```

>

```
> - Shift up.
Converts all characters that follow to uppercase.
```

الترجمة

تحويل الأحرف التالية إلى **حروف كبيرة**.

المعنى

بعد كتابة:

```
>
```

فإن جميع الأحرف التالية تتحول تلقائيًا إلى:

```
Uppercase
```

مثال

```
>LLLL
```

إذا كتب المستخدم:

```
abcd
```

ستصبح:

```
ABCD
```

|

```
| - Disable a previous shift up or shift down.
```

الترجمة

إلغاء تأثير التحويل السابق.

المعنى

إذا كنت قد استخدمت:

```
<
```

أو

```
>
```

ثم أردت إيقاف تأثيرهما، تستخدم:

```
|
```

بعده يعود البرنامج إلى الكتابة العادية دون تحويل الأحرف إلى كبيرة أو صغيرة.

مثال

```
>LL|LL
```

إذا كتب المستخدم:

```
abcd
```

سيصبح أول حرفين:

```
AB
```

ثم بعد:

```
|
```

لن يتم التحويل بعد ذلك، فيبقى الحرفان التاليان كما كتبهما المستخدم.

\

```
\ - Escape.
Escapes a mask character, turning it into a literal.
"\\" is the escape sequence for a backslash.
```

الترجمة

محرف الهروب (**Escape Character**).

المعنى

بعض الرموز داخل الـ Mask لها معنى خاص.

مثل:

```
0
L
A
#
/
:
$
```

إذا أردت أن تعتبر أحد هذه الرموز **حرفًا عاديًا** وليس أمرًا من أوامر الـ Mask، تستخدم قبله:

```
\
```

وهذا يجعل البرنامج يتعامل معه كنص ثابت (**Literal**).

مثال

بدل أن تعتبر:

```
0
```

رقمًا مطلوبًا.

يمكنك كتابة:

```
\0
```

فيظهر للمستخدم الرقم:

```
0
```

كنص ثابت.

ماذا لو أردت كتابة Backslash نفسه؟

تكتب:

```
\\
```

لأن:

```
\
```

له وظيفة خاصة، ولذلك تحتاج إلى كتابته مرتين حتى يظهر للمستخدم كحرف عادى.

أي حرف آخر

```
All other characters - Literals.
All non-mask elements will appear as themselves within MaskedTextBox.
Literals always occupy a static position in the mask at run time, and cannot be moved or de
```

الترجمة

أى أحرف أخرى تعتبر **Literals**.

ما معنى Literal؟

كلمة:

```
Literal
```

تعنى:

> **حرف ثابت** أو **نص ثابت**.

أى أن البرنامج لا يعتبره أمرًا من أوامر الـ Mask.

بل يعرضه كما هو.

المعنى

إذا كتبت داخل الـ Mask أى حرف ليس له معنى خاص، مثل:

```
EMP-
```

فالبرنامج سيعرضه للمستخدم كما هو.

مثال

```
EMP-0000
```

سيظهر:

```
EMP-____
```

ويكتب المستخدم الأرقام فقط.

ماذا يقصد بـ:

```
Static position
```

يعنى:

> **مكان ثابت**.

فهذه الأحرف تظل فى أماكنها دائمًا أثناء تشغيل البرنامج.

ماذا يقصد بـ:

```
Run time
```

يعنى:

> **أثناء تشغيل البرنامج.**

هل يستطيع المستخدم حذف هذه الأحرف؟

لا.

لأنها جزء من الـ Mask نفسها.

ولهذا قال:

```
cannot be moved or deleted by the user
```

أى:

> لا يستطيع المستخدم تحريكها أو حذفها.

فلو كانت الـ Mask:

```
EMP-0000
```

فالمستخدم لا يستطيع حذف:

```
EMP-
```

بل يبدأ الكتابة بعد الشرطة مباشرة.

جدول الملخص

الرمز	المعنى
`0`	رقم إجباري (من 0 إلى 9)
`9`	رقم أو فراغ اختياري
`#`	رقم أو فراغ اختياري، وإذا تُرك فارغًا يظهر كمسافة داخل `Text`
`L`	حرف إنجليزي (ASCII) إجباري
`?`	حرف إنجليزي (ASCII) اختياري
`&`	أي محرف إجباري
`C`	أي محرف اختياري (عدا محارف التحكم)
`A`	حرف أو رقم إجباري
`a`	حرف أو رقم اختياري
`.`	مكان مخصص للفاصل العشري
`,`	مكان مخصص لفاصل الآلاف
`:`	مكان مخصص لفاصل الوقت
`/`	مكان مخصص لفاصل التاريخ
`$`	رمز العملة
`>`	تحويل الأحرف التالية إلى صغيرة (Lowercase)
`<`	تحويل الأحرف التالية إلى كبيرة (Uppercase)
`|`	إلغاء تأثير `>` أو `<`
`\`	يجعل رمز الـ Mask يُعامل كنص ثابت (Literal)
أي حرف آخر	يظهر كنص ثابت داخل الـ Mask ولا يستطيع المستخدم حذفه أو تحريكه أثناء تشغيل البرنامج

ComboBox

ComboBox

يسمى أيضًا:

> **Drop-Down List**

لكن كما اتفقنا قبل ذلك:

> الأستاذ يقصد **المعنى العام أو الدارج**، وليس المعنى البرمجي الحرفي.

فالاسم الحقيقي للكنترول هو:

```
ComboBox
```

أما:

```
Drop-Down List
```

فهي مجرد طريقة لوصفه لأنه يعرض قائمة منسدلة.

ما هو الـ ComboBox؟

الـ **ComboBox** هو كنترول يسمح للمستخدم باختيار **عنصر واحد فقط** من مجموعة عناصر.

مثال:

```
Small
Medium
Large
```

أو

```
C#
C++
Java
Python
```

المستخدم يختار عنصرًا واحدًا فقط.

لماذا نستخدمه؟

بدلًا من كتابة الاختيارات فى عدة RadioButtons.

بدلًا من:

```
o Small
o Medium
o Large
```

يمكن وضعها داخل ComboBox واحدة.

فتصبح الشاشة أكثر ترتيبًا وأقل ازدحامًا.

كيف نضيف عناصر إلى الـ ComboBox؟

يوجد طريقتان.

الطريقة الأولى

Design Time

أى أثناء تصميم البرنامج.

نحدد الـ ComboBox.

ثم نفتح:

```
Properties
```

ثم نجد:

```
Items
```

أو:

```
Items Collection
```

ثم نضغط عليها.

ونبدأ بإضافة العناصر.

مثلاً:

```
Option 1
Option 2
Option 3
```

ثم نضغط:

```
OK
```

وبذلك أصبحت العناصر موجودة داخل الـ ComboBox.

الطريقة الثانية

Run Time

أى أثناء تشغيل البرنامج.

عن طريق الكود.

مثل:

```
#C
comboBox1.Items.Add("F#");
```

نشرحها

```
#C
comboBox1
```

الكنترول الذى سنضيف إليه.

```
#C
Items
```

مجموعة العناصر الموجودة داخله.

```
Add()
```

تعنى:

> أضف عنصرًا جديدًا.

إذن:

```
comboBox1.Items.Add("F#");
```

تعنى:

> أضف العنصر:

```
F#
```

إلى عناصر الـ ComboBox.

وفى المثال الموجود:

```
private void button1_Click(object sender, EventArgs e)
{
    comboBox1.Items.Add("F#");
}
```

كلما ضغط المستخدم على الزر...

سيتم إضافة:

```
F#
```

إلى القائمة.

خاصية مهمة

DropDownStyle

تحدد طريقة عمل الـ ComboBox.

يوجد لها عدة قيم.

وأهم قيمتين ذكرهما الأستاذ هما:

DropDown

وهى القيمة الافتراضية.

تعنى:

> يستطيع المستخدم أن يختار من القائمة.

وأيضًا:

> يستطيع أن يكتب بنفسه داخل الـ ComboBox.

مثلاً:

```
▼ C#
```

يمكن اختيار:

```
Java
```

أو كتابة:

```
Python
```

حتى لو لم تكن موجودة.

DropDownList

وهنا:

لا يستطيع المستخدم الكتابة.

بل يجب أن يختار عنصرًا موجودًا بالفعل داخل القائمة.

مثلاً:

```
▼
```

يفتح القائمة.

ويختار:

```
Option 1
```

أو

```
Option 2
```

ولا يمكنه كتابة عنصر جديد.

ولهذا يستخدمها كثير من المبرمجين عندما يريدون إجبار المستخدم على الاختيار من العناصر الموجودة فقط.

💡 **معلومة إضافية**

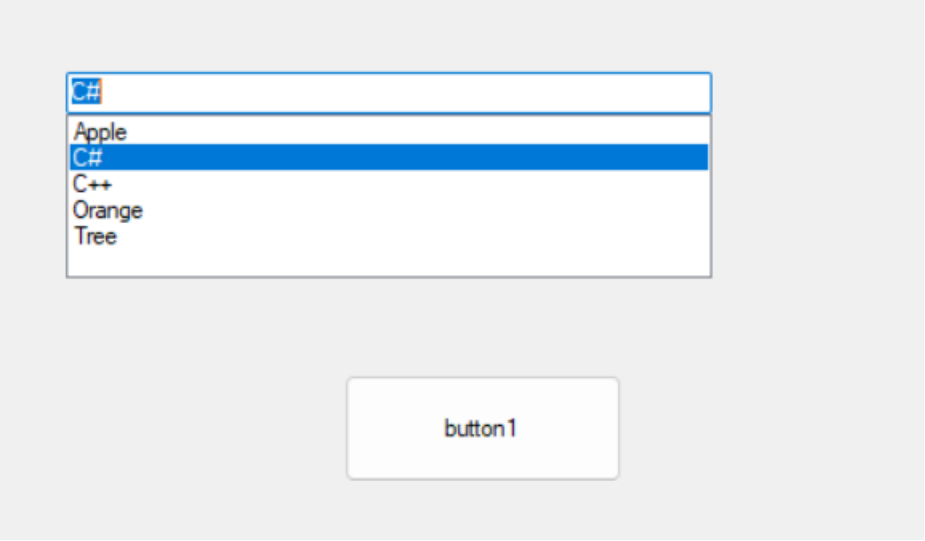
يوجد قيمة ثالثة لم يذكرها الأستاذ، وهى:

Simple

فى هذه الحالة:

- تظل القائمة **مفتوحة دائمًا**.
- لا يحتاج المستخدم للضغط على السهم لرؤية العناصر.
- ويمكنه أيضًا الكتابة داخل الـ ComboBox.

فيصبح شكلها تقريبًا هكذا:



وهذه القيمة نادرًا ما تُستخدم مقارنةً بـ:

- DropDown
- DropDownList

معلومة جميلة جدًا هتخليك تفتكرها

تخيل إن الـ ComboBox عبارة عن:

DropDown

```
TextBox
   +
عند الضغط ListBox زر يفتح
```

DropDownList

```
TextBox (للقراءة فقط)
   +
عند الضغط ListBox زر يفتح
```

Simple

```
TextBox
+
ListBox
```

الاتنين معروضين فى نفس الوقت.

وده أصل التسمية **ComboBox**.

لأنه عبارة عن **Combination** (دمج) بين:

- `TextBox`
- `ListBox`

ولذلك اسمه:

> **ComboBox = Combination Box**

💡 **معلومة إضافية:** دى من أجمل الطرق لحفظ اسمه. فهو لم يُسمَّ "Combo" عشوائيًا، بل لأنه يجمع وظيفتين فى كنترول واحد:

- **TextBox** لعرض أو كتابة النص.
- **ListBox** لعرض قائمة العناصر.

وده كمان يفسر ليه وضع **Simple** هو أقرب شكل يوضح أصل تسمية الـ **ComboBox**، لأنك ترى الـ TextBox والـ ListBox معًا فى نفس الوقت.

خاصية أخرى مهمة

Sorted

ترجمتها:

> **مرتب**.

نوعها:

```
bool
```

أى:

```
True
False
```

إذا كانت:

```
False
```

فستظل العناصر بنفس ترتيب إضافتها.

مثلاً:

```
Java
C#
Python
```

ستظل كما هى.

أما إذا كانت:

```
True
```

فسيقوم الـ ComboBox بترتيب العناصر تلقائيًا.

مثلاً لو أضفت:

```
Java
C#
Python
```

ستصبح:

```
C#
Java
Python
```

💡 **معلومة إضافية**

يقوم الـ ComboBox بترتيب العناصر **ترتيبًا أبجديًا (Alphabetically)**، وليس حسب ترتيب إضافتها.

فإذا كانت العناصر:

```
Ali
Ahmed
Mohamed
Youssef
```

فستصبح:

```
Ahmed
Ali
Mohamed
Youssef
```

لأنها تُرتب أبجديًا.

Event مهمة

SelectedIndexChanged

ترجمتها الحرفية:

> **العنصر (أو الفهرس) المحدد تغير.**

والمقصود:

> يعمل هذا الـ Event عندما يغير المستخدم العنصر المختار داخل الـ ComboBox.

💡 **معلومة إضافية**

اسم الـ Event مكون من جزئين:

```
SelectedIndex
```

ويعنى:

> **العنصر (أو الفهرس) المحدد.**

و

```
Changed
```

وتعنى:

> **تغير.**

لذلك سمى بهذا الاسم لأنه يعمل عندما **يتغير العنصر المحدد** داخل الـ ComboBox.

وهو نفس المنطق الذى رأيناه مع باقى الـ Events.

مثل:

```
Button_Click
```

لأنه يعمل عند الضغط على الزر.

و

```
TextChanged
```

لأنه يعمل عند تغير النص.

و

```
CheckedChanged
```

لأنه يعمل عند تغير حالة الـ CheckBox أو الـ RadioButton.

الكود:

```C#
private void comboBox1_SelectedIndexChanged(object sender, EventArgs e)
{
    MessageBox.Show(comboBox1.Text);
}
```

كلما اختار المستخدم عنصرًا جديدًا...

يتم استدعاء هذا الـ Event.

ثم ينفذ:

```C#
MessageBox.Show(comboBox1.Text);
```

أى يعرض العنصر الذى اختاره المستخدم.

مثال

لو كانت العناصر:

```
C#
C++
Java
```

واختار المستخدم:

```
Java
```

ستظهر رسالة:

```
Java
```

خاصية مهمة جدًا

SelectedIndex

ترجمتها:

> **رقم العنصر المحدد.**

كل عنصر داخل الـ ComboBox له رقم يبدأ من:

```
0
```

مثل الـ Array تمامًا.

مثال:

العنصر	SelectedIndex
Option 1	0
Option 2	1
Option 3	2

إذا كتبت:

```C#
comboBox1.SelectedIndex = 1;
```

فأنت تقول للبرنامج:

> اختر العنصر الذى رقمه:

```
1
```

وسيكون:

```
Option 2
```

لماذا وضع الأستاذ هذا الكود داخل Form_Load؟

```C#
private void Form1_Load(object sender, EventArgs e)
{
    comboBox1.SelectedIndex = 1;
}
```

لأن:

```
Form_Load
```

يعمل عند تحميل الفورم لأول مرة.

فبمجرد فتح النافذة...

سيختار البرنامج تلقائيًا:

```
Option 2
```

لأن رقمها:

```
1
```

ولاحظ الأستاذ أن:

بمجرد تنفيذ:

```C#
SelectedIndex = 1;
```

تغير العنصر المحدد.

إذن تم تشغيل:

```
SelectedIndexChanged
```

ولذلك ظهرت الـ MessageBox مباشرة.

وهذه نقطة مهمة جدًا:

> سواء تغير الـ SelectedIndex بواسطة **المستخدم** أو بواسطة **الكود**، فإن `SelectedIndexChanged` Event ستعمل لأن العنصر المحدد قد تغير.

💡 معلومة إضافية

إذن المهم ليس **من الذى غيّر العنصر**.

سواء:

- اختاره المستخدم بيده.
- أو قام الكود بتغييره.

فطالما أن **العنصر المحدد قد تغير**، سيتم تشغيل:

```
SelectedIndexChanged
```

💡 معلومة إضافية

لاحظ أن الأستاذ استخدم داخل الـ Event:

```C#
comboBox1.Text
```

ولم يستخدم:

```C#
comboBox1.SelectedIndex
```

وذلك لأن:

```
Text
```

تعيد **النص الخاص بالعنصر المختار حاليًا**.

أما:

```
SelectedIndex
```

فتعيد **رقم العنصر المختار**.

مثال:

إذا كان العنصر المختار هو:

```
Java
```

فإن:

```
#C
comboBox1.Text
```

سترجع:

```
Java
```

أما:

```
#C
comboBox1.SelectedIndex
```

فسترجع:

```
2
```

لأن Java هى العنصر الثالث، والترقيم يبدأ من الصفر.

ملخص الدرس

العنصر	وظيفته
ComboBox	كنترول لاختيار عنصر واحد من مجموعة عناصر.
Items	مجموعة العناصر الموجودة داخل الـ ComboBox.
()Items.Add	إضافة عنصر جديد أثناء تشغيل البرنامج.
DropDown	يسمح بالاختيار والكتابة.
DropDownList	يسمح بالاختيار فقط، ولا يسمح بالكتابة.
Simple	يعرض القائمة مفتوحة دائمًا، ويسمح أيضًا بالكتابة.
Sorted	يرتب العناصر تلقائيًا إذا كانت قيمتها `True` .
SelectedIndex	رقم العنصر المختار، ويبدأ من 0.
SelectedIndexChanged	يعمل عند تغير العنصر المحدد، سواء بواسطة المستخدم أو بواسطة الكود.
Text	يعيد النص الخاص بالعنصر المختار حاليًا.

الخلاصة التى أحب أن تحفظها

1. **ComboBox** يستخدم لاختيار عنصر واحد من عدة عناصر.
2. يمكن إضافة العناصر أثناء **التصميم (Design Time)** أو أثناء **تشغيل البرنامج (Run Time)**.
3. إذا كانت `DropDownStyle = DropDown` يمكن للمستخدم **الاختيار أو الكتابة**.
4. إذا كانت `DropDownStyle = DropDownList` يمكن للمستخدم **الاختيار فقط**.
5. 💡 توجد قيمة ثالثة هى `Simple` ، وتعرض القائمة مفتوحة دائمًا مع السماح أيضًا بالكتابة.
6. `Sorted = True` تجعل العناصر تُرتب تلقائيًا.
7. `SelectedIndex` يمثل **رقم العنصر المختار** ويبدأ من **0**.
8. عند تغير العنصر المختار تعمل الـ `SelectedIndexChanged` Event سواء كان التغيير من المستخدم أو من الكود.
9. `comboBox1.Text` يعيد **النص الخاص بالعنصر المختار حاليًا**، بينما `SelectedIndex` يعيد **رقم العنصر المختار**.

Combo Box Exercise (Solution)

جدول الملخص

الخاصية / الدالة	ماذا ترجع؟	مثال إذا كان المختار `Java`
`SelectedIndex`	رقم العنصر المختار	`2`
`SelectedItem`	العنصر المختار نفسه (Object)	`Java`
`()ToString`	يحول العنصر إلى String	`"Java"`
`()ToLower`	يحول النص إلى أحرف صغيرة	`"java"`

أسهل طريقة لحفظ الفرق

- `SelectedIndex` → **"رقم العنصر المختار."**
- `SelectedItem` → **"العنصر المختار نفسه."**
- `()ToString` → **"حوّل العنصر إلى نص."**
- `()ToLower` → **"حوّل النص إلى حروف صغيرة."**

> **معلومة مهمة:** إذا لم يكن هناك أي عنصر محدد داخل الـ `ComboBox`، فإن `SelectedItem` تكون قيمتها `null`. لذلك استدعاء `()ToString` مباشرة قد يسبب خطأ (`NullReferenceException`). من الأفضل التأكد أولًا أن `SelectedItem` ليست `null` إذا كان من المحتمل ألا يكون هناك عنصر مختار.

LinkLable

LinkLabel

من اسمه:

```
Link + Label
```

أى:

> **Label يظهر على هيئة رابط (Link).**

ما هو الـ LinkLabel؟

الـ **LinkLabel** هو كنترول يشبه الـ **Label** العادى جدًا.

لكن الفرق أنه يظهر للمستخدم على شكل **رابط (Hyperlink)** مثل روابط مواقع الإنترنت.

مثال:

```
Programming Advices
```

سيظهر غالبًا باللون الأزرق وتحته خط، تمامًا مثل الروابط الموجودة فى صفحات الويب.

هل هو مختلف عن الـ Label؟

فى الحقيقة...

الـ **LinkLabel** يحتوى على معظم الخصائص الموجودة فى الـ **Label** العادى.

مثل:

- Text
- Font
- AutoSize
- Enabled
- TextAlign
- RightToLeft
- وغيرها.

لكن تمت إضافة بعض الخصائص الجديدة الخاصة بالروابط.

تغيير النص

مثل أى Label.

يمكن تغيير النص من خاصية:

```
Text
```

مثلاً:

```
Programming Advices
```

فيظهر للمستخدم بهذا الشكل.

هل الـ LinkLabel يفتح موقعًا بمجرد الضغط عليه؟

الإجابة:

لا.

وهذه نقطة مهمة جدًا ذكرها الأستاذ.

مجرد أن شكله يشبه الرابط...

لا يعنى أنه يفتح موقعًا تلقائيًا.

فهو مجرد **شكل فقط**.

إذا أردت أن يفتح موقعًا...

فلا بد أن تكتب الكود بنفسك.

الأحداث (Events)

لأنه يعتبر Control من Controls الخاصة بـ Windows Forms...

فهو يمتلك معظم الـ Events المعتادة.

مثل:

```
Click
MouseEnter
MouseMove
MouseLeave
```

وغيرها.

لكن يوجد Event مخصص له وهو:

```
LinkClicked
```

وسنتحدث عنه بعد قليل.

خاصية مهمة

LinkBehavior

تحدد شكل الخط الموجود أسفل الرابط.

يوجد لها عدة قيم.

AlwaysUnderline

تعنى:

> يظهر الخط أسفل الرابط دائمًا.

مثلاً:

```
Programming Advices
```

سيظل تحته خط سواء كان مؤشر الماوس فوقه أو بعيدًا عنه.

HoverUnderline

تعنى:

> لا يظهر الخط إلا عندما يمر مؤشر الماوس فوق الرابط.

فعندما يكون الماوس بعيدًا:

```
Programming Advices
```

لن ترى الخط.

أما بمجرد الوقوف عليه بالماوس...

سيظهر الخط تحته.

NeverUnderline

تعنى:

> لا يظهر أى خط نهائيًا.

لكن يظل شكله كرابط.

فعند المرور بالماوس...

سيتغير شكل مؤشر الماوس إلى شكل اليد.

SystemDefault

تعنى:

> استخدم الشكل الافتراضى الخاص بنظام التشغيل.

أى أن Windows هو الذى يحدد طريقة عرض الرابط.

Event مهمة

LinkClicked

تعمل عندما يضغط المستخدم على الـ LinkLabel.

وهى الـ Event المناسبة إذا أردت تنفيذ أمر عند الضغط على الرابط.

مثل:

- فتح موقع.
- فتح ملف.
- فتح مجلد.
- أو تنفيذ أى كود آخر.

الكود الذى استخدمه الأستاذ

```C#
linkLabel1.LinkVisited = true;

System.Diagnostics.Process.Start("https://programmingadvices.com");
```

سنشرحه سطرًا سطرًا.

السطر الأول

```C#
linkLabel1.LinkVisited = true;
```

LinkVisited

تعنى:

> هل تمت زيارة هذا الرابط من قبل؟

عندما نجعلها:

```C#
true
```

فإن الـ LinkLabel يعتبر أن المستخدم زار الرابط.

ولهذا غالبًا سيتغير لون الرابط.

لماذا؟

لأن هذا يشبه تمامًا ما يحدث فى متصفح الإنترنت.

فعندما تزور رابطًا لأول مرة...

يكون لونه مثلًا:

```
أزرق
```

وبعد زيارته...

يتغير إلى لون آخر (غالبًا بنفسجى حسب إعدادات النظام أو البرنامج).

وهذا يساعد المستخدم على معرفة أنه فتح هذا الرابط من قبل.

💡 **معلومة إضافية**

إذا كانت:

```
#C ‹/›
LinkVisited = false;
```

فسيعتبر البرنامج أن الرابط لم تتم زيارته بعد.

💡 **معلومة إضافية**

خاصية `LinkVisited` لا تغير اللون بنفسها، لكنها **تخبر الـ LinkLabel فقط أن هذا الرابط تمت زيارته**.

بعد ذلك يقوم الـ LinkLabel بعرض اللون المناسب اعتمادًا على خصائص الألوان الخاصة به.

يوجد ثلاث خصائص مسئولة عن ألوان الرابط:

```
LinkColor
```

لون الرابط **قبل زيارته**.

مثلاً:

```
أزرق
```

```
VisitedLinkColor
```

لون الرابط **بعد زيارته**.

مثلاً:

```
بنفسجى
```

```
ActiveLinkColor
```

لون الرابط **أثناء الضغط عليه** بالماوس.

مثلاً:

```
أحمر
```

سيناريو كامل

نفترض أن لدينا الإعدادات التالية:

```
LinkColor = Blue
VisitedLinkColor = Purple
ActiveLinkColor = Red
```

وعند بداية تشغيل البرنامج يكون الكود:

```C#
linkLabel1.LinkVisited = false;
```

سيظهر الرابط للمستخدم بهذا اللون:

```
Programming Advices   (Blue)
```

لأن البرنامج يعتبر أن الرابط **لم تتم زيارته بعد**.

الآن ضغط المستخدم على الرابط، وتم تنفيذ:

```
#C ‹/›
linkLabel1.LinkVisited = true;
```

سيعتبر الـ LinkLabel أن هذا الرابط **تمت زيارته**.

لذلك سيعرضه باللون الموجود فى:

```
VisitedLinkColor
```

وليكن:

```
Purple
```

وأثناء لحظة الضغط نفسها قد يظهر باللون الموجود فى:

```
ActiveLinkColor
```

وليكن:

```
Red
```

ثم بعد انتهاء الضغط يعود إلى لون:

```
VisitedLinkColor
```

لأنه أصبح رابطًا تمت زيارته.

إذن التسلسل يكون كالتالى:

```
قبل الزيارة
        |
        ▼
LinkColor (Blue)

عند الضغط
        |
        ▼
ActiveLinkColor (Red)

بعد فتح الرابط
        |
        ▼
VisitedLinkColor (Purple)
```

وهذا هو السبب فى أن الأستاذ كتب:

```C#
linkLabel1.LinkVisited = true;
```

لكى يخبر الـ **LinkLabel** أن المستخدم **زار الرابط**، فينتقل تلقائيًا إلى استخدام اللون الموجود فى خاصية:

```
VisitedLinkColor
```

بدلًا من اللون الموجود فى:

```
LinkColor
```

السطر الثانى

```C#
System.Diagnostics.Process.Start("https://programmingadvices.com");
```

كما ذكر الأستاذ:

> هذا السطر يفتح **المتصفح الافتراضى (Default Browser)** الموجود على جهاز المستخدم، ثم يفتح الـ **URL** الذى كتبته بداخله.

فإذا كان الرابط:

```
https://programmingadvices.com
```

سيتم فتح هذا الموقع داخل المتصفح الافتراضى.

💡 معلومة إضافية

إذا أحببت أن تعرف كيف يعمل هذا السطر داخليًا:

- `System.Diagnostics` هو Namespace.
- `Process` هو Class للتعامل مع العمليات (Processes).
- `()Start` هى Method تستخدم لتشغيل برنامج أو فتح ملف أو فتح رابط.

لكن هذه التفاصيل لم يتناولها الأستاذ فى هذا الدرس، وستكون أوضح عندما تدرس الـ **Classes** و **Namespaces** بشكل أعمق.

النتيجة

عند الضغط على الـ LinkLabel...

سيحدث أمران:

1. سيتغير لون الرابط لأنه أصبح Visited.
2. سيفتح الموقع داخل المتصفح الافتراضي.

خصائص أخرى ذكرها الأستاذ

Enabled

إذا كانت:

```
True
```

يمكن الضغط على الرابط.

أما إذا كانت:

```
False
```

فسيصبح الرابط معطلًا.

TabStop

تحدد هل يمكن الوصول إلى الـ LinkLabel باستخدام زر:

```
Tab
```

أثناء التنقل بين عناصر الفورم.

Font

لتغيير:

- نوع الخط.
- حجمه.
- شكله.

ولهذا قام الأستاذ بتكبير الخط حتى يظهر الرابط بوضوح.

LinkColor

تحدد لون الرابط.

فبدلًا من اللون الأزرق...

يمكن جعله:

- أحمر
- أخضر
- أصفر

أو أى لون آخر.

خاصية مهمة

AutoSize

تشبه تمامًا الموجودة فى الـ Label.

إذا كانت:

```
True
```

سيكبر أو يصغر حجم الـ LinkLabel تلقائيًا حسب طول النص.

مثلاً:

```
Programming Advices
```

إذا أصبح النص أطول...

سيزداد حجم الـ LinkLabel تلقائيًا.

أما إذا كانت:

```
False
```

فأنت الذى تحدد حجمه بنفسك.

ولو كان النص أطول من المساحة المحددة...

فسيقوم البرنامج بلف النص (Wrapping) داخل نفس المساحة.

مثلاً:

```
Programming
Advices
```

بدلًا من أن يمتد فى سطر واحد.

💡 معلومة إضافية

هذه الخاصية لا تخص الـ LinkLabel فقط.

بل تعمل بنفس الطريقة فى الـ **Label** أيضًا، ولذلك قال الأستاذ:

> ما ينطبق على الـ LinkLabel ينطبق أيضًا على الـ Label العادى.

خصائص أخرى

ذكر الأستاذ أيضًا أن الخصائص التالية موجودة كما هى:

- TextAlign
- RightToLeft
- LeftToRight

وتعمل بنفس الطريقة التى درستها سابقًا مع الـ Label.

ملخص الدرس

الخاصية / العنصر	وظيفتها
LinkLabel	Label يظهر على شكل رابط (Hyperlink).
Text	النص الذى سيظهر داخل الرابط.
LinkBehavior	يحدد طريقة ظهور الخط أسفل الرابط.
AlwaysUnderline	الخط يظهر دائمًا.
HoverUnderline	الخط يظهر عند مرور الماوس فقط.
NeverUnderline	لا يظهر خط نهائيًا.
SystemDefault	يستخدم الشكل الافتراضى للنظام.
LinkClicked	يعمل عند الضغط على الرابط.
LinkVisited	يحدد هل تمت زيارة الرابط أم لا.
()Process.Start	يفتح الرابط باستخدام المتصفح الافتراضى.
LinkColor	يغير لون الرابط.
AutoSize	يجعل حجم الـ LinkLabel يتغير تلقائيًا حسب حجم النص.
Enabled	تفعيل أو تعطيل الرابط.
TabStop	يسمح بالوصول إليه باستخدام زر Tab.

الخلاصة التى أحب أن تحفظها

1. **LinkLabel** هو **Label** لكن يظهر على شكل رابط.
2. شكله كرابط **لا يعنى أنه يفتح موقعًا تلقائيًا**؛ يجب كتابة الكود بنفسك.
3. الخاصية **LinkBehavior** تحدد متى يظهر الخط أسفل الرابط.
4. الـ **LinkClicked** Event هى المناسبة للتعامل مع الضغط على الرابط.
5. الخاصية **LinkVisited** تجعل الرابط يظهر كرابط تمت زيارته (يتغير لونه غالبًا).
6. الدالة `()Process.Start` تستخدم لفتح الرابط فى **المتصفح الافتراضى** للجهاز.
7. معظم خصائص الـ **Label** موجودة أيضًا داخل **LinkLabel** مثل: `Text` و `Font` و `AutoSize` و `TextAlign` و `Enabled` .

كيف يحدد الـ LinkLabel اللون الذى سيظهر؟ وهل توجد خاصية Boolean لكل من LinkColor وVisitedLinkColor وActiveLinkColor؟

الإجابة

لا، **لا توجد خاصية Boolean لكل لون**.

الخاصية الوحيدة من هذا النوع هى:

```C#
LinkVisited
```

وهى من النوع:

```C#
bool
```

وقيمتها تكون:

```
true
أو
false
```

أما الخصائص:

```C#
LinkColor
VisitedLinkColor
ActiveLinkColor
```

فهى **ليست Boolean**، وإنما هى خصائص من النوع:

```C#
Color
```

أى أنها تخزن لونًا فقط.

مثلاً:

```C#
linkLabel1.LinkColor = Color.Blue;
linkLabel1.VisitedLinkColor = Color.Purple;
linkLabel1.ActiveLinkColor = Color.Red;
```

كل خاصية منها تحدد لونًا معينًا، لكن لا تحدد **متى** يستخدم هذا اللون.

الذى يحدد ذلك هو حالة الـ LinkLabel.

كيف يختار الـ LinkLabel اللون؟

الحالة الأولى

إذا كانت:

```
#C
linkLabel1.LinkVisited = false;
```

فهذا يعنى أن الرابط **لم تتم زيارته بعد**.

إذن يستخدم اللون الموجود فى:

```
LinkColor
```

الحالة الثانية

إذا أصبحت:

```
#C
linkLabel1.LinkVisited = true;
```

فهذا يعنى أن الرابط **تمت زيارته**.

إذن يستخدم اللون الموجود فى:

```
VisitedLinkColor
```

الحالة الثالثة

عندما يضغط المستخدم بالماوس على الرابط...

لا ينظر البرنامج فى هذه اللحظة إلى قيمة:

```
#C
LinkVisited
```

بل يستخدم مباشرة اللون الموجود فى:

```
ActiveLinkColor
```

لأن هذه خاصية مخصصة لحالة الضغط فقط.

بعد أن يترك المستخدم زر الماوس

تنتهى حالة الضغط.

فيعود البرنامج مرة أخرى ليفحص:

```C#
LinkVisited
```

- إذا كانت:

```C#
false
```

يستخدم:

```
LinkColor
```

- وإذا كانت:

```C#
true
```

يستخدم:

```
VisitedLinkColor
```

يمكن تخيل ما يحدث داخليًا بهذا الشكل

```C#
if (MouseIsPressed)
{
    استخدم ActiveLinkColor;
}
else
{
    if (LinkVisited)
        استخدم VisitedLinkColor;
    else
        استخدم LinkColor;
}
```

> **ملاحظة:** هذا ليس الكود الحقيقى داخل .NET، ولكنه مجرد تصور ذهنى يوضح طريقة اتخاذ القرار.

سيناريو كامل

نفترض أنك ضبطت الألوان كالتالى:

```
#C
LinkColor = Blue;
VisitedLinkColor = Purple;
ActiveLinkColor = Red;
```

وكانت:

```
#C
LinkVisited = false;
```

سيكون الرابط:

```
أزرق
```

عندما تضغط عليه بالماوس:

```
أحمر
```

لأن البرنامج يستخدم:

```
ActiveLinkColor
```

بعد أن تترك زر الماوس، وكان الكود قد نفذ:

```
#C
linkLabel1.LinkVisited = true;
```

سيصبح الرابط:

```
بنفسجى
```

لأن البرنامج عاد وفحص:

```
#C
LinkVisited
```

فوجدها:

```
true
```

فاستخدم:

```
VisitedLinkColor
```

الخلاصة التى أحب أن تحفظها

- `LinkVisited` هى **الخاصية الـ Boolean الوحيدة** التى تحدد هل الرابط تمت زيارته أم لا.
- `LinkColor` يحدد لون الرابط **قبل الزيارة**.
- `VisitedLinkColor` يحدد لون الرابط **بعد الزيارة**.
- `ActiveLinkColor` يحدد لون الرابط **أثناء الضغط عليه فقط**.
- الـ `LinkLabel` هو الذى يختار أى لون يستخدم حسب حالة الرابط، وأنت فقط تقوم بتحديد ألوان كل حالة.

CheckedListBox

CheckedListBox

يتكون الاسم من ثلاث كلمات:

```
Checked + List + Box
```

Checked

ترجمتها:

> **تم وضع علامة اختيار (✔) عليه.**

أو

> **مؤشر عليه بعلامة صح.**

List

ترجمتها:

> **قائمة.**

Box

ترجمتها:

> **صندوق.**

إذن الترجمة الحرفية لـ:

```
CheckedListBox
```

هى:

> **صندوق قائمة بعلامات اختيار.**

ويمكن تبسيط معناها إلى:

> **قائمة تحتوى على عناصر، ويوجد بجانب كل عنصر مربع اختيار (CheckBox).**

ما هو الـ CheckedListBox؟

الـ **CheckedListBox** هو كنترول يستخدم لعرض مجموعة من العناصر داخل قائمة.

وكل عنصر داخل هذه القائمة يوجد بجانبه مربع اختيار (**CheckBox**)، بحيث يستطيع المستخدم وضع علامة ✔ على عنصر أو أكثر.

مثال:

```
☐ Item1
☐ Item2
☐ Item3
☐ Item4
☐ Item5
```

فعندما يضغط المستخدم على مربع الاختيار يصبح الشكل:

```
☑ Item1
☐ Item2
☑ Item3
☐ Item4
☐ Item5
```

وبذلك يكون قد اختار أكثر من عنصر فى نفس الوقت.

كيف نضيف الـ CheckedListBox إلى الفورم؟

من نافذة:

```
ToolBox
```

نبحث عن:

```
CheckedListBox
```

ثم نقوم بسحبه وإفلاته (Drag & Drop) داخل الـ Form.

وبذلك أصبح لدينا كنترول من نوع:

```
CheckedListBox
```

جاهز للاستخدام.

كيف نضيف عناصر إلى الـ CheckedListBox؟

يوجد طريقتان.

الطريقة الأولى

Design Time

أى أثناء تصميم البرنامج.

نحدد الـ CheckedListBox.

ثم نفتح:

```
Properties
```

ثم نبحث عن:

```
Items
```

أو

```
Items Collection
```

ثم نضغط عليها.

ونبدأ بإضافة العناصر يدويًا.

مثلاً:

```
Item1
Item2
Item3
Item4
Item5
```

ثم نضغط:

```
OK
```

فتصبح العناصر موجودة داخل الـ CheckedListBox قبل تشغيل البرنامج.

الطريقة الثانية

Run Time

أى أثناء تشغيل البرنامج.

عن طريق الكود.

الكود المستخدم هو:

```C#
for (int i = 1; i <= 5; i++)
{
    checkedListBox1.Items.Add("Item" + i);
}
```

نشرح الكود خطوة بخطوة

أولاً:

```C#
checkedListBox1
```

هو اسم الكنترول الذى سنضيف إليه العناصر.

بعد ذلك:

```C#
Items
```

وهى مجموعة العناصر الموجودة داخل الـ CheckedListBox.

💡 معلومة إضافية

`Items` ليست Array.

بل هى **Collection Object** يحتوى على جميع العناصر الموجودة داخل الـ CheckedListBox.

بعد ذلك:

```C#
Add()
```

وهى Method وظيفتها:

> **إضافة عنصر جديد.**

إذن:

```C#
checkedListBox1.Items.Add("Item" + i);
```

تعنى:

> أضف عنصرًا جديدًا داخل الـ CheckedListBox.

حتى:

```
Item5
```

فتصبح القائمة بالشكل التالى:

```
☐ Item1
☐ Item2
☐ Item3
☐ Item4
☐ Item5
```

عرض العناصر التى تم عمل Check لها

بعد إضافة العناصر، قد نريد معرفة:

> **ما هى العناصر التى قام المستخدم بوضع علامة ✔ عليها؟**

لذلك استخدمنا الزر:

```
Show Selected Items
```

والكود الخاص به هو:

```C#
private void button2_Click(object sender, EventArgs e)
{
    for (int i = 0; i < checkedListBox1.CheckedItems.Count; i++)
    {
        MessageBox.Show(checkedListBox1.CheckedItems[i].ToString());
    }
}
```

نشرح الكود خطوة بخطوة

أولاً

```C#
checkedListBox1
```

هو الكنترول الذى نعمل عليه.

ثم

```C#
CheckedItems
```

وهى مجموعة (**Collection**) تحتوى **فقط** على العناصر التى عليها علامة ✔.

مثلاً لو كانت القائمة:

```
☑ Item1
☑ Item2
☐ Item3
☑ Item4
☐ Item5
```

فإن:

```C#
checkedListBox1.CheckedItems
```

ستحتوى على:

```
Item1
Item2
Item4
```

ولن تحتوى على:

```
Item3
Item5
```

لأنهما غير محددين بعلامة ✔.

💡 معلومة إضافية

لاحظ الفرق بين:

```C#
checkedListBox1.Items
```

وهى تحتوى على **كل العناصر** الموجودة داخل الـ CheckedListBox.

وبين:

```C#
checkedListBox1.CheckedItems
```

وهى تحتوى فقط على العناصر التى تم عمل **Check** لها.

ثم

```C#
Count
```

تعنى:

> **عدد العناصر.**

إذن:

```C#
checkedListBox1.CheckedItems.Count
```

تعنى:

> **عدد العناصر التى عليها علامة ✔ فقط.**

مثال:

لو كانت القائمة:

```
☑ Item1
☑ Item2
☐ Item3
☑ Item4
☐ Item5
```

فإن:

```C#
checkedListBox1.CheckedItems.Count
```

ستساوى:

```
3
```

وليس:

```
5
```

لأن العناصر التى عليها علامة ✔ ثلاثة فقط.

ثم

داخل الحلقة نكتب:

```C#
checkedListBox1.CheckedItems[i]
```

ومعناها:

> أحضر العنصر رقم **i** من مجموعة العناصر المحددة بعلامة ✔.

فلو كانت العناصر المحددة:

```
Item1
Item2
Item4
```

فإن:

```C#
CheckedItems[0]
```

تعيد:

```
Item1
```

و

```C#
CheckedItems[1]
```

تعيد:

```
Item2
```

و

```C#
CheckedItems[2]
```

تعيد:

```
Item4
```

ثم

```
#C
.ToString()
```

تحول العنصر إلى نص (**String**) حتى يمكن عرضه داخل:

```
#C
MessageBox.Show()
```

ثم

```
#C
MessageBox.Show(...)
```

تعرض اسم العنصر داخل نافذة رسالة.

ماذا ستكون النتيجة؟

إذا كانت القائمة:

```
☑ Item1
☑ Item2
☐ Item3
☑ Item4
☐ Item5
```

فعند الضغط على:

```
Show Selected Items
```

ستظهر ثلاث رسائل متتالية:

```
Item1
```

ثم:

```
Item2
```

ثم:

```
Item4
```

لأنها هى العناصر التى تم عمل **Check** لها.

لماذا استخدمنا `CheckedItems` وليس `Items` ؟

لأن المطلوب هو عرض **العناصر التى عليها علامة ✔ فقط**.

أما لو استخدمنا:

```C#
checkedListBox1.Items
```

فسيتم التعامل مع **كل العناصر** الموجودة داخل القائمة، سواء كانت محددة أم لا.

لذلك كانت:

```C#
CheckedItems
```

هى الاختيار الصحيح فى هذا المثال.

عمل Check لجميع العناصر

بعد إضافة العناصر، قد نريد أن نجعل **كل العناصر محددة بعلامة ✔ مرة واحدة**.

لذلك استخدمنا الزر:

```
Check All Items
```

والكود هو:

```C#
private void button3_Click(object sender, EventArgs e)
{
    for (int i = 0; i < checkedListBox1.Items.Count; i++)
    {
        checkedListBox1.SetItemChecked(i, true);
    }
}
```

نشرح الكود خطوة بخطوة

أولاً:

```C#
checkedListBox1
```

هو الكنترول الذى نعمل عليه.

بعد ذلك:

```C#
Items
```

وهى مجموعة (**Collection**) تحتوى على **جميع العناصر** الموجودة داخل الـ CheckedListBox.

ثم:

```C#
Count
```

تعنى:

> **عدد العناصر الموجودة داخل القائمة.**

فإذا كانت القائمة تحتوى على:

```
☐ Item1
☐ Item2
☐ Item3
☐ Item4
☐ Item5
```

فإن:

```C#
checkedListBox1.Items.Count
```

ستساوى:

```
5
```

💡 معلومة إضافية

هنا استخدمنا:

```C#
Items.Count
```

ولم نستخدم:

```C#
CheckedItems.Count
```

لأننا نريد المرور على **كل العناصر** الموجودة فى القائمة، سواء كانت محددة أو غير محددة.

أما `CheckedItems.Count` فيحسب فقط العناصر التى عليها ✔، وهذا ليس ما نريده هنا.

بعد ذلك:

```C#
SetItemChecked()
```

وهى Method وظيفتها:

> **وضع أو إزالة علامة ✔ من عنصر معين.**

تستقبل معاملين:

```C#
SetItemChecked(index, value);
```

المعامل الأول

```C#
index
```

هو رقم العنصر الذى نريد التعامل معه.

مثلاً:

```C#
SetItemChecked(0, true);
```

تعنى:

> ضع علامة ✔ على أول عنصر.

المعامل الثانى

```
#C
true
```

تعنى:

> اجعل العنصر **Checked**.

أما:

```
#C
false
```

فتعنى:

> اجعل العنصر **Unchecked**.

إذن:

```
#C
checkedListBox1.SetItemChecked(i, true);
```

تعنى:

> اجعل العنصر رقم **i** عليه علامة ✔ .

ماذا ستكون النتيجة؟

قبل الضغط على الزر:

```
☐ Item1
☐ Item2
☐ Item3
☐ Item4
☐ Item5
```

بعد الضغط على:

```
Check All Items
```

ستصبح:

```
☑ Item1
☑ Item2
☑ Item3
☑ Item4
☑ Item5
```

أى أن جميع العناصر أصبحت محددة.

إزالة الـ Check من جميع العناصر

الزر التالى هو:

```
UnCheck All Items
```

والكود هو:

```C#
private void button4_Click(object sender, EventArgs e)
{
    for (int i = 0; i < checkedListBox1.Items.Count; i++)
    {
        checkedListBox1.SetItemChecked(i, false);
    }
}
```

لماذا الكود يشبه السابق؟

لأننا أيضًا نريد المرور على **كل العناصر**.

لذلك استخدمنا:

```C#
checkedListBox1.Items.Count
```

وليس:

```C#
CheckedItems.Count
```

الفرق الوحيد هو:

بدلاً من:

```C#
true
```

كتبنا:

```
#C
false
```

أى:

> أزل علامة ✔ من العنصر.

إذن:

```
#C
checkedListBox1.SetItemChecked(i, false);
```

تعنى:

> اجعل العنصر رقم i غير محدد.

ماذا ستكون النتيجة؟

إذا كانت القائمة:

```
☑ Item1
☑ Item2
☑ Item3
☑ Item4
☑ Item5
```

ثم ضغطنا:

```
UnCheck All Items
```

ستصبح:

```
☐ Item1
☐ Item2
☐ Item3
☐ Item4
☐ Item5
```

أى تمت إزالة علامة ✔ من جميع العناصر.

💡 **معلومة إضافية**

لاحظ أن:

```
#C
SetItemChecked()
```

لا تقوم بإضافة عنصر جديد، ولا بحذف عنصر.

هى فقط تغير **حالة العنصر**:

- `true` → عليه علامة ✔.
- `false` → بدون علامة ✔.

حذف عنصر من الـ CheckedListBox

قد نريد حذف عنصر معين من القائمة.

لذلك استخدمنا الزر:

```
Remove Third Item
```

والكود هو:

```
#C
private void button5_Click(object sender, EventArgs e)
{
    checkedListBox1.Items.RemoveAt(2);
}
```

نشرح الكود خطوة بخطوة

أولاً:

```C#
checkedListBox1
```

هو الكنترول الذى نعمل عليه.

بعد ذلك:

```C#
Items
```

وهى مجموعة (**Collection**) التى تحتوى على جميع العناصر الموجودة داخل الـ CheckedListBox.

بعد ذلك:

```C#
RemoveAt()
```

وهى Method وظيفتها:

> **حذف عنصر باستخدام رقم الفهرس (Index).**

لاحظ أنها تستقبل معاملًا واحدًا فقط:

```C#
RemoveAt(index);
```

وهذا الـ **index** هو رقم العنصر الذى تريد حذفه.

فى الكود كتبنا:

```C#
checkedListBox1.Items.RemoveAt(2);
```

أى:

> احذف العنصر الذى رقمه **2**.

ولأن الترقيم يبدأ من:

```
0
```

فإن ترتيب العناصر يكون:

العنصر	Index
Item1	0
Item2	1
Item3	2
Item4	3
Item5	4

إذن:

```
#C ‹/›
RemoveAt(2);
```

سيحذف:

```
Item3
```

ماذا ستكون النتيجة؟

قبل الضغط على الزر:

```
☐ Item1
☐ Item2
☐ Item3
☐ Item4
☐ Item5
```

بعد الضغط على:

```
Remove Third Item
```

تصبح:

```
☐ Item1
☐ Item2
☐ Item4
☐ Item5
```

حيث تم حذف:

```
Item3
```

💡 **معلومة إضافية**

إذا حاولت حذف عنصر غير موجود، مثل:

```C#
checkedListBox1.Items.RemoveAt(10);
```

وكانت القائمة تحتوى على خمسة عناصر فقط...

فسيظهر **خطأ (Exception)** لأن العنصر رقم 10 غير موجود.

لذلك يجب التأكد دائمًا أن الـ Index موجود قبل الحذف.

الفرق بين Selected و Checked

هذه نقطة مهمة جدًا ظهرت أثناء تجربة البرنامج.

قد تلاحظ أنك ضغطت على عنصر، لكنه **ليس عليه علامة** ✔.

والسبب أن هناك فرقًا بين:

Selected

وترجمتها:

> **محدد.**

والمقصود بها:

> العنصر الذى يقف عليه المستخدم حاليًا أو قام بتحديده بالماوس.

مثلاً:

```
☐ Item1
☐ Item2   ← محدد (Selected)
☐ Item3
```

العنصر الثانى أصبح محددًا، لكنه **لم يتم عمل Check له**.

Checked

وترجمتها:

> **تم وضع علامة ✔ عليه.**

مثلاً:

```
☑ Item1
☐ Item2
☑ Item3
```

هنا العنصران الأول والثالث عليهما علامة ✔.

إذن:

> **Selected لا تعنى Checked.**

يمكن أن يكون العنصر:

- Selected فقط.
- Checked فقط.
- Selected و Checked فى نفس الوقت.

مثال

إذا ضغطت مرة واحدة على:

```
Item2
```

قد يصبح:

```
☐ Item2
```

لكنه محدد فقط (**Selected**) وليس عليه ✔.

أما إذا ضغطت على مربع الاختيار نفسه:

```
☑ Item2
```

فقد أصبح:

- Selected.
- Checked.

💡 معلومة إضافية

لهذا السبب عندما أردنا عرض العناصر التى اختارها المستخدم بعلامة ✔ استخدمنا:

```C#
CheckedItems
```

ولم نستخدم:

```C#
SelectedItems
```

لأن المطلوب هو العناصر التى عليها علامة ✔، وليس العناصر المحددة فقط.

ملخص الدرس

العنصر	وظيفته
CheckedListBox	قائمة تحتوى على عناصر وبجانب كل عنصر مربع اختيار (✔).
Items	مجموعة جميع العناصر الموجودة داخل القائمة.
CheckedItems	مجموعة العناصر التى عليها علامة ✔ فقط.
()Items.Add	إضافة عنصر جديد.
Items.Count	عدد جميع العناصر الموجودة داخل القائمة.
CheckedItems.Count	عدد العناصر التى عليها علامة ✔ فقط.
SetItemChecked(index, true)	وضع علامة ✔ على عنصر معين.
SetItemChecked(index, false)	إزالة علامة ✔ من عنصر معين.
Items.RemoveAt(index)	حذف عنصر باستخدام الـ Index.
Selected	العنصر المحدد حاليًا.
Checked	العنصر الذى عليه علامة ✔.

الخلاصة التى أحب أن تحفظها

1. **CheckedListBox** يسمح باختيار أكثر من عنصر باستخدام علامة ✔.
2. يمكن إضافة العناصر أثناء **Design Time** أو **Run Time**.
3. `Items` تحتوى على جميع العناصر.
4. `CheckedItems` تحتوى فقط على العناصر التى عليها علامة ✔.
5. `Items.Count` يعطى عدد جميع العناصر.
6. `CheckedItems.Count` يعطى عدد العناصر المحددة بعلامة ✔ فقط.
7. `SetItemChecked(index, true)` تضع علامة ✔ على العنصر.
8. `SetItemChecked(index, false)` تزيل علامة ✔ من العنصر.
9. `RemoveAt(index)` تحذف عنصرًا باستخدام رقم الفهرس.
10. **Selected** تعنى "العنصر المحدد"، أما **Checked** فتعنى "العنصر الذى عليه علامة ✔"، وهما ليسا نفس الشيء.

DateTimePicker

DateTimePicker

يتكون الاسم من ثلاث كلمات:

```
Date + Time + Picker
```

Date

ترجمتها:

> **تاريخ**

مثل:

```
22/07/2026
```

Time

ترجمتها:

> **وقت**

مثل:

```
10:30 PM
```

Picker

ترجمتها:

> **منتقى**

أو

> **أداة اختيار**

أى أنه يسمح للمستخدم باختيار قيمة.

إذن الترجمة الحرفية لـ:

```
DateTimePicker
```

هى:

> **أداة لاختيار التاريخ والوقت.**

ويمكن تبسيط معناها إلى:

> **كنترول يسمح للمستخدم باختيار تاريخ أو وقت (أو كليهما) بسهولة دون أن يكتبهم بنفسه.**

ما هو الـ DateTimePicker؟

الـ **DateTimePicker** هو كنترول يسمح للمستخدم باختيار التاريخ.

وعند الحاجة يمكن أيضًا استخدامه لاختيار الوقت.

فعندما يضغط المستخدم على السهم الموجود بجانبه...

تظهر له تقويم (Calendar).

ثم يختار اليوم الذى يريده.

مثلاً:

```
22 July 2026
```

أو

```
15 March 2023
```

وبمجرد اختياره...

تظهر القيمة داخل الـ DateTimePicker.

لماذا نستخدمه؟

بدلًا من أن يكتب المستخدم التاريخ بنفسه.

مثل:

```
22/07/2026
```

فقد يكتب:

```
22/70/2026
```

أو

```
99/99/2026
```

وهذا تاريخ غير صحيح.

أما باستخدام:

```
DateTimePicker
```

فالمستخدم يختار التاريخ من التقويم مباشرة.

وبذلك تقل الأخطاء أثناء إدخال البيانات.

كيف نضيف الـ DateTimePicker إلى الفورم؟

من نافذة:

```
ToolBox
```

نبحث عن:

```
DateTimePicker
```

ثم نقوم بسحبه وإفلاته (Drag & Drop) داخل الـ Form.

وبذلك أصبح لدينا كنترول من نوع:

```
DateTimePicker
```

جاهز للاستخدام.

الشكل الافتراضى (Default)

عند تشغيل البرنامج لأول مرة...

سيظهر الـ DateTimePicker بالشكل الافتراضى.

مثلاً:

```
Wednesday, July 22, 2026 ▼
```

وبمجرد الضغط على السهم...

سيظهر التقويم.

ثم يختار المستخدم التاريخ الذى يريده.

خاصية مهمة

Format

ترجمتها:

> **تنسيق**

أو

> **طريقة عرض التاريخ.**

وهى تحدد:

> كيف سيظهر التاريخ داخل الـ DateTimePicker.

يوجد لها عدة قيم.

Long

وهى القيمة الافتراضية.

تعرض التاريخ بالكامل.

مثلاً:

```
Wednesday, July 22, 2026
```

فتظهر:

- اسم اليوم.
- اسم الشهر.
- رقم اليوم.
- السنة.

Short

تعرض التاريخ بصورة مختصرة.

مثلاً:

```
22/07/2026
```

أو حسب إعدادات نظام التشغيل.

فتكون أقصر من:

```
Long
```

Time

بدلًا من عرض التاريخ...

تعرض الوقت فقط.

مثلاً:

```
10:30 PM
```

أو

```
08:15 AM
```

حسب إعدادات الجهاز.

Custom

تعنى:

> **تنسيق مخصص.**

أى أنك أنت الذى تحدد شكل عرض التاريخ بالطريقة التى تريدها.

مثلاً:

```
22-Jul-2026
```

أو

```
July 22, 2026
```

أو

```
2026/07/22
```

وسيتم شرح طريقة كتابة هذه التنسيقات لاحقًا عند شرح:

```C#
ToString("Format")
```

لأنها تعتمد على نفس فكرة الـ Format.

💡 **معلومة إضافية**

خاصية:

```
Format
```

تغير **طريقة عرض التاريخ فقط**.

أما التاريخ نفسه فلا يتغير.

فمثلاً إذا كان التاريخ المختار هو:

```
22 July 2026
```

فيمكن عرضه بهذه الصور:

```
22/07/2026
```

أو

```
Wednesday, July 22, 2026
```

أو

```
22-Jul-2026
```

لكن فى جميع الحالات...

التاريخ المخزن داخليًا ما زال هو نفس التاريخ.

الذى تغير فقط هو **شكل عرضه للمستخدم**.

ماذا فعل البرنامج فى المثال؟

عند اختيار تاريخ جديد...

كان يتم تحديث الـ Label الموجود بجانب الـ DateTimePicker.

ويتم عرض نفس التاريخ...

ولكن بعدة أشكال مختلفة (Formats).

مرة:

```
Wednesday, July 22, 2026
```

ومرة:

```
22-Jul-2026
```

ومرة:

```
07-22-2026
```

وغيرها من التنسيقات المختلفة.

وسيتم شرح كيفية كتابة هذه التنسيقات بالتفصيل فى الجزء الثالث عند شرح:

```C#
ToString("Format")
```

خاصية مهمة

ShowCheckBox

ترجمتها:

> **إظهار مربع اختيار.**

نوعها:

```
bool
```

أى:

```
True
False
```

إذا كانت:

```
False
```

(وهى القيمة الافتراضية)

فسيظهر الـ DateTimePicker بالشكل المعتاد.

مثلاً:

```
Wednesday, July 22, 2026 ▼
```

ويستطيع المستخدم اختيار التاريخ مباشرة.

أما إذا كانت:

```
True
```

فسيظهر مربع اختيار (CheckBox) بجانب التاريخ.

مثلاً:

```
☐ Wednesday, July 22, 2026 ▼
```

فى هذه الحالة...

إذا كان مربع الاختيار غير محدد:

```
☐
```

فلن يستطيع المستخدم تغيير التاريخ.

أما إذا وضع علامة صح:

```
☑
```

فيمكنه بعد ذلك اختيار التاريخ.

ولهذا عند تشغيل البرنامج...

لن تستطيع اختيار تاريخ إلا بعد تفعيل الـ CheckBox أولًا.

💡 معلومة إضافية

تُستخدم هذه الخاصية كثيرًا عندما يكون التاريخ **اختياريًا**.

مثلاً:

لديك نموذج تسجيل موظف.

يوجد حقل:

```
تاريخ انتهاء العقد
```

لكن ليس كل الموظفين لديهم تاريخ انتهاء.

فيمكنك استخدام:

```
ShowCheckBox = True
```

إذا وضع المستخدم علامة ✔

فمعناه:

> أريد إدخال تاريخ.

أما إذا تركها بدون علامة...

فمعناه:

> لا يوجد تاريخ.

ملحوظه جانبيه

لما تجعل:

```
ShowCheckBox = True
```

فإن الـ `DateTimePicker` يصبح له حالتان:

الحالة الأولى

مربع الصح غير مُفعّل:

```
☐ 17 June 2026
```

يعنى التاريخ غير مُفعّل.

الحالة الثانية

مربع الصح مُفعّل:

```
☑ 17 June 2026
```

وهنا التاريخ أصبح مُفعّل ويمكن استخدامه.

كيف يمكن للمستخدم تفعيل مربع الصح؟

يوجد طريقتان:

الطريقة الأولى

الضغط على مربع الصح نفسه.

```
☐
```

↓

```
☑
```

الطريقة الثانية

الضغط على سهم القائمة الموجود يمين الـ DateTimePicker.

فعندما تضغط على السهم:

```
▾
```

يريد Windows أن يفتح لك التقويم.

لكن لا يمكن فتح التقويم بينما التاريخ غير مُفعّل.

لذلك يقوم النظام داخليًا بالآتى تلقائيًا:

1. يضع علامة ✔ على الـ CheckBox.
2. ثم يفتح التقويم مباشرة.

يعنى كأنه نفذ داخليًا:

```
☐
```

↓

```
☑
```

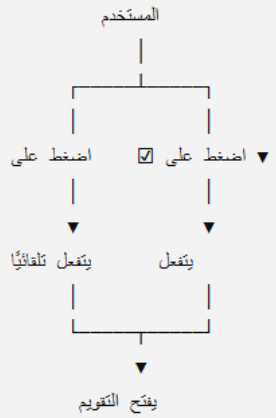
↓

🗓️ فتح التقويم.

ولهذا لاحظت أن علامة الصح ظهرت تلقائيًا، وهذا **السلوك الطبيعى** للـ `DateTimePicker`.

ارسمها فى دماغك

```
ShowCheckBox = True
```



إذن استنتاجك صحيح:

> عندما يكون `ShowCheckBox = True` ، يمكن تفعيل التاريخ **إما بالضغط على مربع الصح مباشرة، أو بمجرد الضغط على سهم التقويم، حيث يقوم Windows بتفعيل مربع الصح تلقائيًا ثم يفتح التقويم.**

خاصية مهمة

ShowUpDown

ترجمتها:

> **إظهار أزرار الزيادة والنقصان.**

نوعها:

```
bool
```

إذا كانت:

```
False
```

(القيمة الافتراضية)

سيظهر السهم.

وعند الضغط عليه...

سيظهر التقويم.

مثل:

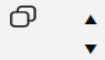
```
Wednesday, July 22, 2026 ▼
```

أما إذا كانت:

```
True
```

فلن يظهر التقويم.

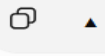
وسيظهر بدلًا منه زران:



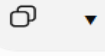
ويستطيع المستخدم تغيير:

- اليوم.
- الشهر.
- السنة.

عن طريق الضغط على:



أو



بدلًا من فتح التقويم.

ولهذا عند تشغيل البرنامج...

لن يظهر لك Calendar.

بل ستزيد أو تنقص التاريخ باستخدام الأسهم.

💡 معلومة إضافية

عند استخدام:

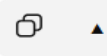
```
ShowUpDown = False
```

يمكنك الضغط على **اليوم** أو **الشهر** أو **السنة** لتحديدها، لكن إذا فتحت التقويم (**Calendar**) بعد ذلك واخترت تاريخًا جديدًا، فإن هذا التحديد غالبًا لا تكون له أهمية، لأن التقويم سيغير **اليوم والشهر والسنة معًا**.

أما عند استخدام:

```
ShowUpDown = True
```

فيصبح تحديد **اليوم** أو **الشهر** أو **السنة** خطوة مهمة، لأن السهمين:



سيقومان بزيادة أو إنقاص **الجزء الذى قمت بتحديده فقط**. لذلك يجب أولًا تحديد الجزء المراد تعديله، ثم استخدام السهمين لتغييره.

💡 معلومة إضافية

هذه الخاصية تكون مفيدة عندما تريد من المستخدم تعديل التاريخ بسرعة دون فتح التقويم فى كل مرة.

خاصية مهمة

MinDate

ترجمتها:

> **أقل تاريخ مسموح به.**

تحدد أقدم تاريخ يستطيع المستخدم اختياره.

فإذا كانت:

```
MinDate
```

تساوى:

```
01/01/2000
```

فلن يستطيع اختيار أى تاريخ قبل سنة:

```
2000
```

مثال

إذا حاول المستخدم الرجوع إلى:

```
1995
```

فلن يسمح له البرنامج بذلك.

ولهذا تُستخدم لمنع المستخدم من اختيار تواريخ غير مناسبة.

خاصية مهمة

MaxDate

ترجمتها:

> **أكبر تاريخ مسموح به.**

وهى عكس:

```
MinDate
```

فهى تحدد آخر تاريخ يمكن للمستخدم اختياره.

مثلاً:

إذا جعلتها:

```
31/12/2026
```

فلن يستطيع المستخدم اختيار أى تاريخ بعد:

```
2026
```

ولهذا يمكن استخدام:

```
MinDate
```

و

```
MaxDate
```

لتحديد فترة زمنية معينة فقط.

مثال

إذا كنت تصمم برنامجًا لتسجيل تاريخ ميلاد.

يمكنك منع المستخدم من اختيار تاريخ فى المستقبل.

أو منع اختيار تاريخ قديم جدًا.

وبذلك لا يستطيع اختيار إلا التواريخ المسموح بها.

💡 **معلومة إضافية**

يستخدم المبرمجون غالبًا:

```
MinDate
```

و

```
MaxDate
```

للتحقق من صحة البيانات **قبل أن يكتب المستخدم أى كود للتحقق**، لأن الـ DateTimePicker نفسه يمنع اختيار تاريخ خارج هذا النطاق.

خاصية مهمة جدًا

Value

ترجمتها:

> **القيمة.**

وهى تمثل:

> **التاريخ (أو الوقت) المختار حاليًا داخل الـ DateTimePicker.**

فإذا اختار المستخدم:

```
22 July 2026
```

فإن:

```
#C
dateTimePicker1.Value
```

ستمثل هذا التاريخ.

وهى أهم خاصية داخل الـ DateTimePicker.

لأن معظم العمليات تعتمد عليها.

مثل:

- عرض التاريخ.
- حفظ التاريخ.
- تحويله إلى نص.
- مقارنة التواريخ.

ولهذا استخدم البرنامج:

```C#
dateTimePicker1.Value
```

ثم استدعى عليها عدة دوال مثل:

```C#
ToShortDateString()
```

و

```C#
ToLongDateString()
```

و

```C#
ToString(...)
```

وسيتم شرحهم بالتفصيل فى الجزء القادم.

ماذا فعلت الأزرار الموجودة فى المثال؟

كان يوجد زران.

الزر الأول

يعرض التاريخ المختار فى الصورة المختصرة.

عن طريق:

```C#
dateTimePicker1.Value.ToShortDateString()
```

الزر الثانى

يعرض التاريخ المختار فى الصورة الطويلة.

عن طريق:

```C#
dateTimePicker1.Value.ToLongDateString()
```

وسيتم شرح هاتين الدالتين بالتفصيل فى الجزء الثالث.

Event مهمة

ValueChanged

ترجمتها الحرفية:

> **القيمة تغيرت.**

والمقصود:

> تعمل هذه الـ Event عندما يغير المستخدم التاريخ الموجود داخل الـ DateTimePicker.

كلما اختار المستخدم تاريخًا جديدًا...

سيتم استدعاء هذه الـ Event تلقائيًا.

وكان الكود الموجود داخلها هو:

```C#
private void dateTimePicker1_ValueChanged(object sender, EventArgs e)
{
    label1.Text = dateTimePicker1.Text + Environment.NewLine;

    label1.Text += dateTimePicker1.Value.ToString("dd-MMM-yyyy") + Environment.NewLine;

    label1.Text += dateTimePicker1.Value.ToString("dddd-MMM-yyyy") + Environment.NewLine;

    label1.Text += dateTimePicker1.Value.ToString("MM-dd-yyyy") + Environment.NewLine;

    label1.Text += dateTimePicker1.Value.ToString("dd/MM/yy") + Environment.NewLine;

    label1.Text += dateTimePicker1.Value.ToString("dddd,dd-MMM-yyyy") + Environment.NewLine;
}
```

أول سطر

```C#
label1.Text = dateTimePicker1.Text;
```

نشرحه

```C#
label1
```

هو الـ Label الذى سيتم عرض النص بداخله.

```C#
Text
```

خاصية تمثل النص الذى يظهر داخل الـ Label.

```C#
dateTimePicker1.Text
```

تعيد التاريخ المختار كنص (String).

وبنفس الـ Format الظاهر داخل الـ DateTimePicker.

مثلاً

إذا كان الـ DateTimePicker يعرض:

```
Wednesday, July 22, 2026
```

فإن:

```C#
dateTimePicker1.Text
```

سترجع:

```
Wednesday, July 22, 2026
```

ثم يتم وضع هذا النص داخل الـ Label.

السطر التالى

```C#
Environment.NewLine
```

ترجمته:

> **سطر جديد.**

وظيفته:

النزول إلى سطر جديد.

مثل:

```
Line 1
Line 2
Line 3
```

بدلاً من:

```
Line 1Line 2Line 3
```

فى C++ كنت تستخدم غالبًا:

```C++
"\n"
```

أو

```C++
endl
```

أما فى Windows Forms فمن الأفضل استخدام:

```C#
Environment.NewLine
```

لأنه يضيف سطرًا جديدًا بالطريقة المناسبة لنظام التشغيل.

💡 معلومة إضافية

يمكن أيضًا استخدام:

```C#
"\n"
```

فى كثير من الحالات.

لكن:

```C#
Environment.NewLine
```

هو الأسلوب الموصى به داخل تطبيقات C# لأنه يعتمد على نظام التشغيل.

بعد ذلك

```C#
dateTimePicker1.Value.ToString(...)
```

سنشرحها جزءًا جزءًا.

أولاً

```C#
Value
```

وهى التاريخ المختار.

ثم:

```C#
ToString()
```

ترجمتها:

> **حوّل إلى String.**

لكن هذه المرة كتبنا:

```C#
ToString("...")
```

أى:

> حول التاريخ إلى نص، ولكن باستخدام التنسيق الذى أحدده أنا.

المثال الأول

```C#
dateTimePicker1.Value.ToString("dd-MMM-yyyy")
```

---

dd

تعنى:

> اليوم برقمين.

مثال:

```
22
```

MMM

تعنى:

> اسم الشهر المختصر.

مثال:

```
Jan
Feb
Mar
Apr
Jul
```

---

yyyy

تعنى:

> السنة كاملة.

مثال:

```
2026
```

إذن ستكون النتيجة:

```
22-Jul-2026
```

المثال الثانى

```C# ‹›
dateTimePicker1.Value.ToString("dddd-MMM-yyyy")
```

---

dddd

تعنى:

> اسم اليوم كامل.

مثال:

```
Monday
Tuesday
Wednesday
```

فتكون النتيجة:

```
Wednesday-Jul-2026
```

المثال الثالث

```C#
dateTimePicker1.Value.ToString("MM-dd-yyyy")
```

MM

تعنى:

> رقم الشهر.

مثلاً:

```
07
```

فتكون النتيجة:

```
07-22-2026
```

المثال الرابع

```C#
dateTimePicker1.Value.ToString("dd/MM/yy")
```

yy

تعنى:

> آخر رقمين من السنة.

مثلاً:

```
26
```

فتصبح النتيجة:

```
22/07/26
```

المثال الخامس

```C#
dateTimePicker1.Value.ToString("dddd,dd-MMM-yyyy")
```

فتكون النتيجة:

```
Wednesday,22-Jul-2026
```

💡 معلومة إضافية

يمكنك تغيير ترتيب هذه الرموز كما تريد.

مثلاً:

```C#
ToString("yyyy/MM/dd")
```

ستكون النتيجة:

```
2026/07/22
```

أو:

```C#
ToString("MMM dd yyyy")
```

فتكون:

```
Jul 22 2026
```

ولهذا تعتبر:

```C#
ToString("...")
```

من أقوى الطرق لتنسيق التاريخ بالطريقة التى تريدها.

أهم رموز تنسيق التاريخ

الرمز	معناه	مثال
`dd`	اليوم برقمين	22
`d`	اليوم بدون صفر إذا كان رقمًا واحدًا	5
`MM`	رقم الشهر	07
`M`	رقم الشهر بدون صفر إذا كان رقمًا واحدًا	7
`MMM`	اسم الشهر المختصر	Jul
`MMMM`	اسم الشهر كامل	July
`yy`	آخر رقمين من السنة	26
`yyyy`	السنة كاملة	2026
`ddd`	اسم اليوم المختصر	Wed
`dddd`	اسم اليوم كامل	Wednesday

💡 **معلومة إضافية**

هذه الرموز لا تخص الـ **DateTimePicker** فقط.

بل تخص كائن:

```C#
DateTime
```

لذلك يمكنك استخدامها مع أى متغير من نوع:

```C#
DateTime
```

وليس فقط داخل الـ DateTimePicker.

خصائص أخرى

يمكن أيضًا استخدام الخصائص التالية مع الـ DateTimePicker:

- `Enabled` لتعطيل أو تفعيل الكنترول.
- `Visible` لإظهار أو إخفاء الكنترول.

وتعمل بنفس الطريقة التى درستها سابقًا مع باقى عناصر Windows Forms.

ملخص الدرس

الخاصية / العنصر	وظيفتها
DateTimePicker	كنترول لاختيار التاريخ والوقت.
Format	يحدد طريقة عرض التاريخ.
Long	يعرض التاريخ بالكامل.
Short	يعرض التاريخ المختصر.
Time	يعرض الوقت فقط.
Custom	يسمح بكتابة تنسيق خاص.
ShowCheckBox	يعرض CheckBox بجانب التاريخ.
ShowUpDown	يستخدم الأسهم بدلًا من التقويم.
MinDate	أقل تاريخ مسموح به.
MaxDate	أكبر تاريخ مسموح به.
Value	التاريخ المختار حاليًا.
Text	يعرض التاريخ كنص حسب الـ Format الحالى.

ValueChanged	تعمل عند تغيير التاريخ.
()ToShortDateString	يعرض التاريخ بالصيغة المختصرة.
()ToLongDateString	يعرض التاريخ بالصيغة الطويلة.
("...")ToString	يعرض التاريخ بالتنسيق الذى تحدده.
Environment.NewLine	ينزل إلى سطر جديد.

الخلاصة التى أحب أن تحفظها

1. **DateTimePicker** يستخدم لاختيار التاريخ (وأحيانًا الوقت).
2. الخاصية **Format** تحدد طريقة عرض التاريخ (`Long` أو `Short` أو `Time` أو `Custom`).
3. الخاصية **Value** هى أهم خاصية لأنها تحتوى على التاريخ المختار.
4. الخاصية **Text** تعيد التاريخ كنص بالشكل الظاهر داخل الكنترول.
5. تعمل الـ `ValueChanged` Event كلما تغير التاريخ.
6. الدالة `()ToShortDateString` تعرض التاريخ بصيغة مختصرة.
7. الدالة `()ToLongDateString` تعرض التاريخ بصيغة طويلة.
8. باستخدام `("...")ToString` يمكنك تصميم شكل التاريخ كما تريد باستخدام رموز مثل `dd` و `MM` و `MMM` و `yyyy` .
9. `Environment.NewLine` يستخدم للانتقال إلى سطر جديد داخل النصوص.
10. يمكن تقييد التواريخ المسموح بها باستخدام `MinDate` و** `MaxDate` **.

MonthCalandar

MonthCalendar

يتكون الاسم من كلمتين:

```
Month + Calendar
```

Month

ترجمتها:

> **شهر**

Calendar

ترجمتها:

> **تقويم**

أو

> **رزنامة**

إذن الترجمة الحرفية لـ:

```
MonthCalendar
```

هى:

> **تقويم الشهر**

ما هو الـ MonthCalendar؟

الـ **MonthCalendar** هو كنترول يعرض للمستخدم تقويمًا كاملًا (Calendar)، بحيث يستطيع اختيار تاريخ واحد أو مجموعة من التواريخ.

مثال:

```
July 2026

Sun Mon Tue Wed Thu Fri Sat
 5   6   7   8   9  10  11
12  13  14  15  16  17  18
```

ومن خلاله يستطيع المستخدم الضغط على أى يوم لاختياره.

أين نجده؟

من نافذة:

```
ToolBox
```

نبحث عن:

```
MonthCalendar
```

ثم نسحبه إلى الـ Form.

هل يشبه DateTimePicker؟

نعم.

الـ **MonthCalendar** يشترك مع **DateTimePicker** فى كثير من الخصائص التى درستها بالفعل.

مثل:

- MinimumDate
- MaximumDate
- Enabled
- RightToLeft
- الألوان
- التعامل مع التاريخ

ولذلك لن نعيد شرحها لأنها تعمل بنفس الطريقة.

خاصية مهمة

MaxSelectionCount

ترجمتها:

> **أقصى عدد للتواريخ التى يمكن تحديدها.**

وظيفتها:

تحدد أكبر عدد من الأيام التى يسمح للمستخدم باختيارها.

مثلاً إذا كانت:

```
MaxSelectionCount = 7
```

فيستطيع المستخدم تحديد:

```
من 10 يوليو
إلى 16 يوليو
```

لأنها سبعة أيام.

أما إذا حاول تحديد أكثر من سبعة أيام...

فلن يسمح له البرنامج بذلك.

لماذا نستخدمها؟

فى البرامج التى تعتمد على الحجز.

مثل:

- حجز الفنادق.
- حجز الشاليهات.
- حجز السيارات.
- حجز قاعات الأفراح.

مثلاً:

```
من:
12 يوليو

إلى:
17 يوليو
```

فيقوم المستخدم بتحديد الفترة كلها مرة واحدة.

💡 **معلومة إضافية**

إذا كانت قيمة:

```
MaxSelectionCount = 1
```

فلن يستطيع المستخدم اختيار إلا يوم واحد فقط.

ويصبح سلوك الـ MonthCalendar قريبًا من اختيار تاريخ واحد فقط.

خاصية

ShowToday

وظيفتها:

تحدد هل يظهر سطر:

```
Today:
```

أسفل التقويم أم لا.

إذا كانت:

```
True
```

سيظهر:

```
Today: 7/11/2026
```

أما إذا كانت:

```
False
```

فسيختفى هذا السطر.

خاصية

ShowTodayCircle

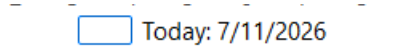
وظيفتها:

تحدد هل يتم وضع دائرة حول تاريخ اليوم الحالى أم لا.

إذا كانت:

```
True
```

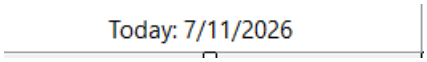
سيظهر اليوم الحالى داخل دائرة.



أما إذا كانت:

```
False
```

فلن تظهر هذه الدائرة.



خاصية

ShowWeekNumbers

ترجمتها:

> **إظهار أرقام الأسابيع.**

إذا كانت:

```
True
```

سيظهر رقم كل أسبوع بجانب التقويم.

مثلاً:

```
Week 28
Week 29
Week 30
```

أما إذا كانت:

```
False
```

فلن تظهر أرقام الأسابيع.

SelectionRange

ترجمتها:

> **نطاق التحديد.**

وهى تمثل جميع الأيام التى قام المستخدم بتحديدها.

مثلاً:

إذا اختار المستخدم:

```
من:
5 يوليو

إلى:
9 يوليو
```

فإن:

```C#
monthCalendar1.SelectionRange
```

تمثل الفترة كاملة:

```
5 يوليو → 9 يوليو
```

الكود:

```C#
MessageBox.Show(monthCalendar1.SelectionRange.ToString());
```

يعرض الفترة التى اختارها المستخدم كاملة.

SelectionStart

ترجمتها:

> **بداية التحديد.**

تعيد أول يوم فى الفترة التى اختارها المستخدم.

مثلاً:

إذا كانت الفترة:

```
5 يوليو

إلى

9 يوليو
```

فإن:

```C#
monthCalendar1.SelectionStart
```

ستعيد:

```
5 يوليو
```

الكود:

```C#
MessageBox.Show(monthCalendar1.SelectionStart.ToString());
```

يعرض أول يوم فى الفترة.

SelectionEnd

ترجمتها:

> **نهاية التحديد.**

تعيد آخر يوم فى الفترة التى اختارها المستخدم.

فى المثال السابق:

```
5 يوليو

إلى

9 يوليو
```

فإن:

```
#C ‹›
monthCalendar1.SelectionEnd
```

ستعيد:

```
9 يوليو
```

الكود:

```
#C ‹›
MessageBox.Show(monthCalendar1.SelectionEnd.ToString());
```

يعرض آخر يوم فى الفترة.

إذا اختار المستخدم يومًا واحدًا فقط، مثل:

```
22 July
```

فإن:

```
SelectionStart = 22 July
SelectionEnd   = 22 July
```

لأن بداية الفترة ونهايتها هما نفس اليوم.

التعامل مع التاريخ

القيم التى تعيدها:

```
SelectionStart
```

و

```
SelectionEnd
```

هى من نوع **DateTime**.

لذلك يمكن استخدامها بنفس الطرق التى درستها فى **DateTimePicker**.

مثل:

```C#
ToShortDateString()
```

أو

```C#
ToLongDateString()
```

أو

```C#
ToString("dd-MMM-yyyy")
```

وغيرها من تنسيقات التاريخ، لأنها تعمل بنفس الطريقة تمامًا.

خصائص أخرى

كما فى DateTimePicker يوجد أيضًا:

- Enabled
- RightToLeft
- LeftToRight
- MinimumDate
- MaximumDate
- الألوان

وجميعها تعمل بنفس الطريقة التى درستها فى الدرس السابق.

ملخص الدرس

العنصر	وظيفته
MonthCalendar	يعرض تقويمًا كاملًا لاختيار تاريخ أو فترة زمنية.
MaxSelectionCount	يحدد أكبر عدد من الأيام التى يمكن اختيارها.
ShowToday	إظهار أو إخفاء سطر Today أسفل التقويم.
ShowTodayCircle	إظهار أو إخفاء الدائرة حول تاريخ اليوم الحالى.
ShowWeekNumbers	إظهار أو إخفاء أرقام الأسابيع.
SelectionRange	يعيد الفترة الزمنية التى اختارها المستخدم كاملة.
SelectionStart	يعيد أول يوم فى الفترة المختارة.
SelectionEnd	يعيد آخر يوم فى الفترة المختارة.

الخلاصة التى أحب أن تحفظها

1. **MonthCalendar** يعرض تقويمًا كاملًا داخل الفورم.
2. يشترك مع **DateTimePicker** فى كثير من الخصائص مثل **MinimumDate** و**MaximumDate**، لذلك التعامل مع التاريخ فيه متشابه.
3. الخاصية **MaxSelectionCount** تحدد أكبر عدد من الأيام التى يمكن للمستخدم اختيارها.
4. **SelectionRange** تعيد الفترة الزمنية كاملة.
5. **SelectionStart** تعيد أول يوم فى الفترة.
6. **SelectionEnd** تعيد آخر يوم فى الفترة.
7. لأن `SelectionStart` و `SelectionEnd` من نوع **DateTime**، فيمكن استخدام جميع دوال تنسيق التاريخ التى درستها مع **DateTimePicker** مثل `ToShortDateString()` و `ToLongDateString()` و `ToString("dd-MMM-yyyy")`.

Timer

Timer

يتكون الاسم من كلمة واحدة:

```
Timer
```

Timer

ترجمتها:

> **مؤقت**.

أو

> **عداد زمنى**.

ما هو الـ Timer؟

الـ **Timer** هو Control يستخدم لتنفيذ كود معين **كل فترة زمنية محددة**.

بمعنى:

بدلاً من أن ينتظر البرنامج أن يضغط المستخدم على زر...

يمكن للبرنامج أن ينفذ كودًا تلقائيًا كل:

- ثانية.
- ثانيتين.
- خمس ثوانٍ.
- دقيقة.

حسب الفترة التى تحددها أنت.

كيف نضيف الـ Timer؟

من الـ:

```
ToolBox
```

نبحث عن:

```
Timer
```

ثم نضغط عليه ضغطتين (**Double Click**).

سيتم إضافته إلى الفورم.

لكن ستلاحظ أنه **لن يظهر داخل الفورم** مثل Button أو Label.

بل يظهر أسفل نافذة التصميم.

مثلاً:

```
timer1
```

وذلك لأنه ليس عنصرًا مرئيًا للمستخدم، وإنما يعمل فى الخلفية.

أهم خصائص الـ Timer

Enabled

تحدد هل الـ Timer يعمل أم لا.

إذا كانت:

```
False
```

فالـ Timer متوقف.

ولا ينفذ أى شيء.

أما إذا كانت:

```
True
```

فالـ Timer يبدأ فى العمل.

Interval

ترجمتها:

> **الفترة الزمنية.**

وهى تحدد:

> كل كام مللى ثانية (Millisecond) ينفذ الـ Timer الـ Event الخاصة به.

أمثلة:

```
1000
```

تعنى:

> ثانية واحدة.

```
2000
```

تعنى:

> ثانيتان.

```
5000
```

تعنى:

> خمس ثواني.

إذن:

كلما انتهت قيمة الـ Interval...

سيقوم الـ Timer بتنفيذ الـ Event الخاصة به.

💡 معلومة إضافية:

الخاصية `Interval` مصممة بحيث تستقبل القيمة بوحدة **Millisecond**، لذلك عندما تكتب:

```C#
timer1.Interval = 5000;
```

فإن البرنامج يفهمها على أنها **5000 Milliseconds** وليس **5000 Seconds**.

إذن يمكن أن تتخيلها هكذا:

```
Interval
    |
    ▼
القيمة التى تكتبها
    |
    ▼
تُفسر دائمًا على أنها Milliseconds
```

ولذلك لا تكتب:

```C#
timer1.Interval = 5;
```

وأنت تقصد 5 ثوانٍ.

لأن البرنامج سيفهمها:

> **5 Milliseconds فقط**، وليس 5 ثوانٍ.

ولهذا إذا أردت 5 ثوانٍ تكتب:

```C#
timer1.Interval = 5000;
```

لأن وحدة القياس الخاصة بالخاصية هى **Millisecond**.

Event مهمة

Tick

وهى أهم Event فى الـ Timer.

كلما انتهت الفترة الزمنية الموجودة داخل:

```
Interval
```

وكان:

```
Enabled = true
```

يقوم الـ Timer بتنفيذ Event اسمها:

```
Tick
```

أى أنه يستدعى الكود الموجود داخلها.

💡 **معلومة إضافية: الفرق بين Tick فى `Timer` و Tick فى `DateTime`**

قد يختلط الأمر لأن الاسم واحد، لكن المقصود مختلف تمامًا.

لماذا سُميت Tick؟

كلمة **Tick** فى اللغة الإنجليزية تعنى:

> **دقة** أو **نبضة**، مثل صوت الساعة:

```
Tick... Tick... Tick...
```

لذلك استُخدمت فى الحالتين، لكن بمعنيين مختلفين.

1- `DateTime.Tick`

هنا **Tick** هى **وحدة قياس زمن**.

```
1 Tick = 100 Nanoseconds
```

أى أنها تمثل فترة زمنية صغيرة جدًا.

2- `Timer.Tick`

هنا **Tick** ليست وحدة زمنية.

بل هى **اسم Event** يتم استدعاؤها كلما انتهت الفترة الزمنية المحددة فى:

```C#
timer1.Interval
```

فمثلًا:

```C#
timer1.Interval = 1000;
```

تعنى:

```
1000 انتظر Milliseconds
        ↓
استدعاء Tick Event
        ↓
1000 انتظر Milliseconds
        ↓
استدعاء Tick Event
```

⚠️ لا تخلط بينهما

`DateTime.Tick`	`Timer.Tick`
وحدة قياس زمن	Event
1 Tick = 100 Nanoseconds	تعمل بعد انتهاء الـ Interval
تستخدم مع `DateTime`	تستخدم مع `Timer`

هذه المعلومة هتخليك كلما ترى كلمة **Tick** تسأل نفسك أولًا:

> **"هى تابعة لأى كلاس؟"**

- إذا كانت مع `DateTime` ⇒ فهى **وحدة زمن (100 ns)**.
- إذا كانت مع `Timer` ⇒ فهى **Event** يتم استدعاؤها بعد انتهاء الـ **Interval**.

مثال البرنامج

تم إنشاء متغير:

```C#
int Counter = 0;
```

ليحفظ عدد المرات التى يعمل فيها الـ Timer.

ثم داخل Event:

```C#
timer1_Tick
```

يوجد الكود:

```C#
Counter++;
label1.Text = Counter.ToString();
```

فعند كل مرة تعمل فيها الـ Tick:

يزيد العداد واحدًا.

ثم يعرض قيمته داخل الـ Label.

ماذا يحدث عند تشغيل البرنامج؟

عند بداية تشغيل البرنامج:

الـ Timer متوقف لأن:

```
Enabled = false
```

إذن:

لن يحدث أى تغيير داخل الـ Label.

عند الضغط على زر:

```
Start
```

ينفذ الكود:

```C#
timer1.Enabled = true;
```

فيبدأ الـ Timer فى العمل.

وبما أن:

```
Interval = 1000
```

فإنه كل ثانية:

يقوم بتنفيذ:

```
Tick
```

فتزداد قيمة:

```
Counter
```

ثم يتم تحديث الـ Label.

فتصبح القيم:

```
1
2
3
4
5
...
```

زر Stop

عند الضغط على:

```
End
```

ينفذ الكود:

```
#C ‹›
timer1.Enabled = false;
```

وبذلك يتوقف الـ Timer.

ولن يتم تنفيذ:

```
Tick
```

مرة أخرى.

ويبقى العداد عند آخر قيمة وصل إليها.

تشغيل الـ Timer مرة أخرى

إذا ضغط المستخدم على:

```
Start
```

مرة أخرى.

يصبح:

```C#
timer1.Enabled = true;
```

فيستكمل الـ Timer العد من آخر قيمة وصل إليها.

استخدامات الـ Timer

يمكن استخدام الـ Timer فى أشياء كثيرة.

مثل:

- تنفيذ كود كل فترة زمنية.
- عمل عداد.
- تحديث الوقت باستمرار.
- تنفيذ معالجة للملفات كل عدة دقائق.
- عمل برنامج تسبيح يزيد العد كل فترة زمنية.

💡 **معلومة إضافية**

لاحظ أن الـ Timer **لا ينفذ الكود باستمرار**.

بل ينتظر حتى تنتهى قيمة:

```
Interval
```

ثم ينفذ Event واحدة فقط وهى:

```
Tick
```

ثم ينتظر مرة أخرى نفس الفترة.

لذلك إذا كانت:

```
Interval = 1000
```

فإن التنفيذ يكون هكذا:

```
انتظار ثانية
        ↓
تنفيذ Tick
        ↓
انتظار ثانية
        ↓
تنفيذ Tick
        ↓
انتظار ثانية
        ↓
تنفيذ Tick
```

وليس تنفيذًا مستمرًا بدون توقف.

يمكن أن يحتوى البرنامج على أكثر من Timer.

كل Timer له:

- Interval خاصة به.
- Enabled خاصة به.
- Tick خاصة به.

وبذلك يمكن لكل Timer تنفيذ مهمة مختلفة بشكل مستقل عن الآخر.

ملخص الدرس

العنصر	وظيفته
Timer	مؤقت ينفذ كودًا كل فترة زمنية محددة.
Enabled	تشغيل (`true`) أو إيقاف (`false`) الـ Timer.
Interval	الفترة الزمنية بين كل تنفيذ للـ Tick (بالملى ثانية).
Tick	الـ Event التى تُنفذ عند انتهاء كل Interval.
ms 1000	ثانية واحدة.
ms 2000	ثانيتان.
ms 5000	خمس ثوانٍ.
Start	يجعل `Enabled = true` فيبدأ الـ Timer.
End	يجعل `Enabled = false` فيتوقف الـ Timer.

الخلاصة التى أحب أن تحفظها

1. **Timer** هو Control يستخدم لتنفيذ كود بشكل متكرر كل فترة زمنية.
2. الفترة الزمنية تحددها الخاصية **Interval**، وتقاس بالـ **Millisecond**.
3. إذا كانت **Enabled = true** يبدأ الـ Timer فى العمل، وإذا كانت **false** يتوقف.
4. عند انتهاء كل **Interval** يتم تنفيذ الـ Event **Tick**.
5. الكود الذى تريد تنفيذه بشكل متكرر تضعه داخل **Tick**.
6. يمكن أن يحتوى البرنامج على أكثر من **Timer**، وكل Timer يعمل بإعداداته الخاصة.

لماذا Interval مصممة (مبرمجة) أساسًا لتستقبل قيمة بوحدة الـ Millisecond ؟

سؤال جميل جدًا، والإجابة ليست لأن الـ Millisecond "أفضل"، وإنما لأنها **أكثر عملية**.

تخيل لو كانت `Interval` بالثوانى فقط.

كنت ستكتب:

```C#
timer1.Interval = 1;
```

يعنى ثانية.

طيب لو أردت نصف ثانية؟

❌ لن تستطيع.

ولو أردت ربع ثانية؟

❌ لن تستطيع.

أما عندما تكون الوحدة هى **Millisecond**، يصبح عندك مرونة كبيرة.

مثلاً:

```C#
timer1.Interval = 100;   // 0.1 ثانية
```

```C#
timer1.Interval = 250;   // ربع ثانية
```

```C#
timer1.Interval = 500;   // نصف ثانية
```

```C#
timer1.Interval = 1000;  // ثانية
```

```C#
timer1.Interval = 1500;  // ثانية ونصف
```

```C#
timer1.Interval = 5000;  // خمس ثوانٍ
```

لاحظ أنك تستطيع تمثيل الثوانى وأجزاء الثوانى بسهولة.

لماذا لم يجعلوها بالـ Nanosecond؟

لأن:

```
1 Nanosecond
```

صغيرة جدًا.

فلو أردت ثانية واحدة ستكتب:

```
1,000,000,000
```

(مليار!)

وهذا غير عملي.

ولماذا ليست بالـ Second؟

لأنها كبيرة نسبيًا.

لن تستطيع كتابة:

```
ثانية 0.2
```

أو

```
ثانية 0.5
```

بسهولة لو كانت الخاصية تقبل أعدادًا صحيحة فقط.

إذن لماذا اختاروا الـ Millisecond؟

لأنها تعتبر **حلًا وسطًا**:

- أصغر من الثانية، فتسمح بدقة جيدة.
- أكبر بكثير من الـ Nanosecond، فلا تضطر لكتابة أرقام ضخمة.
- مناسبة لمعظم برامج واجهات المستخدم (Windows Forms).

لذلك اختار مصممو .NET أن تكون وحدة `Interval` هي **Millisecond**.

كيف يعمل الـ Timer داخليًا؟ وهل يعتمد على عداد (Counter) أم على آلية أخرى حتى يستدعى حدث Tick؟

ماذا يحدث عندما أكتب؟

```C#
timer1.Interval = 5000;
timer1.Enabled = true;
```

يحدث تقريبًا الآتى:

الخطوة الأولى

أنت تقول للـ **Timer**:

> **أريد تنفيذ شىء بعد 5000 Milliseconds.**

الخطوة الثانية

الـ **Timer** لا يبدأ العد بنفسه.

بل يطلب من **Windows** أن يضبط له مؤقتًا (Timer)، تمامًا مثلما تضبط منبهًا فى هاتفك.

يعنى كأنه يقول لـ Windows:

> **"اضبط لى منبهًا لمدة 5 ثوانٍ، وعندما ينتهى الوقت أخبرنى."**

الخطوة الثالثة

بعد أن يضبط **Windows** هذا "المنبه"...

لا يجلس الـ Timer يعد:

```
1...
2...
3...
```

بل ينتظر إشعارًا من **Windows**.

الخطوة الرابعة

بعد مرور الـ **5 ثوانٍ**...

يقول **Windows** للـ Timer:

> **"الوقت انتهى."**

الخطوة الخامسة

عندها يقوم الـ **Timer** باستدعاء الـ Event:

```C#
timer1_Tick(...)
```

فيتم تنفيذ الكود الموجود داخلها.

الخطوة السادسة

بعد انتهاء تنفيذ `Tick` ...

إذا كان:

```C#
timer1.Enabled == true
```

فإن الـ Timer يطلب من **Windows** مرة أخرى:

> **"اضبط لى منبهًا جديدًا لمدة 5 ثوانٍ."**

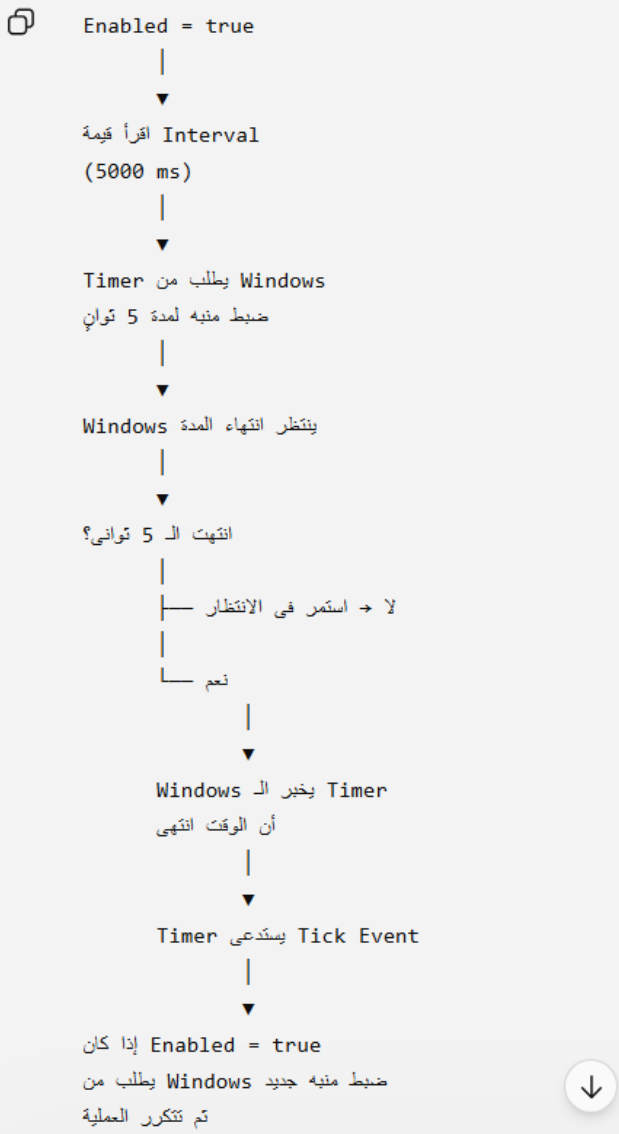
ثم ينتظر.

وعندما ينتهى الوقت...

يعيد **Windows** إخباره مرة أخرى.

وهكذا تستمر العملية.

ارسمها فى ذهنك هكذا



💡 **ملحوظة (للفهم فقط):**

هذا المثال بالمنبه ليس هو التنفيذ الحرفى داخل Windows، لكنه **أقرب تصور ذهنى صحيح** لفهم طريقة عمل `Timer` دون الدخول فى تفاصيل متقدمة عن نظام التشغيل.

NotifyIcon

NotifyIcon

يتكون الاسم من كلمتين:

```
Notify + Icon
```

Notify

ترجمتها:

> **إشعار** أو **تنبيه**.

Icon

ترجمتها:

> **أيقونة**.

إذن الترجمة الحرفية لـ:

```
NotifyIcon
```

هى:

> **أيقونة الإشعارات.**

ويمكن تبسيط معناها إلى:

> **كنترول يسمح بوضع أيقونة البرنامج داخل منطقة الإشعارات (System Tray)، ومن خلالها يمكن عرض إشعارات للمستخدم.**

ما هو الـ NotifyIcon؟

الـ **NotifyIcon** هو كنترول يستخدم لإظهار **أيقونة البرنامج** داخل منطقة الإشعارات الموجودة بجانب الساعة فى شريط المهام (**System Tray**).

ومن خلال هذا الكنترول يستطيع البرنامج أيضًا عرض رسالة إشعار تسمى:

```
Balloon Tip
```

كيف نضيف NotifyIcon إلى الفورم؟

من نافذة:

```
ToolBox
```

نبحث عن:

```
NotifyIcon
```

ثم نقوم بسحبه إلى الفورم.

بمجرد إضافته ستلاحظ أنه **لن يظهر داخل الفورم** مثل Button أو TextBox.

بل سيظهر أسفل الفورم فى منطقة الكنترولات غير المرئية (**Component Tray**).

وذلك لأنه **كنترول غير مرئى (Non-Visual Control)**.

ماذا يفعل NotifyIcon؟

وظيفته الأساسية هى:

- وضع أيقونة البرنامج داخل **System Tray**.
- إمكانية عرض إشعار للمستخدم (**Balloon Tip**).
- توفير أحداث (Events) للتعامل مع الضغط على الإشعار.

أول خطوة فى الكود

داخل الزر كتبنا:

```C#
notifyIcon1.Icon = SystemIcons.Application;
```

نشرح السطر

أولًا:

```C#
notifyIcon1
```

هو اسم كنترول الـ **NotifyIcon** الموجود على الفورم.

بعد ذلك:

```C#
Icon
```

وهى خاصية تحدد:

> **الأيقونة التى سيعرضها البرنامج داخل System Tray.**

أما:

```C#
SystemIcons.Application
```

فهى تعنى:

> استخدم أيقونة التطبيق الافتراضية الخاصة بويندوز.

وبذلك سيظهر لبرنامجك أيقونة داخل منطقة **System Tray**.

💡 معلومة إضافية

لا يقتصر الأمر على:

```C#
SystemIcons.Application
```

فالفئة:

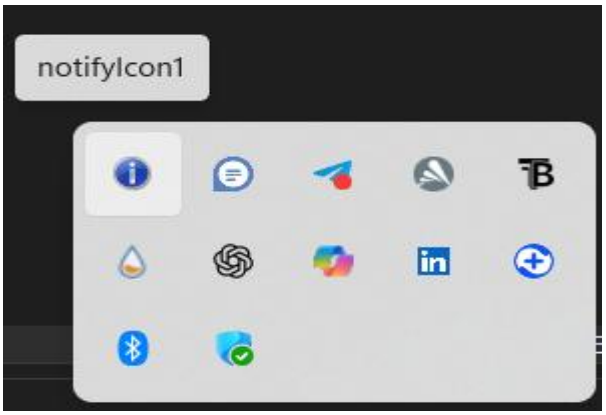
```C#
SystemIcons
```

تحتوى على مجموعة من الأيقونات الجاهزة التى يوفرها نظام ويندوز، ويمكن استخدامها مباشرة داخل الـ **NotifyIcon**.

ومن أشهرها:

الكود	الأيقونة	الاستخدام
`SystemIcons.Application`		أيقونة التطبيق الافتراضية.
`SystemIcons.Error`		تستخدم للدلالة على وجود خطأ.
`SystemIcons.Warning`		تستخدم للتحذير.
`SystemIcons.Information`		تستخدم للمعلومات.
`SystemIcons.Question`		تستخدم للاستفسار أو السؤال.
`SystemIcons.Shield`		تستخدم غالبًا مع الصلاحيات والأمان (UAC).

يمكنك تجربة كل واحدة منها، وستلاحظ أن الأيقونة التى تظهر فى **System Tray** تتغير تبعًا للأيقونة التى اخترتها.



خصائص الـ Balloon Tip

بعد تحديد الأيقونة الخاصة بالبرنامج، بدأنا نحدد شكل الإشعار الذى سيظهر للمستخدم.

أولًا

```C#
notifyIcon1.BalloonTipIcon = ToolTipIcon.Error;
```

نشرح السطر

أولًا:

```C#
BalloonTipIcon
```

وتتكون من ثلاث كلمات:

```
Balloon + Tip + Icon
```

Balloon

ترجمتها:

> **بالونة.**

وسميت بذلك لأن شكل الإشعار قديمًا كان يشبه البالونة التى تخرج من شخصيات الكرتون.

Tip

ترجمتها:

> **تلميح** أو **إشعار.**

Icon

ترجمتها:

> **أيقونة.**

إذن:

```
BalloonTipIcon
```

تعنى:

> **الأيقونة التى ستظهر داخل رسالة الإشعار.**

بعد ذلك:

```C#
ToolTipIcon.Error
```

تعنى:

> اعرض أيقونة خطأ داخل رسالة الإشعار.

مثل علامة:

❌

ويمكن أيضًا اختيار أنواع أخرى مثل:

```C#
ToolTipIcon.Info
```

لعرض أيقونة معلومات.

أو

```C#
ToolTipIcon.Warning
```

لعرض أيقونة تحذير.

ثانيًا

```C#
notifyIcon1.BalloonTipTitle = "This is a Title";
```

نشرح السطر

أولًا:

```C#
BalloonTipTitle
```

تتكون من:

```
Balloon + Tip + Title
```

Title

ترجمتها:

> **عنوان.**

إذن:

```
BalloonTipTitle
```

تعنى:

> **عنوان رسالة الإشعار.**

وفى المثال:

```C#
"This is a Title"
```

سيظهر كعنوان أعلى الرسالة.

ثالثًا

```C#
notifyIcon1.BalloonTipText = "This is a message";
```

نشرح السطر

أولًا:

```C#
BalloonTipText
```

Text

ترجمتها:

> **النص.**

إذن:

```
BalloonTipText
```

تعنى:

> **النص الذى سيظهر داخل رسالة الإشعار.**

وفى المثال:

```
#C
"This is a message"
```

سيظهر كمحتوى الرسالة.

رابعًا

```
#C
notifyIcon1.ShowBalloonTip(1000);
```

نشرح السطر

أولًا:

```
#C
Show
```

ترجمتها:

> **اعرض.**

ثم:

```
Balloon Tip
```

أى:

> **رسالة الإشعار.**

إذن:

```
ShowBalloonTip()
```

تعنى:

> **اعرض رسالة الإشعار.**

أما:

```C#
1000
```

فهى تمثل **مدة عرض الإشعار** بالملى ثانية (**Milliseconds**).

فى المثال كتب:

```
1000
```

أى:

> ثانية واحدة.

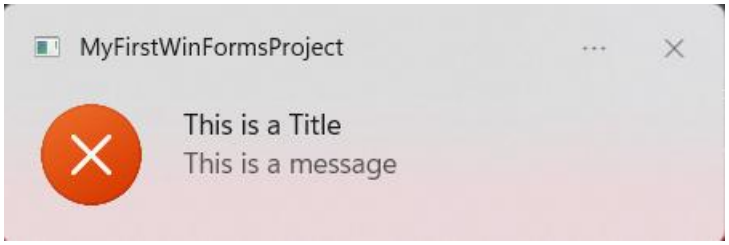
💡 معلومة إضافية (من الدرس)

ذكر أن ويندوز لا يسمح عادةً بأن تظل رسالة الـ **Balloon Tip** أقل من **10 ثوانٍ** أو أكثر من **30 ثانية**، لذلك قد يتجاهل القيمة التى تكتبها ويستخدم الحدود التى يحددها النظام.

النتيجة

بعد تنفيذ الأسطر السابقة سيظهر إشعار يحتوى على:

- أيقونة خطأ.
- عنوان.
- رسالة.
- ثم يختفى بعد مدة يحددها النظام.



أحداث (Events) الـ NotifyIcon

بعد أن تعلمنا كيف نظهر رسالة الإشعار (**Balloon Tip**)، ننتقل إلى الأحداث (**Events**) الخاصة بها.

الأحداث تسمح لنا بتنفيذ كود عند حدوث فعل معين من المستخدم.

الحدث

#C

```
private void notifyIcon1_BalloonTipClicked(object sender, EventArgs e)
{
    MessageBox.Show("BalloonTip Clicked :-)");
}
```

نشرح اسم الحدث

يتكون من:

```
notifyIcon1
```

وهو اسم الكنترول.

ثم:

```
BalloonTipClicked
```

Balloon

ترجمتها:

> **بالونة.**

Tip

ترجمتها:

> **إشعار.**

Clicked

ترجمتها:

> **تم الضغط عليه.**

إذن:

```
BalloonTipClicked
```

تعنى:

> **يحدث عند الضغط على رسالة الإشعار (Balloon Tip).**

ماذا يحدث داخل الحدث؟

الكود هو:

```C#
MessageBox.Show("BalloonTip Clicked :-)");
```

فعندما يضغط المستخدم على رسالة الإشعار...

سيظهر له:

```
BalloonTip Clicked :-)
```

داخل MessageBox.

إذن وظيفة هذا الحدث هى:

> **تنفيذ كود عند الضغط على رسالة الإشعار.**

💡 معلومة إضافية

ذكر فى الدرس أن هذا الحدث مفيد جدًا.

فبدلاً من إظهار MessageBox...

يمكنك مثلاً:

- فتح نافذة البرنامج.
- فتح شاشة الطلبات.
- فتح البريد الإلكترونى.
- تنفيذ أى عملية تريدها بعد ضغط المستخدم على الإشعار.

أهم خصائص الـ NotifyIcon التى ذكرها الدرس

الخاصية	وظيفتها
`Icon`	تحدد أيقونة البرنامج داخل الـ System Tray.
`BalloonTipIcon`	تحدد الأيقونة التى تظهر داخل رسالة الإشعار.
`BalloonTipTitle`	تحدد عنوان رسالة الإشعار.
`BalloonTipText`	تحدد نص رسالة الإشعار.
`()ShowBalloonTip`	تعرض رسالة الإشعار.

أهم الأحداث (Events)

الحدث	يحدث عند
`BalloonTipClicked`	الضغط على رسالة الإشعار.

الخلاصة

الـ **NotifyIcon** هو كنترول يستخدم لإظهار **أيقونة البرنامج داخل منطقة System Tray**.

ومن خلاله يمكن:

- تحديد أيقونة البرنامج.
- عرض رسالة إشعار (**Balloon Tip**).
- تحديد عنوان الرسالة ونصها وأيقونتها.
- تنفيذ كود عند الضغط على الرسالة.

💡 ملخص الفكرة

احفظ العلاقة بهذا الشكل:

```
البرنامج
    |
    ▼
NotifyIcon
(أيقونة البرنامج فى System Tray)
    |
    ▼
ShowBalloonTip()
    |
    ▼
Balloon Tip
(رسالة الإشعار)
```

أى أن:

- **NotifyIcon** تمثل البرنامج داخل **System Tray**.
- **Balloon Tip** هى رسالة مؤقتة يعرضها الـ **NotifyIcon** لإبلاغ المستخدم بحدث معين.

س: وضح وظيفةNotifyIcon ، وما هى علاقة BalloonTip به، ولماذا لا يمكن استخدام BalloonTip بمفردها؟

أولًا: لماذا اسمه NotifyIcon؟

قسم الاسم.

```
Notify + Icon
```

Notify

ترجمتها:

> **إشعار** أو **تنبيه**.

Icon

ترجمتها:

> **أيقونة**.

إذن الترجمة الحرفية:

> **أيقونة الإشعارات**.

لاحظ أنه **لم يسمها Microsoft BalloonTip**.

لأن الكنترول **ليس وظيفته الأساسية الرسالة**.

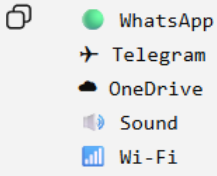
وظيفته الأساسية:

> **وضع أيقونة البرنامج داخل منطقة الإشعارات (System Tray)**.

طيب ما هى الـ System Tray؟

هى المنطقة الموجودة بجانب الساعة.

مثل:



كل هذه اسمها:

```
System Tray
```

لماذا وضع ويندوز هذه المنطقة أصلاً؟

تخيل أنك تعمل على الكمبيوتر.

وفاتح:

- Visual Studio
- Chrome
- Word

وفى نفس الوقت يوجد برنامج:

```
Telegram
```

هل تريد أن يظل Telegram فاتحًا أمامك؟

لا.

لكنك تريده أن:

- يستقبل الرسائل.
- يخبرك إذا جاءتك رسالة.

إذن أين يضع نفسه؟

ويندوز قال:

> "سأعمل مكانًا صغيرًا للبرامج التى تعمل فى الخلفية."

وهذا المكان هو:

```
System Tray
```

الآن جاء دور NotifyIcon

البرنامج يقول:

> "أنا أريد أن يكون لى مكان داخل الـ System Tray."

فيستخدم:

```
NotifyIcon
```

إذن وظيفة NotifyIcon هى:

> **تمثيل البرنامج داخل الـ System Tray بواسطة أيقونة.**

مثال من الواقع

تخيل أنك داخل مطار.

وكل شركة طيران لها مكتب.

وأمام كل مكتب توجد **شاشة إلكترونية صغيرة** تحمل شعار الشركة.

مثل:

```
✈ EgyptAir

Emirates

Qatar Airways
```

هذه الشاشة لا تقوم بإبلاغك بأى شىء فى البداية.

هى فقط:

> **تخبرك أن هذا المكتب تابع لهذه الشركة.**

الـ **NotifyIcon** تشبه هذه الشاشة.

فمثلاً:

```
WhatsApp
```

هى ليست البرنامج نفسه.

ولكنها:

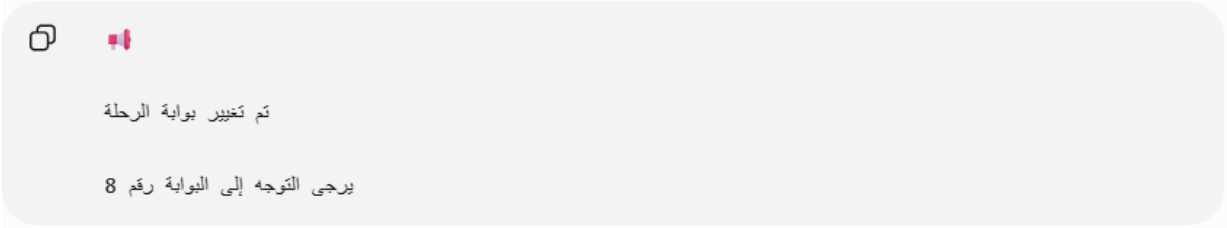
> **تمثل البرنامج وتدل على أنه يعمل.**

طيب أين تأتى BalloonTip؟

تخيل أن شركة الطيران تريد أن تبلغ المسافرين بخبر مهم.

بدلًا من تغيير الشاشة...

تظهر رسالة مؤقتة بجوارها تقول:



هذه الرسالة هى:

```
BalloonTip
```

لاحظ شيئًا مهمًا...

الشاشة ما زالت موجودة كما هى.

لكنها استخدمت لإظهار رسالة مؤقتة للمسافرين.

إذن العلاقة أصبحت:

```
NotifyIcon
        |
        ▼
يمثل البرنامج
داخل System Tray
        |
        ▼
إذا أراد البرنامج إرسال إشعار
        |
        ▼
يظهر BalloonTip
```

لماذا Microsoft وضعت BalloonTip داخل NotifyIcon؟

لأنها قالت:

> **الـ BalloonTip لا تعيش وحدها.**

فهي دائمًا مرتبطة بـ NotifyIcon.

ولذلك لم تنشئ كنترول اسمه:

```
#C
BalloonTip
```

بل جعلت خصائصها داخل:

```
#C
notifyIcon1
```

مثل:

```
#C
notifyIcon1.BalloonTipTitle
notifyIcon1.BalloonTipText
notifyIcon1.BalloonTipIcon
notifyIcon1.ShowBalloonTip()
```

لماذا كنت تشعر أن العلاقة غريبة؟

لأنك كنت تقارنها بكنترولات مثل:

```
TextBox
```

الاسم:

```
TextBox
```

وظيفته:

> كتابة نص.

كل شيء منطقي.

أما هنا:

```
NotifyIcon
```

ثم تجد بداخله:

```
BalloonTip
```

فتقول:

> "هو اسمه NotifyIcon ولا BalloonTip؟"

والإجابة:

اسمه NotifyIcon لأن:

> **هذا هو الكنترول.**

أما BalloonTip فهي:

> **ميزة (Feature)** يقدمها هذا الكنترول.

تمامًا مثل السيارة.

السيارة اسمها:

```
Car
```

لكن داخلها:

- تكييف.
- راديو.
- شاشة.

هل نسميها:

```
AirCondition Car
```

لا.

ما زالت اسمها:

```
Car
```

والتكييف مجرد ميزة داخلها.

لماذا لا يوجد BalloonTip بدون NotifyIcon؟

اسأل نفسك سؤالًا.

لو Microsoft عملت كنترول:

```C#
BalloonTip
```

أين ستظهر؟

فى منتصف الشاشة؟

فى أعلى الشاشة؟

فى أسفل الشاشة؟

لا يوجد مكان ثابت.

لكن عندما ربطتها بـ NotifyIcon أصبح معروفًا:

> **الإشعار سيعرض بواسطة الـ NotifyIcon الموجودة فى الـ System Tray.**

الرسم الذى أريدك أن تحفظه

```
برنامج يعمل
      |
      ▼
NotifyIcon
(أيقونة البرنامج)
      |
      ▼
تظهر فى الـ System Tray
      |
      ▼
حدث معين
(رسالة - Timer - خطأ - تحميل...)
      |
      ▼
ShowBalloonTip()
      |
      ▼
BalloonTip
```

مثال كامل من الحياة

تخيل أن عندك تطبيق اسمه:

```
Food Delivery
```

أول ما يشغل المستخدم البرنامج.

يظهر فى الـ System Tray:

```
🍔
```

هذه هى:

```
NotifyIcon
```

بعد خمس دقائق...

وصل طلب جديد.

البرنامج ينفذ:

```#C
notifyIcon1.ShowBalloonTip(...);
```

فيطلب من ويندوز عرض إشعار للمستخدم.

فيظهر للمستخدم:

```
🍔
_____________________
طلب جديد

العميل: أحمد

اضغط لفتح الطلب
_____________________
```

هذه هى:

```
BalloonTip
```

بعد أن يقرأها المستخدم...

تختفى الرسالة.

أما:

```
🍔
```

فتظل موجودة لأنها تمثل البرنامج داخل الـ **System Tray**.

الخلاصة التى أريدك أن تحفظها

NotifyIcon	BalloonTip
أيقونة البرنامج	رسالة مؤقتة
تظهر فى System Tray	تظهر بواسطة الـ NotifyIcon
تمثل البرنامج	تنقل إشعارًا للمستخدم
قد تظل موجودة طوال تشغيل البرنامج	تظهر لعدة ثوانٍ ثم تختفى
هى الكنترول	هى ميزة يقدمها الكنترول

💡 **وأعتقد أن أكثر نقطة كانت تلخبطك هى أنك كنت تعتبر الـ BalloonTip والـ NotifyIcon شيئًا واحدًا.**

لكن الحقيقة:

- **NotifyIcon = الأيقونة الدائمة التى تمثل البرنامج داخل الـ System Tray.**
- **BalloonTip = الرسالة المؤقتة التى يعرضها الـ NotifyIcon لإبلاغ المستخدم بحدث معين.**

ولهذا السبب وضعت Microsoft خصائص الـ **BalloonTip** داخل كنترول **NotifyIcon**، لأن الـ **BalloonTip لا تعمل بمفردها**، وإنما تعتمد على الـ **NotifyIcon** لعرض الإشعارات.

TreeView and ImageList

للتفاصيل اكتر شاهد 5 فديوهات



ليه عشان اضيف صور فى الترى فيو لازم ادخل جواها ImageList

؟مش كان الافضل انحط الصوره مباشره من غيرها ؟

الإجابة المختصرة:

> **لأن الـ TreeView مش بيخزن الصور بنفسه، هو بيخزن فقط رقم الصورة (Index) أو اسمها (Key)، أما الصور نفسها فتتخزن فى ImageList.**

خلينى أوضحها.

لو مفيش ImageList

تخيل عندك شجرة فيها 1000 Node.

ولو كل Node خزنت صورة بداخلها مباشرة:

```
TreeNode 1
├── Text
└── Image

TreeNode 2
├── Text
└── Image

TreeNode 3
├── Text
└── Image
```

لو كلهم بيستخدموا نفس صورة المجلد 📁، هتبقى الصورة مكررة 1000 مرة فى الذاكرة.

وده إهدار كبير.

علشان كده عملوا ImageList

تخيلها كأنها مكتبة للصور.

```
ImageList
|
├── [0] Folder.png
├── [1] File.png
├── [2] CSharp.png
└── [3] Python.png
```

والـ TreeNode متخزنش الصورة نفسها.

هى بس تقول:

```
أنا استخدم الصورة رقم 0
```

أو

```
ImageIndex = 0
```

فيبقى الشكل

```
ImageList
|
├── [0] Folder
└── [1] File


TreeView
|
├── Programming   ----> ImageIndex = 0
├── C#            ----> ImageIndex = 1
└── C++           ----> ImageIndex = 1
```

لاحظ إن صورة الـ File موجودة **مرة واحدة فقط** فى الـ ImageList، وكل الـ Nodes تشير إليها.

لو مفهمتش النقطه دى كويس

تخيل إن Microsoft عملت TreeView بالشكل ده

```C#
treeNode1.Image = folder.png;
treeNode2.Image = folder.png;
treeNode3.Image = folder.png;
```

السؤال:

الصورة `folder.png` دى اتكررت كام مرة؟

لو عندك 1000 Node، يبقى الصورة اتخزنت 1000 مرة.

وده غباء فى التصميم.

طيب يعملوا إيه؟

قالوا:

بدل ما كل Node تشيل الصورة...

نعمل مخزن واحد للصور.

اسمه:

```
ImageList
```

يحطوا فيه:

```
0 --> Folder
1 --> File
2 --> C#
```

وبعدين الـ Node تقول:

> "أنا عايز الصورة رقم 0"

بس.

خلينى أغير المثال

تخيل عندك مدرسة.

هل كل طالب هيشيل ماكينة تصوير معاه؟

ولا المدرسة فيها ماكينة تصوير واحدة وكل الطلاب يستخدموها؟

أكيد واحدة.

الـ ImageList هى ماكينة التصوير.

أو مثال أقرب

أنت عامل مشروع العيادة.

تخيل كل مريض يجيب معاه دكتور.

مستحيل.

الصح:

```
Clinic
|
├── Doctors
|     ├── Ahmed
|     ├── Sara
|     └── Omar
|
└── Patients
      |
      ├── Patient1 ---> Doctor = Ahmed
      ├── Patient2 ---> Doctor = Ahmed
      └── Patient3 ---> Doctor = Sara
```

لاحظ:

الدكتور أحمد موجود **مرة واحدة**.

لكن 100 مريض ممكن يشيروا إليه.

نفس الفكرة هنا.

```
ImageList
|
├── Folder
├── File
└── C#
```

والـ Nodes:

```
Programming ---> Folder
Methods -----> File
Arrays ------> File
Classes -----> File
```

كلهم بيستخدموا نفس الصورة.

طب سؤال

لو عندك **Node واحدة فقط** فى البرنامج كله...

هل كان ممكن يحطوا الصورة جواها مباشرة؟

الإجابة: نعم، كان ممكن.

لكن WinForms اتصممت علشان تخدم برامج فيها آلاف الـ Nodes، فاختاروا تصميمًا يوفر الذاكرة ويعيد استخدام الصور.

ليه Microsoft عملتها كده؟

لأسباب كتير:

1- توفير الذاكرة

بدل ما تخزن نفس الصورة 500 مرة.

2- سهولة تغيير الصور

لو غيرت الصورة فى الـ ImageList:

```
Folder.png
⬇
NewFolder.png
```

كل الـ Nodes اللى كانت بتستخدمها هتتغير تلقائى.

3- إعادة الاستخدام

ممكن نفس الـ ImageList تستخدمها مع:

- TreeView
- ListView
- وغيرها من الكنترولات.

الجملة اللى قلتها دى صحيحة جدًا، وأحب أصيغها بشكل أدق شوية:

> **الـ TreeView هو اللى بيحتوى على الـ ImageList، وكل الـ TreeNode الموجودة داخله بتستخدم الصور الموجودة فى الـ ImageList ده.**

يعنى الرسم ده صح:

```
TreeView
|
├── ImageList
|       |
|       ├── [0] Folder
|       ├── [1] File
|       └── [2] C#
|
└── Nodes
        |
        ├── Programming (ImageIndex = 0)
        ├── C#          (ImageIndex = 2)
        └── Methods     (ImageIndex = 1)
```

لاحظ:

- ✅ الـ **TreeView** هو اللى ماسك الـ **ImageList**.
- ✅ كل **TreeNode** بتقول: "أنا عايزة الصورة رقم كذا."
- ✅ الـ TreeNode **مش بتخزن الصورة**، ولا عندها ImageList خاصة بيها.
- ✅ الـ TreeView هو اللى يجيب الصورة من الـ ImageList ويعرضها.

ProgressBar

ProgressBar

الترجمة

Progress Bar = شريط التقدم.

التعريف

هو كنترول يستخدم لعرض نسبة تقدم عملية معينة للمستخدم، مثل تحميل ملف أو نسخ بيانات أو تثبيت برنامج.

بدلاً من أن يرى المستخدم البرنامج متوقفاً، يشاهد شريطاً يتحرك تدريجياً حتى انتهاء العملية.

الخصائص الأساسية

1- Minimum

الترجمة

Minimum = الحد الأدنى

تمثل أقل قيمة يمكن أن يصل إليها الـ ProgressBar.

مثال:

```C#
progressBar1.Minimum = 0;
```

2- Maximum

الترجمة

Maximum = الحد الأقصى

تمثل أكبر قيمة يمكن أن يصل إليها الـ ProgressBar.

مثال:

```C#
progressBar1.Maximum = 100;
```

3- Value

الترجمة

Value = القيمة الحالية

وهى القيمة التى يمثلها الشريط الآن.

مثال:

```C#
progressBar1.Value = 40;
```

تصور ذهنى

تخيل أن عندك كوب ماء.

```
Minimum = 0

Maximum = 100

Value = 40
```

أنت الآن ملأت الكوب حتى 40%.

كلما زادت قيمة Value زاد امتلاء الكوب.

أو تخيله هكذا:

```
0%      25%      50%      75%      100%

□□□□□□□□□□

████□□□□□□

█████□□□□□

███████□□□

██████████
```

فالـ ProgressBar لا يعرف شيئاً عن النسبة المئوية.

هو فقط يعرف:

```
Minimum

Maximum

Value
```

ومنهم يحسب مقدار الجزء الأخضر الذى سيرسمه.

```
private void button1_Click(object sender, EventArgs e)
{
    progressBar1.Value = 0;
    progressBar1.Maximum = 100;
    progressBar1.Minimum = 0;

    for (int i = 1; i <= 10; i++)
    {
        if (progressBar1.Value < progressBar1.Maximum)
        {
            Thread.Sleep(500);
            progressBar1.Value += 10;

            label1.Text = (((float)progressBar1.Value / progressBar1.Maximum) * 100) + "%";

            progressBar1.Refresh();
            label1.Refresh();
        }
        else
        {
            button1.Enabled = false;
        }
    }
```

شرح الكود

```#C
progressBar1.Value = 0;
```

نبدأ من الصفر.

```#C
progressBar1.Maximum = 100;
```

أكبر قيمة للشريط هى 100.

```#C
progressBar1.Minimum = 0;
```

أقل قيمة للشريط هى صفر.

بعد ذلك يبدأ تنفيذ اللوب.

```#C
for(int i=1;i<=10;i++)
```

ستتكرر عشر مرات.

فى كل مرة يتم تنفيذ الشرط.

```C#
if(progressBar1.Value<progressBar1.Maximum)
```

إذا كانت القيمة الحالية أقل من 100.

استمر.

وإلا لا تكمل.

بعد ذلك

```C#
Thread.Sleep(500);
```

Thread.Sleep

الترجمة

Sleep = نوم

Thread = الخيط التنفيذى

أى:

إيقاف الخيط التنفيذى مؤقتاً.

وظيفتها

إيقاف تنفيذ الكود لمدة محددة.

فى مثالنا

```C#
Thread.Sleep(500);
```

يتوقف البرنامج لمدة نصف ثانية.

لاحظ جيداً

Thread.Sleep **لا تقوم برسم الشاشة**

ولا تقوم بعمل Refresh

ولا تحدث الكنترولات.

وظيفتها الوحيدة هى:

إيقاف تنفيذ الكود.

بعد ذلك

```C#
progressBar1.Value+=10;
```

يزيد الشريط عشر درجات.

فتصبح

```
10

20

30

40

...

100
```

بعد ذلك

```C#
label1.Text=
(((float)progressBar1.Value/progressBar1.Maximum)*100)+"%";
```

يحسب النسبة المئوية.

مثلاً

```
Value=40

Maximum=100

40÷100=0.4

0.4×100=40%
```

ثم يعرض

```
40%
```

داخل الـ Label.

Refresh

الترجمة

Refresh = إعادة تحديث / إعادة رسم

وظيفتها

إجبار الكنترول على إعادة رسم نفسه فوراً.

لماذا نستخدم Refresh ؟

لفهم ذلك يجب أن نعرف كيف يعمل WinForms.

كيف يعمل WinForms ؟

عندما تضغط على الزر يحدث الآتى:

```
ضغط المستخدم على الزر

↓

بدأ تنفيذ Button_Click

↓

ينفذ السطر الأول

↓

ينفذ السطر الثانى

↓

ينفذ اللوب بالكامل

↓

ينتهى الحدث

↓

بعد انتهاء الحدث

يقوم Windows بإعادة رسم الكنترولات

↓

تظهر النتيجة للمستخدم
```

لاحظ شيئاً مهماً جداً.

الويندوز لا ينفذ الكود ويرسم الشاشة فى نفس الوقت.

بل يعمل بالترتيب.

```
أولاً

تنفيذ الكود

↓

ثانياً

رسم الشاشة
```

ولهذا السبب قد يتغير

```#C
label1.Text
```

عشر مرات.

لكن المستخدم لا يرى إلا آخر قيمة.

لأن الويندوز لم يبدأ الرسم إلا بعد انتهاء الحدث.

إذن لماذا Refresh ؟

عندما نكتب

```#C
label1.Refresh();
```

فنحن نقول للويندوز:

> لا تنتظر حتى ينتهى الحدث.

> ارسم الكنترول الآن.

ثم أكمل تنفيذ باقى الكود.

ماذا اكتشفنا أثناء التجربة ؟

بعد التجربة العملية اكتشفنا شيئاً مهماً.

قمنا بحذف

```
#C
progressBar1.Refresh();
```

ومع ذلك ظل الـ ProgressBar يتحرك بصورة طبيعية.

واكتشفنا أن:

> بعض الكنترولات تقوم بإعادة رسم نفسها تلقائياً عند تغيير بعض الخصائص.

مثل

```
#C
progressBar1.Value
```

فالـ ProgressBar داخلياً يطلب إعادة رسم نفسه عند تغيير Value.

لذلك لم نحتج إلى Refresh فى هذا المثال.

أما بالنسبة للـ Label

عندما حذفنا

```
#C
label1.Refresh();
```

لاحظنا أن النص لم يتغير تدريجياً.

بل ظهر

```
100%
```

مرة واحدة بعد انتهاء اللوب.

وذلك لأن الـ Label لم يقم بإعادة رسم نفسه أثناء تنفيذ الحدث.

ملاحظة مهمة

ليس كل الكنترولات تتصرف بنفس الطريقة.

بعض الكنترولات تعيد رسم نفسها تلقائياً.

وبعضها يحتاج إلى استدعاء Refresh حتى يظهر التغيير أثناء تنفيذ الكود.

ولهذا السبب لا يجب أن نفترض أن جميع الكنترولات تعمل بنفس الأسلوب.

ملخص الدرس

✅ تعلمنا معنى ProgressBar.

✅ فهمنا الفرق بين Minimum و Maximum و Value.

✅ عرفنا أن Value هى التى تحدد مقدار امتلاء الشريط.

✅ تعلمنا وظيفة Thread.Sleep وهى إيقاف تنفيذ الكود فقط.

✅ فهمنا أن WinForms لا ينفذ الكود ويرسم الشاشة فى نفس الوقت، بل ينفذ الكود أولاً ثم يعيد رسم الشاشة بعد انتهاء الحدث.

✅ عرفنا أن Refresh تجبر الكنترول على إعادة رسم نفسه فوراً بدلاً من انتظار انتهاء الحدث.

✅ اكتشفنا بالتجربة أن بعض الكنترولات مثل **ProgressBar** تعيد رسم نفسها تلقائياً عند تغيير خصائص معينة مثل `Value`.

✅ واكتشفنا أيضاً أن **Label** فى هذا المثال احتاج إلى `()Refresh` حتى يظهر تحديث النص أثناء تنفيذ اللوب.

ListView

شرح من اسم الكنترول حتى زرار Add

https://drive.google.com/file/d/1t35W4t_BbZTvK6MFuK5lykg5mbP5ALFm/view?usp=drive_link

شرح Remove() and Fill Random()

https://drive.google.com/file/d/18hYbDxad34BiimKvtD1U_9FIvDHNdNu1/view?usp=drive_link

هذا الملخص خاص بهذا الجزء

ملخص المراجعة النهائي

النقطة	معناها باختصار
`listView1.Items`	كل الصفوف الموجودة في الـ ListView
`listView1.SelectedItems`	الصفوف اللي المستخدم محددها
`SelectedItems[0]`	أول صف **من الصفوف المحددة**
`Items[0]`	أول صف **من كل الصفوف الموجودة**
`(...)Items.Remove`	يحذف الـ `ListViewItem` بالكامل
`Items.Remove(SelectedItems[0])`	يحذف أول صف محدد بالكامل
لو محدد 5 صفوف	`[0]` يحذف **واحد فقط**، مش الخمسة
`for` في Fill Random	يعمل 10 صفوف تلقائيًا
`()i.ToString`	يحول رقم `i` إلى String
`i % 2 == 0`	يحدد هل `i` زوجي
`ImageIndex`	يحدد صورة الصف
`Person" + i"`	ينتج `Person1` ، `Person2` ...
`(...)SubItems.Add`	يضيف اسم الشخص في الصف
`Items.Add(item)`	يدخل الصف النهائي داخل الـ ListView

احفظ الجملة دي:

`Items` = كل الصفوف، `SelectedItems` = الصفوف المحددة، و `[0]` = أول عنصر في الـ Collection اللي قبلها.

شرح خاصيتيين AcceptButton و CancelButton ؟

https://drive.google.com/file/d/1H_0qCahGFr-h3M8hcIzVJNd5li2Y8qgJ/view?usp=drive_link

🔥 الخريطة الكاملة في شكل هرمي واحد

```
LISTVIEW
│
├── List = مجموعة / قائمة العناصر
│
├── View = طريقة عرض العناصر
│   │
│   ├── Details     → جدول
│   ├── LargeIcon   → أيقونات كبيرة
│   ├── SmallIcon   → أيقونات صغيرة
│   ├── List        → قائمة
│   └── Tile        → بلاطات
│
├── Columns
│   └── أعمدة الجدول
│       ├── ID
│       └── Name
│
├── Items
│   └── Collection = كل الصفوف
│       │
│       ├── [0] → صف
│       ├── [1] → صف
│       └── ...
│
│       └── ListViewItem = صف واحد
│           │
│           ├── Text
│           │   └── أول خلية
│           │
│           ├── SubItems
│           │   └── Collection = باقي خلايا الصف
│           │       ├── [1] → الخلية الثانية
│           │       ├── [2] → الخلية الثالثة
│           │       └── ...
```

```
|           |
|           └── ImageIndex
|               └── رقم الصورة من ImageList
|
├── SelectedItems
|   └── Collection = الصفوف المحددة
|       ├── [0] → أول صف محدد
|       ├── [1] → ثاني صف محدد
|       └── ...
|
├── ADD
|   |
|   ├── Validation
|   |
|   ├── new ListViewItem()
|   |
|   ├── item.Text = ID
|   |       └── أول خلية
|   |
|   ├── item.SubItems.Add(Name)
|   |       └── الخلية الثانية
|   |
|   ├── item.ImageIndex
|   |       └── تحديد الصورة
|   |
|   └── listView1.Items.Add(item)
|           └── إضافة الصف للجدول
|
├── REMOVE
|   |
|   ├── SelectedItems
|   |
|   ├── [0]
|   |    └── أول صف محدد
```

```
│   │
│   └── Items.Remove(...)
│         └── حذف الصف بالكامل
│
├── FILL RANDOM
│   │
│   ├── for 1 → 10
│   ├── i.ToString()
│   │     └── أول خلية
│   ├── "Person" + i
│   │     └── SubItems → الخلية الثانية
│   ├── i % 2
│   │     └── زوجي / فردي
│   ├── ImageIndex
│   └── Items.Add(item)
│
└── DOUBLE CLICK
    │
    ├── SelectedItems[0]
    │     └── أول صف محدد
    │
    ├── .Text
    │     └── أول خلية
    │
    └── MessageBox.Show()
          └── عرض القيمة
```

ErrorProvider

درس ErrorProvider في C# Windows Forms

https://drive.google.com/file/d/1aI17PG_-3dmlEnYHtCz1Bb0fYBqWNXkV/view?usp=drive_link

كيف يعمل e.Cancel داخل Event الـ Validating، وما تأثيره على تسلسل الأحداث؟

https://drive.google.com/file/d/1F4HuiGOpEGjV0bLM6Y4VeYabW6Jfx_FU/view?usp=drive_link

AutoValidate في C# Windows Forms

https://drive.google.com/file/d/1_svpYEv4aSPw8YhD6yfS5kmj_44KYyKv/view?usp=drive_link

TrackBar

درس TrackBar في C# Windows Forms

https://drive.google.com/file/d/1kV-POwa9VmQKurw1alm28IJGdjmSCnnF/view?usp=drive_link

تكمله البروبرتيز

درس TickStyle و TickFrequency في C# Windows Forms

https://drive.google.com/file/d/1r7MgsJ3AkyKXyVw5_CB-wsnqvVKh_Ba8/view?usp=drive_link

NumericUpDown

درس NumericUpDown في C# Windows Forms

https://drive.google.com/file/d/11fL8404IM75vlcb6EcYUm0GFAIpaLUpv/view?usp=drive_link

GroupBox vs Panel

https://drive.google.com/file/d/1gITrxlxvCJaufBeRA6hHNfjftpDuufML/view?usp=drive_link

درس ColorDialog و FontDialog الشامل

https://drive.google.com/file/d/1C_yuyjKpEymGB1j0XgqQVVnNJoo1-1-N/view?usp=sharing

درس SaveFileDialog — من اسم الـControl لحد إنشاء الملف

https://drive.google.com/file/d/1yqZSO1xyeb3j3yw8ibIOM2uxm50adXBb/view?usp=sharing

درس OpenFileDialog — من اسم الـControl لحد اختيار ملف واحد أو عدة ملفات

https://drive.google.com/file/d/1tvQaenrK0NEkEjtSfuPEpNCpmAdpad-F/view?usp=sharing

شرح FolderBrowserDialog في الـ Windows Forms

https://drive.google.com/file/d/1gONYiGjdXoFHdPAVHlWbYYjM6XVoYKiw/view?usp=sharing

درسMenuStripالـ فى Windows Forms

https://drive.google.com/file/d/14mie6hYiByvsFjAtaXDd3vQfAqm3Vww_/view?usp=sharing

درس ContextMenuStripفي الـWindows Forms

https://drive.google.com/file/d/1PeUpgO0DAHEDLewSO4cl5bXerOUuGiBe/view?usp=sharing